

VECTOR

APL2000 at Naples Beach ...

- | | |
|--------------------------------|-----|
| • Peelle on Josephus in J | 46 |
| • APL2000 Conference Reports | 66 |
| • FM Waid on Webservices | 71 |
| • Coxe on Snowflake Automata | 113 |
| • Jouko on Rain and PowerPoint | 130 |
| • McDonnell on Black-Scholes | 137 |



*The Journal of the
British APL Association*

ISSN 0955-1433
www.vector.org.uk
Vol.19 No.3 January 2003

A Specialist Group of the British Computer Society

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (b/w or colour prints welcome). Unless otherwise specified, each item will be considered for publication as a personal statement by the author. The Editor accepts no responsibility for the contents of sustaining members' news, or advertising.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file on an IBM PC compatible diskette or via email. APL code can be accepted in workspaces from I-APL, APL+Win, IBM APL2/PC or Dyalog APL/W, or in documents from Windows Write (use the APL2741 TrueType font, available free from Vector Production), and MS Word (any version).

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the Editor of your intention to re-use material from VECTOR.

Membership Rates 2002-2003

Category	Fee	Vectors	Passes
UK Private	£20	1	1
Overseas Private	£22	1	1
(Supplement for Airmail, not needed for Europe)	£4		
UK Corporate Membership	£100	5	5
Overseas Corporate	£110	5	
Sustaining	£500	10	5
Non-voting Member (Student, OAP, unemployed)	£10	1	1

The membership year normally runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa, Mastercard or JCB, at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready format (A4 or A5) with a 20mm blank border after reduction. Illustrations should be photographs (b/w or colour prints) or line drawings. Rates (excl VAT) are £250 per full page, £125 for half-page or less (there is a £75 surcharge per page if spot colour is required).

Deadlines for bookings and copy are given under the Quick Reference Diary. Advertisements should be booked with, and sent to Gill Smith, Vector Production, Brook House, Gilling East, YORK YO62 4JJ. Tel: 01439-788385.

Email: apl385@compuserve.com.

Contents

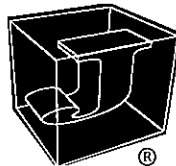
		Page
Guest Editorial: Unicode Revisited	Adrian Smith	3
APL NEWS		
Quick Reference Diary		3
British APL Association News		
News from Sustaining Members		10
APL Product Guide	Gill Smith	14
The Education Vector		
Zark Newsletter Extracts	edited by Jon Sandles	29
Crossword		37
J-ottings 35: How to Do Things with Words	Norman Thomson	40
A Tutorial in J Programming: The Second-Order Josephus Sequence	Howard Peelle	46
CONFERENCE REPORT – APL2000		
Notes and Pictures	Adrian Smith	66
APL and Web Services	FM Waid	71
GENERAL ARTICLES		
Trip Report: One Way From APL to J and K	David Ness	79
Paritons and Cognitons		
Towards a New Theory of Information	The late Gérard A. Langlet	93
Boolean Hexagonal Automata	Angela Coxe and Cliff Reiter	113
Provisional Unicode Reference	Adrian Smith	122
TECHNICAL SECTION		
Technical Correspondence		128
Rain-graphs to PowerPoint-land via OLE	Jouko Kangasniemi	130
At Play with J: Beware Scholes!	Gene McDonnell	137
Index to Advertisers		143

**Download
from the
web for
free!**

Jsoftware™ Version 5.01

- ♦ **Session Manager, window driver, IDE and GUI support in Unix and Linux**
- ♦ **GUI event support extended to more closely match VB and Java**
- ♦ **New Debug facilities**
- ♦ **New Documentation**
- ♦ **Performance Improvements**

**Jsoftware Inc.
P.O. Box 330
Excelsior, MN 55331
Info@jsoftware.com
Tel (952) 470-7345
Fax (952) 470-9202**



The programming language
ideal for complex analytics
and data manipulation.

www.jsoftware.com

Guest Editorial: Unicode Revisited

by Adrian Smith

*One Font to rule them all
One Font to find them
One Font to bring them all
And in the darkness bind them.*

Here is something that seemed like a distant theoretical dream, which has suddenly become very urgent, which is why I asked Stefano if I could hijack the editorial slot for this Vector.

In the past, the different APL vendors have solved in different ways the problem of mapping □AV to the small number of available ASCII characters, which has meant that APL code often cannot not be easily exchanged between interpreters, let alone represented in email messages or newsgroups.

Surely, Unicode (although a distant dream) would be the final definitive answer to the cross-mapping between interpreters? Indeed, it should eventually enable APL code to be traded as easily in newsgroups as J and K code. Now that Windows Notepad fully supports Unicode, one can see newsgroup readers following along, and email software like Outlook should make it as easy to use APL in email messages as it already is to use Korean and Cyrillic.

But first we must all agree on where the symbols go in the font.

Can this still be an issue? IBM have worked hard behind the scenes to get APL firmly embedded in the unicode standard, and there is the official **Unicode Reference** available for us all to read. (See www.unicode.org/charts/PDF/U2300.pdf for a typical section.) So what's the problem?

In the rush to ship the beta of .NET (and hence of Pocket APL) Dyadic mapped several of the APL symbols to the *Greek* section of the font, rather than the (visually identical) symbols in the *Miscellaneous Technical* section. To my shame, I did not pick this up until I had finally got so annoyed with the appearance of the APL symbols in Arial that I spent a few days dusting off my FontLab skills and made a fully Unicode APL font in the style of APL Sans. Of course, I just followed the spec (as I understood it) and sent it off to Jonathan to test with Dyalog's IME for V10. About 10 seconds later I got a phone call "Your font doesn't work". As you would expect, the very first thing he tried was ι 12 which (being mapped to the Greek iota) failed dismally.

OK, not a big issue. Dyadic fixed these immediately (to the dismay of the PocketAPL testers who had terrible trouble getting the hunt-and-tap keyboard driver re-installed) but it was pure luck that the error was spotted in time. If this had got out of the door as a real shipped product, it would have been much harder to change. Just think of all those scripts that would suddenly stop working!

Nailing this one down turns out to have been the easy bit. The Unicode Standard may be fixed in stone, but there is (unfortunately) some scope for different interpretations. It turns out that three characters are still open to dispute:

- **Quad.** Unfortunately, when the standard was first drawn up, the Unicode people took the rather harsh view that if there was a matching glyph already defined, then oddballs like APL should use it. I can see the logic of this for symbols like ν where the APL function directly implements its mathematical precursor, but the only vaguely quad-ish thing was in the *Geometric Shapes* section at 25AF. After much protest, the APL cause was won, and something much more like a proper APL quad was added as a late runner to the *Misc Technical* block, and documented in the standard as such. The snag is that it never made it into Arial Unicode MS, so it looks very odd on screen.
- **Diamond.** There is one in *Geometric Shapes* at 25CA and one in *Mathematical Operators* at 22C4. They both look the same, so take your pick. Of course, APL2 and Dyadic picked different ones. Such is life.
- **Tilde.** Now this is a real annoyance. It is actually documented in the standard (in the maths section at 223C) as 'APL tilde' but it is visually the same as the ASCII tilde. For that matter there is a perfectly good 5-pointed star at 22C6 which is also noted as 'APL' in the standard. Now as far as I know, everyone uses the ASCII '*' as the APL 'power' so probably we should also use the ASCII '~' as our 'without' symbol. The snag is that anyone coming in from the cold and trying to do the right thing will read the standard and will choose the 'correct' one. In this case IBM chose ASCII and Dyadic chose maths.

What Needs to Happen

We have been very fortunate in the co-operation between two (now 3) of the vendors of Gui APL systems in the near convergence of the mapping from $\square\Delta V$ to the characters you see on screen. I could have typed that little fragment of APL into Dyalog, APL+Win or APLX and hit Ctrl+C to put it on the clipboard. Then I could have pasted it into either of the others and it would have worked correctly.

What we need next is a rapid agreement between all the vendors on these obvious danger points, and a published and accepted consensus on the entire font. This should be easy enough, as Dyalog are now in full agreement with IBM with both PocketAPL and APL.net.

Lee Dickey has already done sterling spadework which is well documented at:

<http://www.math.uwaterloo.ca/~ljdickey/apl-rep/tables/set3.html>

so perhaps he could look after the 'definitive' statement on his web-site?

There remains the interesting issue of how many alternatives an interpreter should accept as inbound mappings. There is already a precedent here – Dyalog APL will accept either variety of $\mid$ as the 'remainder' symbol, so we could be relaxed about this and have interpreters map (for example) either $\sim$ or $\neg$ to the 'without' function. Somehow, I strongly dislike the idea. For one thing, it should be perfectly OK for a Greek APL user to type $\alpha\iota\pi$ as the α and the rightmost ι are perfectly reasonable one-letter variable names in Greek! My instinct is to promote an APL Unicode font which has something resembling a particularly aggressive 'NO ENTRY' symbol at the ambiguous points which we have chosen not to use. Where there are valid Greek characters, they should be included (along with Cyrillic) but should be visually as different as possible. What do you think?

You can check what it might look like on page 122 which has a full sample of the font (as it stands today) and some more notes on the characters. Of course, it will be made available on the Vector website and will be freely redistributable.

British APL Association AGM

2:00 p.m. Friday 23rd May 2003

Royal Statistical Society, Errol Street, London EC1Y 8LX

Agenda: Minutes, Chairman's report, Treasurer's report, Questions to Committee, Appointment of Auditor and Other Business. A member who wishes to raise other business should give a month's notice to the secretary.

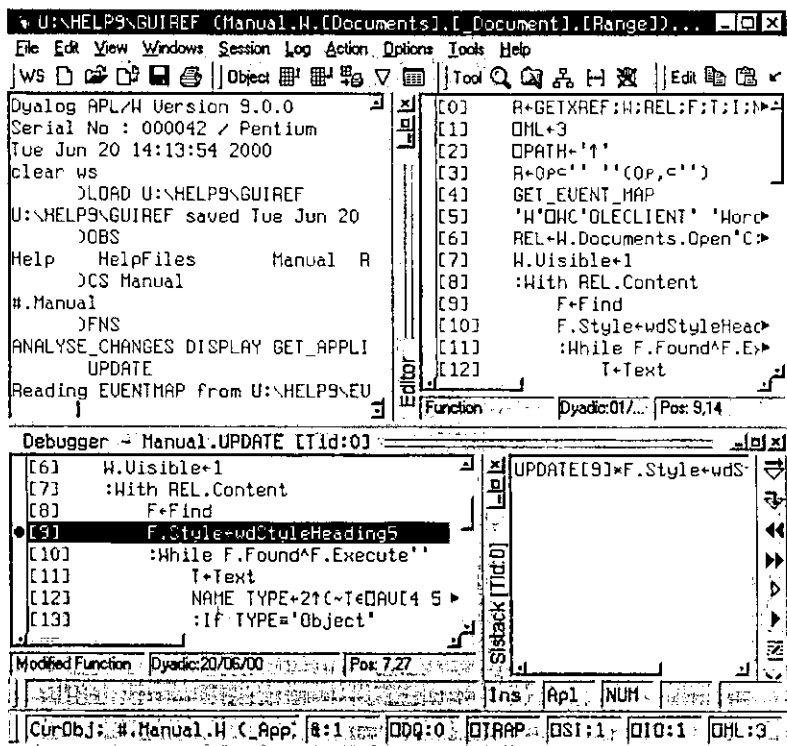
Nominations are invited for the Committee to take office after the AGM.

Nominees must agree, be proposed and seconded by members, and be notified to the secretary by March 31st 2003. If an election is needed it will be held by post.

Anthony Camacho, Hon. Sec. January 2003

dyalog APL

The Definitive APL for Windows™



Version 9 - an even better IDE(A)

<http://www.dyadic.com>

Dyadic Systems Limited, Riverside View, Basing Road, Old Basing, Basingstoke,
Hants. RG24 7AL, United Kingdom.

Tel:+44 1256 811125 Fax:+44 1256 811130 Email: sales@dyadic.com

Microsoft is a registered trademark and Windows and the Windows Logo are trademarks of Microsoft Corporation

Quick Reference Diary

Date	Venue	Event
Feb/March	London	WebServices Evening Seminars with the BCS OOPS Group See www.bcs-oops.org.uk for details
27-28 March, 2003	Helsinki, Finland	Forests in Finland 2 day meeting, details to be announced.
23 May 2003	RSS, London	BAA AGM and Meeting
11-14 June, 2003	San Diego, USA	FCRC:APL2003 joint event See Call for Papers in Vector 19.2
4-6 Sept, 2003	Gilleleje, Denmark	Viking APL Conference See http://www.insight.dk for details
Oct/Nov 2003	USA	APL2000 User's Meeting. Date and location to be announced.

Dates for Future Issues of VECTOR

	Vol.19 No.4	Vol.20 No.1	Vol.20 No.2
Copy date	7th March	20th June	3rd Sept
Ad booking	14th March	27th June	12th Sept
Ad Copy	21st March	4th July	19th Sept
Distribution	April 2003	July/Aug 2003	October 2003

CORRESPONDENCE

From: Manuel Alfonseca

January 2003

I am surprised by the last paragraph of your comments in Vector. Since I am personally mentioned there by my Christian name, I think I have the right to answer. I assure you that I'm definitely NOT trying to help my university come out of the Franco closet. It doesn't need it. You seem to be a little outdated. Franco died 27 years ago. I think it's about time you stop reminding us of certain things. I doubt you would have dared to say something similar about a German University in 1972, 27 years after Hitler's death.

I can also assure you that I have never hoped the English-speaking world to go away, either in the APL field or otherwise. I have the greatest respect for the works of that "world" in science, the arts, and other fields. If you need proof, for my dictionary of "1000 great scientists," which I published in 1996 (see my web page), I chose 474 born or naturalized in English speaking countries. Before publishing text that may offend the feelings of persons or people, I would advise you to start by forgetting prejudices and stereotypes and really getting to know what you are trying to criticise. You could have said similar things in a very different way.

Regards, Manuel Alfonseca

Ian Clark replies ...

Reading Professor Alfonseca's dignified letter, I think he has a legitimate grievance about what I wrote in my impressions of the APL2002 conference (VECTOR, Vol 19 No 2 pp 63-64). Consequently I'm not inclined to attempt a rebuttal, so it falls to me to apologise for the offence I have given over my remarks, particularly those concerning 'the Franco closet', a 'minor league country' and 'if they just ignore the English-speaking world it will eventually go away'. On sober reflection I can see no valid interpretation being put on them beyond their being gratuitously offensive. I wish to withdraw them. Professor Alfonseca says "You could have said similar things in a very different way". To my shame I must agree with him. Which leaves one to ask: why did I write what I did? I don't think I'm alone in detecting a mismatch with expectations over the quality of papers, at what was after all the annual APL world conference. I'm not alone in worrying over this, and the reasons for it, and trying to think what its implications are for the future. Was this the fault of Professor Alfonseca and his

assistants? Emphatically no! Other correspondents in the same issue were full in their praise of the excellent organisation – the few minor criticisms were totally constructive, I thought. These opinions are ones I happen to share – I should have made this clearer. The quality of submissions from the Escuela Tecnica Superior de Informatica itself were nothing to be ashamed of – any suggestion in my article to the contrary was entirely unintended.

So why weren't they supported by papers of comparable quality by the APL community at large? Is APL dead as a world movement? Is the international APL community dead? That means "us" – are we (brain-) dead? I tried to conjecture reasons in my last paragraph, and the reasons I offered were quite wrong. But the questions remain. Maybe I don't deserve to be allowed a second attempt at a diagnosis, but Professor Alfonseca does at least deserve this of me: to declare emphatically that it was not his fault, nor that of his department or the Universidad Autonoma de Madrid, nor of Spain in general.

It was "our" fault, the fault of the international APL community. For not supporting the annual conference with the number and quality of contributions it merited. After all, any selection committee can only work with the material submitted to it.

A note from the Vector Working Group

From time to time, in the spirit of free exchange of ideas within a friendly and informal community, VECTOR publishes candid comments by individual BAA members, without them necessarily representing the view of the editorial team as a whole. Nevertheless we would like to offer our regrets over the offence which Dr. Clark's remarks undoubtedly gave to Professor Alfonseca and the organisers of the APL 2002 Madrid conference, remarks which Dr Clark has since chosen to retract.

BAA: Sustaining Members' News

MicroAPL Ltd

When we launched *APLX* last year, we emphasized its unique cross-platform capability as the only APL interpreter to run on both Windows and Macintosh platforms. What's more, the product was designed so that your APL code would work cross-platform, with not only the pure APL language but also the *QWI* windowing interface working in the same way under both environments. This applies to user-interface controls such as buttons, tree controls and even the RichEdit controls, and also to APL graphics using the *QWIDraw* method.

Well, the good cross-platform news just got better. The full windowing version of *APLX* is now also available for Linux, offering the same advanced APL development environment which our Windows and Mac products offer. And once again, the full *QWI* environment is implemented, so that you can program the user-interface from APL on all three platforms, usually with little or no platform-specific code. This means that, with *APLX*, you can write a modern, user-friendly APL application which can be run under Windows 95, 98, ME, NT, 2000 and XP, MacOS 8.6, 9.x and 10.x, and now under the main Linux desktop distributions including RedHat, Debian, Mandrake and SuSe. Different platforms share the same workspace format, and of course component files can be shared across a heterogeneous network of *APLX* systems. Just *)LOAD* and go!

To help promote the wider adoption of APL, we are making the Desktop Edition of *APLX for Linux* available free of charge for personal use. You can download it from our website <http://www.microapl.co.uk/apl>. For commercial use, the product is priced at £499 (excluding UK Value Added Tax), the same price as the Windows and Macintosh versions. The Server Edition of *APLX* for Linux, which is designed for legacy and server applications, remains available as a separate product but now includes the Desktop Edition (for APL development) at no extra cost.

HMW Systems

To add to *Olmec*, *Inca* and *Maya* we are introducing *Nazca* (continuing the ancient american theme). Like our other products, it is an extension of the work we've been doing for the past 15 years on *4extra* (our dealing system product). This had to have a flexible, generalised database for static data (the parts of the database that don't change too often, as opposed to the transactional data). The original system was developed before any of the now popular PC based databases had emerged

on the DOS/Windows platform. Different banks wanted different tables, different interfaces, different - well just about everything. Not only that, we were concerned about performance, scalability and maintaining those early offerings. If we adopted one third party product, no doubt this would be "non-standard" in another institution and the thought of supporting local staff (who didn't always speak perfect English) in a product they (and we) were not used to, or of maintaining different versions for different clients, has meant that we have continued with our own database to the present day.

Now, don't misunderstand me, this isn't "head in the sand, APL only" stuff, nor are we seriously trying to challenge or convert the world of Oracle, Sybase, DB2, etc. The advantages are that we have full control over Nazca, it's fast, stable (In all it's years of use it has only ever corrupted one file - by writing back records in alphabetic sort order, not the original order and that was about fourteen years ago) and flexible - just drop a file in the designated directory & it will be picked up, you can specify fields in a header, or through the GUI, all you have to do is add validation from a table and you're up and running!

Enough about Nazca, I want to talk about something else.

We are adding a shopfront to our web site, to (we hope) sell a number of small products based on our "Meso" range (if I can call it that). Users will be able to download a runtime interpreter for free and then buy a product. We are still developing the first items, but have the infrastructure in place. My question is - is there anyone out there who has an application they want to sell? This would generate you a bit of extra income and, I imagine, help with your IR35 status (if you are in the UK). We would package it up with Installshield, add the product to the shop, and collect your fee for you. We would make your ownership of the product quite clear. For support, we can give you an email address ("jane.jones@4xtra.com", actually we can give you that anyway) which would direct to your real address. You would also get a section on our web discussion board.

If anyone is interested in this idea, please contact me - chris.hogan@4xtra.com

Dyadic

Dyadic is pleased to announce the immediate availability of Pocket APL for the Pocket PC 2002 platform. Pocket APL is priced at only \$39.99 and will initially be sold through Handango (www.handango.com) the leading supplier of PDA software.

Pocket APL has been in development for over two years and has involved a significant amount of effort from the Dyalog APL team; it represents a considerable investment. Our objectives are:

- To introduce APL to a wider audience by providing a fully functional and complete implementation of a modern commercial APL at a very attractive price.
- To develop the potential for APL as a concise and powerful application development tool for the PDA market.

Dyadic believes that Pocket APL offers a tremendous opportunity for APL developers to create innovative, exciting and successful commercial applications for PDAs.

The text of our Pocket APL product announcement for Handango follows ...

Pocket APL is an extremely concise yet powerful programming language. Each of the built-in primitive functions is represented by a single symbol. This means that complex and powerful programs can be expressed in an astonishingly small amount of code. Large applications can therefore be written, edited and debugged within the limited amount of screen space available on the PDA. Consequently, Pocket APL is ideally suited for ad-hoc development tasks on a PDA, whether or not you are a desktop APL programmer.

Pocket APL is compatible with Dyalog APL on other platforms, so Dyalog APL applications designed primarily for desktop/laptop use may now be executed on a PDA. Dyalog APL developers and end-users may now take advantage of a PDA in situations where a desktop or laptop computer is unavailable or inappropriate.

Pocket APL supports the development of rich graphical user interface and provides over 60 object classes including the popular Dyalog APL Grid control. Also provided is a drag and drop tool for building screen layouts so that you can create attractive user-interfaces with little or no programming effort.

Pocket APL includes a purpose-built presentation graphics system for generating charts and business graphics that is suitable for a wide range of applications.

Dyadic expects Pocket APL to stimulate the development of new breed of applications for the PDA market. These applications will take advantage of the special characteristics of the APL programming language, that make APL so popular in the financial, statistical, forecasting, mathematical and modelling arenas. These features include very rapid application development (RAD), array handling, and exceptional computational speed.

Pocket APL contains all of the development features of Dyalog APL Version 10.0, including the same integrated development environment for creating, editing, and debugging APL programs. Workspaces and component files are fully compatible with Dyalog APL /W for Windows.

Special interfaces are provided to simplify the entry of APL expressions and for editing APL functions on a PDA.

Pocket APL automatically implements the appropriate appearance and behaviour for certain Windows controls, such as menu bars, that behave differently under Pocket PC 2002. Windows GUI applications will typically require modification to work well on the low-resolution screen, but will otherwise run unchanged.

Insight Systems ApS

The Vikings are Back!

Insight Systems is happy to announce that we have decided to hold a second "Viking APL Conference" on September 4th-6th 2003 (Thursday to Saturday, to allow cheap tickets and the option of Sunday in Copenhagen). The venue will be the same as for the first conference:

On the north coast of Zealand in Gilleleje, about 25km from Helsingør.

The details of the programme are not known at this time, but the focus will be on vendor workshops. Dyadic Systems has indicated that they will be come and give one or two workshops on "hot stuff".

We also expect to repeat the highly successful workshops on *Axe Throwing* and *Beer Drinking* (we're working on getting the Kindergarden Teachers to repeat their parallel seminar to complete the Viking Experience).

Details will appear on the Insight Systems website: <http://www.insight.dk>.

The Vector Product Guide

compiled by Gill Smith

VECTOR's exclusive Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage. The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages.

For convenience to readers, the product list has been divided into the following groups ('poa' indicates 'price on application'):

- Complete Systems (Hardware & Software)
- APL and J Interpreters
- APL-based Packages
- Consultancy
- Other Products
- Overseas Associations
- Vendor Addresses
- World Wide Web and FTP Sites

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

We also welcome information on APL clubs and groups throughout the world.

*Your listing here is absolutely free, will be updated on request, and is also carried on the Vector web site, with a hotlink to your own site. It is the most complete and most used APL address book in the world.
Please help us keep it up to date!*

All contributions and updates to the Vector Product Guide should be sent to: Gill Smith, Brook House, Gilling East, York, YO62 4JJ. Tel: 01439-788385, Email: apl385@compuserve.com

APL INTERPRETERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL Borealis Inc.	Dyalog APL	poa	Distributor of Dyalog APL products from Dyadic
	APL2000	poa	Distributor of APL2000 products
APL Systems IDC SL	APL+Win v3.6 Full version	1895	A 32-bit Windows-hosted interpreter that runs under Windows 95/98/ME/NT/XP
	Upgrade to v3.6 from v3.x	474	
	to v3.6 from APL+Win v2.x	1034	
	to v3.6 from APL+Plus III v1.x	1249	
	Migration to v3.6 from APL+Plus II or APL+Plus PC	1464	
	APL+Unix		(Please contact us for details.)
Beautiful Systems	Dyalog APL/W for Windows	poa	US Distributor of Dyalog APL products from Dyadic.
	Dyalog APL for Unix	poa	See Dyadic listing for product details.
Dinosoft Oy	Dyalog APL/W for Windows	poa	Finnish distributor of Dyalog APL products.
	Dyalog APL for Unix	poa	See Dyadic's listing for product details.
Dittrich & Partner	APL+Win	poa	Cognos/APL2000 Inc products
	Dyalog APL	poa	Dyadic Systems Ltd. products
	IBM APL products	poa	
Dyadic	Dyalog APL for DOS/386	995	Second generation APL for DOS. Runs in 32-bit mode, supports very large workspaces. Unique "window-based" APL Development Environment and Screen Manager. Requires 386/486 based PC or PS/2, at least 2Mb RAM, EGA or VGA, DOS 3.3 or later.
	Dyalog APL/W for Windows	995	As above, plus object-based GUI development tools. Requires Windows 3.0 or later.
	Dyalog APL for Unix	995-12,000	Second generation APL for Unix systems. Available for Altos, Apollo, Bull, Dec, HP, IBM 6150, IBM RS/6000, Masscomp, Pyramid, NCR, Sun and Unisys machines, and for PCs and PC/2s running Xenix or AIX. Oracle interface available for IBM, Sun and Xenix versions.
DynArray	DICE for Windows	poa	Software development kit which includes an APL interpreter as a DLL and the ability to run and link existing and new APL code to non APL code such as VB, C/C++, Java and integration with various Windows software applications and database packages such as MS Office.
I-APL Ltd	I-APL/PC or clones	8	ISO conforming interpreter. Supplied only with manual (see 'Other Products' for accompanying books).
	I-APL/BBC Master	8	As above
	I-APL/Archimedes	8	As above
IBM APL Products	TryAPL2	free	APL2 for educational or demonstration use. Write, fax or Email to APL Products; specify disk size desired.
	Workstation APL2 V2 Version 2	\$1500	AIX, Linux, Solaris, Windows Product 5724-B74, Part Number 45P7514
	APL2 Version 2	poa	Product No. 5688-228. Full APL2 system for S/370 and S/390
	APL2 Application Env't Vn2	poa	Product No. 5688-229. Runtime environment for APL2 packages
Insight Systems	Cognos/APL2000 Inc	poa	Leading distributor of APL2000 products in Denmark
	Dyadic Systems Ltd.	poa	Leading distributor of Dyalog APL products in Denmark
	IBM	poa	Leading distributor of IBM APL & GraphX products in Denmark
J Austria	J	poa	Distributor for Austria and Switzerland
	Dyalog APL	poa	Distributor
	Causeway Products	poa	Distributor

	Structural Analysis Software	poa	Complete package by IG Zenkner&Handel to perform structural analysis/engineering calculations. Also suitable for dynamic problems, e.g. earthquake simulation.
JSoftware Inc.	J on the Web online registration ...		
	J Professional (online reg.)	\$895	includes manual set and one year of updates
	J Standard (online reg.)	Free	Free for download only
	Books and accessories		
	J Dictionary	\$50	
	J Phrases	\$50	
	J Primer	\$50	
	Fractals, Visualization and J	\$80	
	Concrete Math	\$40	
	Exploring Math	\$50	
Lescasse Consulting	APL+PC	poa	Lescasse Consulting is the exclusive APL2000 distributor in France and also distributes in Switzerland and Belgium. Call for price quotes.
	APL+Unix	poa	
	APL+DOS	poa	
	APL+W/in	poa	
	Dyalog APL/W	poa	French distributor for Dyalog
MasterWork Software	Manugistics Products and ISI	poa	New Zealand distributor
MicroAPL	APLX for Windows/MacOS	499	Cross-platform APL development environment with GUI programming facilities. Interpreter modelled on APL2. Available for Windows 95/98/ME/NT/2000/XP, Mac OS 9 and Mac OS X.
	APLX Server Edition	poa	For running large multi-user APL applications on x86 Linux, RS/6000 AIX, and Windows NT/2000.
	APLX for Linux (commercial)	499	Linux desktop edition of APLX, compatible with Windows and MacOS versions, with full development environment and GUI programming facilities. Runs under RedHat, Debian, Mandrake and SuSe Linux distributions.
	APLX for Linux (personal)	Free	Full version of APLX, can be downloaded free for personal use.
Oasis	Dyalog APL	poa	Dyadic Systems
	APL*PLUS	poa	Manugistics
	APL.68000	poa	MicroAPL Ltd
	APL2	poa	IBM
Omega	Zero	poa	A "small simple and fast" alternative to APL
Optima	Dyalog APL/W	995	Dyadic's best, leading edge APL interpreter
RE Time Tracker Oy	APL+PC (APL*PLUS/PC)	poa	Complete APL+ and Statgraphics product range and links to various 3rd party products.
	APL+DOS (APL*PLUS II)		
	APL+W/in (APL*PLUS III), APL+Link		
	APL+UNIX		
	APL*PLUS Sharefile		
Soliton Associates	SHARP APL for OS/390	poa	for IBM OS/390 mainframes
	SHARP APL for UNIX	poa	for SunOS and IBM AIX
	SHARP APL for Linux	poa	for Intel Linux
Strand Software	Canada		
	All APL*PLUS products	poa	All APL*PLUS products including upgrades and educational.
	Dyadic and JSoftware products	poa	

USA

Dyadic and JSoftware products poa

APL PACKAGES

COMPANY	PRODUCT	PRICES(£)	DETAILS
ADAPTA Software	MPS	poa	Master Production Scheduling
	FBS	poa	Forecasting and Budgeting System
	DRP	poa	Distribution Requirements Planning
Adaptable Systems	FLAIR	poa	Finite loader and interactive rescheduler. Customisable full-function scheduling system. (Available outside Australia by special arrangement only.)
Adaytum	Adaytum e.Planning	poa	Adaytum e.Planning offers a Web-based solution that combines planning, forecasting, budgeting, modelling and reporting in a single, integrated application.
APL Software\Services	APL Utilities	poa	Software: mostly .AWS for DOS, utilities for most APL interpreters. Public domain APL*Plus v10 with on-screen documentation and interactive tutorials, APL Conference Software. Books: APL user manuals for STSC, IBM, and Sharp. Request email catalog from dick.holt@juno.com.
	APL Systems IDC SL APL+Linkpro 3.0 New System	646	Database Access SQL Client for ODBC
	Upgrade to v3.0		
	from APL+Linkpro-32	280	
	from APL-Link or APL+Linkpro	474	
	GraphX (Includes ChartX)	599	
	GraphX Lite (No distribution)	340	
	SPREAD (An APL grid control)	86	
	QPLOT (x,y plotting in APL)	47	
	QWIN	215	(Runs APL+DOS functions in APL+Win)
Beautiful Systems	ASF_FILE	\$399	Dyalog APL/W auxiliary processor for access to APL*PLUS/PC APL component files (*.ASF).
	NAT_FILE	\$299	Dyalog APL/W auxiliary processor which emulates the APL*PLUS/PC quad-N native file subsystem for access to the DOS file system.
	DBF_FILE	\$299	Dyalog APL/W auxiliary processor for efficient block mode access to dBASE format files. Designed to get large amounts of data in and out of dBASE. Not suited for random access to small amounts of data (it does not handle keys).
	SF_READ	poa	Dyalog APL/W functions to read APL*PLUS data objects of any type or structure from *.SF style component files created by APL*PLUS II or III.
Causeway	CausewayPro for Dyalog/W	400	Causeway application development platform for Dyalog APL/W.
	RainPro Business Graphics	250	The ultimate graphics toolkit for the APL developer. Adds 3D charting capability, Web publishing and clipboard support to the shareware product. Charts can be included in NewLeaf reports. Functionally compatible across Dyalog/W and APL+Win.
	NewLeaf for Dyalog and +Win	400	Frame-based reporting tool with comprehensive table-generation and text-flow support. Offers multiple master-page capability, bitmap wrap-around and on-screen preview with pan and zoom. Fully supported on Dyalog/W and APL+Win
Cinerea AB	ORCHART	250	Organization chart package for IBM APL2/PC. Full & heavily commented source code included - free integration into other applications. NB: ASCII output with line-drawing (semi-graphic) characters for boxes.

CODEWORK	3-way TANGRAM	poa	3-way TANGRAM is a DSS-OLAP product, basically a powerful and versatile handler of multi-way tables (also known as hypercubes). It entails 46 analysis modules, including computations, cube merging, sensitivity analysis, time intelligence, free format queries, HTML and LaTeX outputs. Current version is 7.0. At the time of writing, the product is available on Dyalog APL 8.3.1 for Windows 95/98/NT. Future plans include an APL+WIN version and later a LINUX version.
CoSy	K.CoSy	\$30 % yr sub	K.CoSy is a general purpose computing and programming environment constructed, all in open code, in Arthur Whitney's very high level, yet structurally transparent Array Programming Language, K, and its tightly coupled User Interface. K.CoSy is an extremely productive environment in one of the most powerful and fastest of APL's progeny, and therefore, likewise, of all languages. K.CoSy provides a workspace-like interactive development environment previously impossible in K. Because of its unique open construction within the language itself, this environment is clearly competitive in a large domain with the APLs from the other vendors. K.CoSy notepad nature, interactivity, and open K code vocabulary make learning Arthur Whitney's K far less daunting and far more productive than its raw console, or any external scripting method. If you are a client of Kx Systems, or are investigating the possibilities, Contact us. See CoSy/K.CoSy for more information.
DynArray	DynaWeb Server	poa	A web server providing web based access to applications running on the DICE interpreter from DynArray, or on an IBM mainframe running APL2.
	DynaHarry	poa	A DSS system which offers the next generation capabilities for current APLDI, IC/E and IC/I users. It comes with ROLAP capabilities, multisystem access to a wide variety of databases and data warehouses.
	DynaLink	poa	An ODBC client interface for DICE and IBM APL2 programs.
Entropy Software	Personal Information Management	poa	Entropy specialises in Personal Information Management Systems (PIMS) which run on a PC. Most systems are inadequate in some way. This is an attempt to fulfil all the needs of George MacLeod who has been an unhappy user of a variety of systems for many years. The product is being developed in modules (Calendar, Address List, To Do List, etc) which will be available at no charge. When all the modules are complete they will be integrated together. There will be a charge for the integrated product.
HMW	4XTRA	poa	Networked, Windows/Unix based Front End and Middle Office Foreign Exchange and Money Market Dealing System. Scalable from 1 user to 120+.
	Inca	poa	Software Change Management System. Enables the user to co-ordinate development work from several sources, resolve clashes, promote work items for testing and configure releases to a live environment.
	Maya	poa	APL code file manager. A comprehensive suite of tools giving a multi-window IDE style interface to file based APL code. Offers features such as copying from file to file, object comparison, string search, style formatting, hot-spot editor for filed objects (including variables), etc.
	Aztec	poa	System shell for APL development. Manages real-time and batch applications across multiple platforms. Offers standardised error trapping, job scheduling, task communication and recovery/restart features.
	Olmec	poa	APL GUI environment, providing menu bar, tool bar, status area, navigation sidebar (with treeviews & listviews) and client area. All are configured by simple text files and require no programming. Client area has a "tab wizard" option to provide ordered transaction processing
	Nazca	poa	fast, flexible and reliable static database and editor.

I-APL Ltd	Educational workspaces	5	PC format disks with the examples from: Thomson, Espinasse (Kits 1-4), Kromberg, Jizba & FinnAPL. All the examples to save your fingers!
IBM APL Products	A Graphical Statistical System	\$250	for DOS, Product Number 5764-009
Insight Systems	Causeway	poa	Leading distributor of Causeway products in Denmark <i>All our old products are now either OEM'd, in the public domain, out of date, or all of the above. We'll be back!</i>
JAD Software	JAD SMS	poa	JAD SMS is a multi-user software management system for Dyalog APL™ based on shared, hierarchical databases. JAD SMS databases let you keep historical versions of apl items as well as attributes such as timestamp, user name and documentation. The software includes a graphical user interface as well as specialized functions for inclusion in applications. No charge for single-user version; \$100/user for multiple users
Lescasse Consulting	APL+Win Monthly Training	\$600	Download 50+ page document about APL+ programming each month. You also get one or more workspaces full of re-usable APL code and sometimes additional files or products.
	Advanced Windows Programming	\$95	200-page book plus companion disk on interfacing APL and Delphi. Contains full coverage of Delphi-2, +Win and Dyalog.
	DLL parser for APL	\$250	Parse any Visual Basic DLL declaration file into a set of quadNA definitions. Turn constants and structures into APL variables. Available for APL+Win and Dyalog/W.
	Delphi Forms Translator	\$195	Design forms with Delphi and turn them automatically into APL programs which recreate the same form (+Win and Dyalog/W).
	APL+Link Pro	poa	ODBC interface for APL+Win
	SQAPL Pro	poa	ODBC interface for Dyalog APL/W
	RainPro	poa	Highly customisable 2D and 3D publication graphics for APL+Win and Dyalog APL/W
	NewLeaf	poa	Page layout and printing tools for APL+Win and Dyalog
	GraphX and ChartFX	poa	High-quality business graphics for APL+Win
	Formula One and Dyalog APL	\$95	100-page book + companion disk on how to use the Formula One VBX with Dyalog APL/W
Lingo Allegro	FACS	poa	EMMA-like interface to DB2 or ODBC databases
	QWIN	poa	Legacy DOS Windowing support for APL+Win
	ODBC/127	poa	IBM AP127-like ODBC Interface for APL+Win and Dyalog APL/W
Optima	ServiceLine	poa	A property management system which keeps a record of all outstanding tasks, produces an up to date list of work to-do and Scheduled reminders plus automated standard letters and basic financial control.
	TravelLine	poa	A system designed to control the workload allocation for a fleet of chauffeur vehicles plus a reminder system for fleet management and Local Authority requirements.
	BPA	poa	"Brand Performance Analysis" allows for the modelling of product/brand performance over time and comparison with competitor products.
	DBI	poa	"Database Interrogator" allows for the non technical user to ask sensible questions of a large database such as a questionnaire and obtain results tables and graphs quickly, easily and accurately.
Qualedi	Qualedi	\$850-\$5,600	Electronic Data Interchange (EDI) translation software for the PC, with strict compliance checking.
RE Time Tracker Oy	UIT/W	poa	Comprehensive high-level Windows User Interface library for APL+Win and +II v 5.1. Comprehensive spreadsheets, replicated fields, special field types, etc. 16 and 32 bit versions available.
	AJGRAPH	poa	Graphpak-compatible 2D graphics package for +Win and +DOS. Includes multi-window support, print and metafile support. No DLLs required.

	ECCO PRO with APL	poa	Leading group and personal information management system with comprehensive customising. Supplied with sample +Win workspace to interface to ECCO databases via DDE.
	NEWT TCP/IP SDK with APL	poa	Lead TCP/IP SDK with interfaces to all protocols. Supplied on 3 CD ROMS together with a sample +Win workspace.
	DB+	poa	Database interface for APL+DOS under Windows. Allows combining character-based APL applications with ODBC-compliant databases such as Oracle and SQL-server.
Warwick University	BATS	250	Menu driven system for time series analysis and forecasting using Bayesian Dynamic modelling. Price is reduced to £35 for academic institutions.
	FAB	free	Training program for the above.
Weighahead Systems	Weighahead Windows Weighing System (3WS)	poa	Recipe Weighing System for Manufacturing Industries. Pharmaceutical, Cosmetics, Foods etc. Works without keyboard or mouse. Uses Electronic Balances, Laser scanners, bar codes and label printers.
Zark	APL Tutor (PC)	\$299	APL computer-based training. Available for APL*PLUS PC & APL*PLUS II. Demo disk \$10.
	APL Tutor (MF)	\$5000	Mainframe version.
	Zark ACE	\$99	APL continuing education. APL tutor news and hotline phone support.
	APL Advanced Techniques....	\$59.95	488pp. book, (ISBN 0-9519067-07) Including 2-disk set of utility functions (APL*PLUS PC format).
	Communications	\$200 pc, \$500 mf	Move workspaces or files between APL environments.

APL CONSULTANCY AND DEVELOPMENT

COMPANY	PRODUCT	PRICES (£)	DETAILS
Adfee	Consultancy	poa	Development, maintenance, conversion, migration, documentation, of APL products in all APL environments
Andrews	Consultancy	poa	APL programming and analysis, algorithms, tree processing and design programs for craft work.
APL Borealis Inc.	Support and Development	poa	APL Software Support and Development. Specialists since 1979 in Sharp APL, APL*Plus, APL+Win, Dyalog APL
APL Solutions Inc	Consultancy	poa	APL systems design, development, maintenance, documentation, testing and training. Providing APL solutions since 1969.
APL Systems IDC SL	Consultancy	poa	Consultancy and maintenance available by retainer or on call.
AUSCAN Software	Consultancy and Training	poa	APL software development, training
Camacho	Consultancy	poa	Manuals; feasibility reports and estimates; analysis and programming; APL and MS Windows applications; Sharp, ISI APL, APL*PLUS, APL2/PC and other APLs spoken. Fixed price systems a speciality
Ian Clark	Consultancy	poa	Interfacing APL, VB, C/C++, ActiveX/COM. Screen design and documentation. National language porting.
Ray Cannon	Consultancy	poa	APL, C, Assembler, Windows, Graphics: PC and mainframe
Causeway	Consultancy and Training	poa	On-site training for Causeway, RainPro and NewLeaf. Customisation and enhancement to meet local needs. Code review and pre-implementation check of Causeway applications.
Paul Chapman	Consultancy	250-500	24-hour programmer: APL, Smalltalk, C; Windows front end design a speciality.
CODEWORK	Consultancy	poa	Development, maintenance, migration, documentation of APL applications. Speciality: info systems for top executives, internet applications.

CoSy	Consultancy	poa	CoSy.com, Coherent Systems, provides rapid development in the K language and associated data base products, with a particular interest in quantitative (financial) modelling.
David Crossley	Consultancy	poa	Experienced in large APL system developments since 1989 for PC or mainframe.
Dinosoft Oy	Consultancy	poa	Specialised in very large databases.
Dittrich & Partner	Consultancy	poa	APL programming and analysis; APL workshops and training on the job
Dyadic	Consultancy	poa	APL and Unix system design, consultancy, programming and training.
DynArray	Consultancy	poa	DynArray offers consulting in the areas of DSS, Y2K and APL programs upgrade/conversion to modern Web enabled platforms.
Evestic AB	Consultancy	poa	Excellent track record from 15+ years of APL applications in banking, insurance, and education services. All dialects, platforms and project phases. SQL expertise.
First Derivative Analytics Ltd.	Consultancy	poa	Analysis, design, prototyping, development & testing of APL (especially financial) applications: Sharp, Dyalog APL/W.
General Software	Consultancy	from 200	Over 20 years experience with every version of APL, large mainframe systems and small PC based programmes.
Godin London Inc	Software Development	poa	We have applications in the food manufacturing field, travel agency and airline bookings field and in product lease management.
HMW	Consultancy	poa	System design consultancy, programming. HMW specialize in banking and prototyping work.
Hoekstra Systems	Consultancy	poa	APL consultancy, programming, etc. Also UNIX system administration
Michael Hughes	Consultancy	poa	APL consultant with 20 years experience with all versions of APL. I can create your dynamic Web sites using the full power of APL working with Microsoft IIS (Internet Information Service) on Windows NT or 2000. I also undertake System design, Programming and Maintenance on all platforms, particularly MS Windows.
INFOSTROY	Consultancy	poa, competitive	Broad experience in various APL platforms. Special skills and knowledge in developing complex applications for investment, financial and construction markets. Implementation of hybrid solutions based on APL, Delphi, C++, VBA, SQL servers.
Insight Systems	Consultancy	poa	We have experience with just about every APL system and platform in common use during the last 20 years, from SHARP APL under MVS or Linux to APL+Win and in particular Dyalog APL under Windows 9x, NT or 2000. If you have decisions to take about adapting your APL application to take advantage of emerging technologies, or would like your strategy reviewed, give us a call. We have extensive experience in all areas of APL development, from legacy systems, up, down and sideways migrations, to the development and support of shrink wrapped solutions based on APL. Even if we don't have time to do the work ourselves, we will know where to find someone who is an expert in your version of APL and your application area, on your continent.
JAD Software	Consultancy	poa	Systems design and development, project management, technical manuals, financial and actuarial expertise in APL.
KJK	Consultancy and software development	poa	APL-based data management: conversions, ad hoc-analyzing tools, well-interfaced methods for defining, processing and browsing of multi-dimensional reports. Rapid custom software development based on proven modular toolset approach.
Lambent Technology	Consultancy	poa	APL programming, consulting & training; web design and construction.
Phil Last	Consultancy	poa	APL consultancy, modelling and programming.

Lescasse Consulting	Consultancy	poa	A range of consultants, experts in Windows programming, with APL+Win and Dyalog APL/W. More than 100 major APL applications already developed. We all have additional expertise in Formula One and Delphi.
Lingo Allegro	Consultancy	poa	General APL consulting, Internet website development, migration and downsizing, performance tuning, education and training.
Lucas Solutions	Consultancy	poa	Rates depend on task and location.
Mackay Kinloch Ltd	Consultancy	poa	Design, analysis and programming for banking, insurance and pensions, financial planning and modelling, corporate performance and legal reporting
MasterWork Software	Consultancy	poa	Consulting and J programming for econometrics and statistics in public policy, health and food industries.
Millinta Inc	Consultancy	poa	Design, development, maintenance, conversion, documentation in all APLs, most APs and some specific Sharp products (LOGOS, ViewPoint, Retrieve). Experience in multi-user, multi-task systems, databases, Windows programming.
Ellis Morgan	Consultancy	250-500	Business Forecasting & APL Systems.
Oasis	Consultancy	poa	Expertise in APL system design, Project management, conversion, migration, tuning, for all APL versions (10+ years experience)
Omega Computing	Consultancy	poa	APL consultancy, programming, etc.
Optima	Consultancy, Training & Development	poa	We have been in business since 1990 and have a team of APL professionals with many years experience in pharmaceutical, industrial and financial systems on both PC and Mainframe platforms. In addition to pure APL we can also offer assistance with network integration, ACT! customisation and issues concerning the internet and/or web design.
RadSys Technologies	Consultancy	poa	Areas of expertise: financial systems, risk analysis systems, healthcare systems.
Resources & Results	Consultancy	poa	Knowledge management company builds decision support, data warehouse, data mining and strategic planning systems for CFO's, CEO's, and senior management, using our Rapid Application Development (RAD) methods and tools. Extensive experience in large-scale system development and ad hoc executive support for Fortune 500 clients.
RE Time Tracker Oy	Consultancy	poa	APL application conversions, APL Windows interfaces, APL to API-level interfacing to any system under Windows, TCP/IP network and database connectivity.
Rex Swain	Consultancy	poa	Independent consultant, 25 years experience. Custom software development, PC and/or mainframe.
Rochester Group	Consultancy	poa	Specialise in MIS using Sharp APL
Shepp & Associates	Consultancy	poa	APL applications development and consulting, especially in the travel industry, especially on small computers. 25 years experience in APL programming.
Snake Island Research Inc	Consultancy	poa	APL interpreter and compiler enhancements, intrinsic functions, performance consulting. APL parallel compiler APEX is giving very good initial performance tests with convolution somewhat faster than FORTRAN.
SovAPL	Consultancy	poa	Offshore APL development service.
Strand Software	Consultancy	poa	Advice on migrating to and from all flavours of APL and hardware platforms. Full-screen interface implementation, APL utilities, benchmarking, efficiency analysis, actuarial software, system development tools, valuation, pricing and modelling systems.
Sykes Systems Inc	Consultancy	poa	Complete APL services specialising in audit, optimisation and conversion of APL systems. Excellent design skills. All dialects and platforms. 17-23 years experience.
Weighahead Systems	Consultancy	poa	Specialising in industrial systems. Links to PLCs, laser scanners, bar codes, weigh scales, label printers etc. Also programmable hand held scanners.

Stephen Wynn	Consultancy	poa	Most experience of financial planning, and mathematical areas: operational research, quality control, experimental design.
--------------	-------------	-----	--

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Adfee	Employment	poa	Contractors and permanent employees
APL-385	Typefaces	poa	Variants of the APL2741 typeface available to specification.
APL Borealis Inc.	APL Training	poa	Hands-on courses in Introductory, Intermediate, Advanced and Windows APL. Courses are customized and flexible, and may be delivered on-site, with strong emphasis on methods for efficient and maintainable APL systems development.
ComLog	Comic-Logger	\$25.95+p&p	APL*PLUS II comic-book inventory system. Shareware version available on America OnLine.
HMW	Employment	poa	Contractors and permanent employees placed.
I-APL Ltd	Books	poa	I-APL stocks books written to go with the I-APL interpreter and some APL Press books. For a list write to 11 Auburn Road, Bristol BS6 6LS, ring 0117 973 0036 or email 100612.1057@compuserve.com.
Oasis	Training	poa	Introductory courses in APL Advanced courses for different APL versions
Optima	IBM Compatible	poa	Complete business solutions including hardware, software and support plus additional network installation and maintenance if required.
Renaissance Data Systems	Booksellers		The widest range of APL books available anywhere. See Vector advertisements.
Right Seat Software	Vox Proxy	\$199 (comm)	Vox Proxy is authorware for PowerPoint(r) 2000 or 2002 which allows the use of Microsoft Agent Technology (3D talking animated characters) within slide shows. VP appears on PowerPoint's main menu and provides editing side-by-side with slides. Automated script-writing provides control of PowerPoint, allowing the use of characters for live presentations or fully-automated tutorials, demos, or training programs. Optional CD Prep program allows the user to create auto-starting CD's that will play on any version of PowerPoint or without PowerPoint.
		\$69.95 (edu)	

OVERSEAS ASSOCIATIONS

GROUP	LOCATION	JOURNAL	OTHER SERVICES	Ann.Sub.
ACM SigAPL	International	APL QuoteQuad	Conferences; APL white pages; web site	\$30
APL Bay Area	USA N. California	APLBUG	Monthly Meetings (2nd Monday)	\$20
APL Club Austria	Austria	-	Quarterly Meetings	200AS(indiv), 1000AS(corp)
APL Germany e.V.	Germany	APL Journal	Semi-annual meetings	DM60
Ass. Francophone pour la promotion d'APL	France	Les Nouvelles d'APL	FF350 (private) FF2800 (Company)	
BACUS	Belgium	APL-CAM	Conferences & Seminars	£18 (\$30)
Capital PCUG	Washington, D.C.	Monitor	Monthly meetings, occasional classes	free
Danish SIG	Denmark			
Dutch APL Assoc.	Holland	Vector provided	Mini-congress, APL ShareWare Initiative	
FinnAPL	Helsinki, Finland	FinnAPL Newsletter	Seminars on APL	100FIM(private), 30(student), 1000 (Co)
Japan APL Assoc	Tokyo	APL Journal	Monthly meetings (4th Sat)	10,000yen to join
NY SigAPL	New York, USA	Big Apple APL	Monthly meetings	\$35/\$25(ACM)
Rome/Italy SIG	Roma, Italy			
SE APL Users Grp	Atlanta, Georgia	SEAPL Newsletter	Quarterly meetings	\$10

SovAPL	Moscow, Russia	-	Seminars and Annual Meeting	
SwedAPL	Sweden	SwedAPL Nytt	Semi-annual meetings, seminars	SEK 75
SWAPL	Texas, USA	SWAPL		\$18
Swiss APL (SAUG) Bern		Part of Qlty SI-Info		SF60 (SI) + SF20 (SAUG)
Toronto SIG	Toronto, Canada		Occasional meetings, APL Skills Database, Toronto Toolkit	

ADDRESSES

ORGANISATION	CONTACT	ADDRESS, TELEPHONE, FAX, EMAIL etc.
ACM SigAPL	David Siegel	ACM, 1515 Broadway, 17th Floor, New York, NY 10036, USA (Subs only)
ADAPTA Software GmbH	Michael Baas	Stellinger Weg 19, 20255 Hamburg, Germany. Tel: +49 40 40170951 Fax: +49 40 40170952. Email: info@adapta.de
Adaptable Systems	Lois & Richard Hill	49 First Street, Black Rock 3193, Australia. Tel: +61 3 9569 5578 Fax: +61 3 9589 3220 Email: hiltj@melbpc.org.au
Adaytum Limited	Douglas Rowley	Castlegate, Tower Hill, BRISTOL BS12 0JA. Tel: 0117 921 5555 Fax: 0117 922 7749. Email: sales@adaytum.co.uk
Adfee	Bernard Smoor	Dorpsstraat 50, 4128 BZ Lexmond, Netherlands. Tel: +31 347 342 337 Fax: +31 347 342 342 Email: adfee@concepts.nl
Andrews	Dr Anne D Wilson	12 Thorny Hills, Kendal, Cumbria LA9 7AL, UK. Tel: 01539-731205 Email: ADWilson@kencomp.net
APL-385	Adrian Smith	Brook House, Gilling East, York YO62 4JJ, UK. Tel: 01439-788385 Email: apl385@compuserve.com
APL2000 (Europe)	Fred Honea	see APL Systems IDC SL
APL Bay Area APLBUG	Curtis Jones (Sec)	228 South 15th Street, San Jose, CA 95112-2150, USA Tel: +1 (408) 292-4060 Email: jonesca@vnet.ibm.com
APL Borealis Inc.	Richard Procter	381 Manor Road East, Toronto, Ontario M4S 1S7, Canada. Tel: (416) 457-7828. Fax: (416) 482-6582 Email: info@aplborealis.com
APL Club Austria	Harald F. Nelson	c/o N-TECH, Siebenbrunnengeldg. 4-6, A-1050 Wien, Austria. Tel: +43 1 5458063 Fax: +43 1 5458063-17
APL Germany e.V.	Dieter Lattermann	Rheinstraße 23, D-69190 Walldorf, Germany. Tel: +49 6227-63469 Compuserve: 100332,1461
APL Software Services	Dick Holt	3802 N Richmond St, Suite 271, Arlington, VA 22207 USA Tel: +1 (703) 528-7624; Fax: +1 (703) 528-7617; Email: dick.holt@juno.org
APL Solutions Inc	Eric Landau	1107 Dale Drive, Silver Spring, MD 20910-1607 USA Tel: +1 (301) 589-4621 Fax: +1 (301) 589-4618 Email: aplsi@starpower.net
APL Systems IDC SL	Fred Honea	Alfredo Marquerie, 12 - 2 F, 28034 Madrid, Spain. Tel: +34 91 730 7008 (Office) +34 60 680 5949 (Mobile). Fax: +1 775 743 6131. Email: uksales@apl2000.net
Association Francophone pour la promotion d'APL	Ludmila Lernagnen	174 Boulevard de Charonne, F-75020 Paris, FRANCE Email: lemagnen@aol.com
AUSCAN Software Ltd	Richard Procter	PO Box 39, Mansfield, Ontario L0N 1M0 Canada Tel: +1-705-434-1239 Email: rjp@ca.inter.net
BACUS	Joseph De Kerf	Rooienberg 72, B-2570 Duffel, Belgium. Tel: +32 15 31 47 24
Beautiful Systems, Inc.	Jim Goff	308 Old York Road, Suite 5, Jenkintown, PA 19046, USA Tel: +1 (215) 886-2636; Fax: +1 (215) 886-4888 Email: BeautifulSystems@gofis.net
Camacho	Anthony Camacho	11 Auburn Road, Redland, Bristol BS6 6LS, UK. Tel: 0117-973 0036. email: acam@tesco.net
Ray Cannon		21 Woodbridge Rd, Blackwater, Camberley, Surrey GU17 0BS, UK. Tel: 01252-874697 Email: ray_cannon@compuserve.com
Causeway Graphical Systems Ltd	Adrian Smith	The Maltings, Castlegate, MALTON, North Yorks YO17 7DP, UK. Tel: 01653-896760 Fax: 01653-697719 Email: adrian@causeway.co.uk
Paul Chapman		51B Lams Conduit Street, London WC1N 3NB, UK. Tel: 020 7404 5401. Compuserve: 100343,3210
Cinerea AB	Rolf Komemark	Box 61, S-193 00 Sigtuna, Sweden. Tel/Fax: +46 859 255 421 Email rolf@cinerea.se

Ian Clark	Ian Clark	1205 Deer Creek Drive, Plainsboro, NJ 08536, USA. Tel: +1 609 716 8832 Email: earthspot200@hotmail.com
CODEWORK	Mauro Guazzo	Corso Lanza 105, 10133 Torino, Italy. Tel: +39 11 6601560 Fax: +39 11 6601560 Email: codework@codework- it.com
ComLog Software	Jeff Pedneau	18728 Bloomfield Road, Olney, MD 20832 USA Tel: +1 (301) 260-1435 Email: jeff@softmed.com
CPCUG	Lynne Sturtz	Capital PC User Group, 51 Monroe Street, Suite PE-2, Rockville, Maryland 20850-2421, USA. Tel: +1 (301) 762-9372 Fax: (301) 762-9375.
CoSy.com	Bob Armstrong	42 Peck Slip #4B, New York, NY 10038-1725, USA. Tel: +1 212-285-1864 Fax: +1 212-285-1864. E-mail: bob@CoSy.com.
David Crossley	David Crossley	187 Le Tour du Pont, 84210 ST DIDIER, France. Tel: +33.4.90.66.08.87 Email: crossley@au-village.com
Danish User Group	Helene Boesen	c/o Insight Systems ApS, Nordre Strandvej 119G, Hellebæk, Denmark
Dinosoft Oy	Pertti Kalliojärvi	Lönnrotinkatu 21C, 00120 Helsinki, FINLAND. Tel: +358 9 70028820 Fax: +358 9 70028824 Email: dinosoft@dinosoft.fi
Ditrich & Partner Consulting GmbH	Axel Holzmüller	Kieler Strasse 17, D-42697 Solingen, Germany. Tel: +49 212-260 660 Fax: +49 212-260 6666; Email: info@dpc.de
Dutch APL Association	Bernard Smoor (Sec)	Postbus 1341, 3430BH Nieuwegein, Netherlands. Tel: +31 347 342 337 Fax: +31 347 342 342
Dyadic Systems Ltd.	Peter Donnelly	Riverside View, Basing Road, Old Basing, Basingstoke, Hants RG24 0AL, UK. Tel: 01256-811125 Fax: 01256-811130
DynArray Corporation	Dr James Brown	16360 Monterey Rd. Suite 260, Morgan Hill, CA 95037, USA Tel: +1 (408)-782-6648 Fax: +1 (408)-782-6627 Email: info@DynArray.com
Entropy Software Limited	George MacLeod	Calle Viena 13, Residencia Larrosa, Guardamar, 03140 Alicante, Spain. Tel: +34 61 00 41 940 Email: george.macleod@ntlworld.com
Evestic AB	Olle Evero	Berteliusvagen 12A, S-146 38 Tullinge, Sweden Tel & Fax: +46 778 4410 Email: olle.evero@mail.com
FinnAPL	Olli Paavola	Suomen APL-Yhdistys RY, FinnAPL RF, PL 1005, 00101 Helsinki 10, Finland Email: oli.paavola@pyr.fi
First Derivative Analytics Ltd.	Ken Chakahwata	114 Lemsford Lane, Welwyn Garden City, Herts AL8 6YP, UK Tel/Fax: 01707-339620. Email: KenChakahwata@compuserve.com
General Software Ltd	M.E. Martin	Little Wester House, Westerhill Road, LINTON, Kent ME17 4BS Tel: 01622 749365 E-mail: martin@gssoft.freemove.co.uk
Godin London Incorporated	Gaëtan Godin	12 Gerrard St., London, Ontario, Canada N6C 4C5 Tel: +1 (519) 679-8290 Fax: +1 (519) 436-6381 Email: info@godin.on.ca
HMW Computing	Chris Hogan	Hamilton House, 1 Temple Avenue, London EC4Y 0HA, UK. Tel: 0870-1010-469; Email: HMW@4xtra.com
Hoekstra Systems Ltd	Bob Hoekstra	Dominique, Salisbury Road, Woking, Surrey, GU22 7UR, UK. Tel: 01483-771028. Fax: 01483-837324 Email: Bob.Hoekstra@HoekstraSystems.ltd.uk
Michael Hughes		28 Rushton Road, Wilbarston, Market Harborough, Leics. LE16 8QL, UK. Tel: 01536-770998 Email: Michael@Hughes.uk.com
I-APL Ltd	Anthony Camacho	11 Auburn Road, Redland, Bristol BS6 6LS, UK. Tel: 0117-873 0035. Email: 100612.1057@compuserve.com
IBM APL Products	Nancy Wheeler	APL Products, IBM Silicon Valley Lab, Dept H36/F40, 555 Bailey Avenue, San Jose CA 95141, USA. Tel: +1 (408) 463-APL2 [+1 (408) 463-2752] Fax: +1 (408) 463-4488 Email: APL2@vnet.ibm.com
INFOTROY	Alexey Miroshnikov	3 S. Tulenini Lane, St. Petersburg 191186 Russia. Tel: +7 812 325-9797 Fax: +7 812 311-2184 Email: ailm@infotroy.ru
Insight Systems ApS	Helene Boesen	Nordre Strandvej 119G, DK-3150 Hellebæk, Denmark Tel: +45 70 26 13 26 Fax: +45 70 26 13 25 Email: info@insight.dk
JAD Software	David Crossley	175 East 96th St., Apt. 17G, New York, NY 10128 Country: USA Tel: +1 (212) 369-6713 Fax: +1 (212) 761-0124 Email: jadsms@usa.net
Japan APL Assoc	Toshio Nishikawa	1-8-13 Masujima Buld.6F Higashi Gotanda Shinagawa-ku, Tokyo Japan 141-0022. Tel: +81 (03) 3260-0411 Fax: +81 (03) 3260-0418 Email: KYY00361@niftyserve.or.jp
J Austria	Joachim Hoffmann	Hochsteingasse 13/26, 8010 Graz, Austria. Tel: 0043 (0)316 91 42 51 Mobile: 0043 (0)699 1 91 42 51 2. Email: joachim.hoffmann@gmx.at
JSsoftware Inc.	Eric Iverson	33 Major Street, Toronto, Ontario, Canada M5S 2K9. Tel: +1 (952) 470- 7345 Fax: +1 (952) 470-9202 Email: info@software.com

KJK-tieto Oy	Kimmo Kekäläinen	Merikasarminkatu 10 B 56, 00160 Helsinki, Finland. Tel: +358 50 55 27 207; Email: Kimmo.Kekalainen@pp.hiv.fi
Lambert Technology Ltd	Stephen Taylor	81 South Hill Park, London NW3 2SS, UK. Tel: +44(0)20 7813 3786. Email: stj@lamberttechnology.com
Phil Last Ltd	Phil Last	145 Crossbrook Street, Cheshunt, Herts, EN8 6JY, UK. Tel: 01992-633607 Fax: 0121-359 0375 Email: phil.last@net.ntl.com
Lescasse Consulting	Eric Lescasse	18 rue de la Belle Feuille, 92100 Boulogne, France. Tel: +33.1.46.05.10.76 Fax: +33.1.46.04.60.23 Email: eric@lescasse.com
Lingo Allegro USA, Inc	Walter G. Fil	203 N. LaSalle Street, Suite 2100, Chicago, Illinois 60601, USA. Tel: +1 (800) 546 4621 E-mail: lingo-allegro@visto.com
Lucas Solutions	Jim Lucas	Stubbedamsvej 9C, 3. tv., 3000 Helsingør, Denmark Tel: +45 49 26 52 42. Email: jel@danbbs.dk
Mackay Kinloch Ltd	Alastair Kinloch	519 Webster's Land, Edinburgh EH1 2RX, Scotland, UK. Tel: +44 (0)131 228 5235 Email: alastair.kinloch@btinternet.com
MasterWork Software Ltd	Fraser Jackson	PO Box 56-035, Tawa, Wellington, New Zealand. Tel: +64 (4) 232-4440 Fax: +64 (4) 232-4452 Email: fraser.jackson@xtra.co.nz
Mercia Software Ltd.	Gareth Brentnall	Mercia House, Ashted Lock, Aston Science Park, Birmingham, B7 4AZ, UK. Tel: 0121-359 5096. Fax: 0121-359 0375
MicroAPL Ltd.	Richard Nabavi	The Roller Mill, Mill Lane, Uckfield, E.Sussex TN22 5AA Tel: 01825 768050. Fax: 01825 749472 Email: MicroAPL@microapl.demon.co.uk
Milinta Inc.	Dan Baronet	Contact Dan Baronet at danb@milinta.com
Ellis Morgan	Ellis Morgan	Myrtle Farm, Winchester Road, Stroud, Petersfield, Hants GU32 3PE, UK. Tel: 01730-263843 Email: Ellis@mrtlrm.demon.co.uk
NY Sig	David Siegel	PO Box 2697, New York, NY 10163-2697, USA. Email: NYSIGAPL@ACM.ORG
Oasis b.v.	Louis Rijkse	Lekstraat 4, 3433 ZB Nieuwegein, Holland. Tel: +31 30 60 66 336 Fax: +31 30 60 65 844 Email: rijkse@oasis.nl
Omega Computing Inc	Alan Graham, Andrew Chou	3 Columbus Avenue, Edison, NJ 08817, USA. Tel: +1 (732) 985 9519 Email: alangraham@gmindspring.com
Optima Systems Ltd	Paul Grosvenor	Optima House, Mill Court, Spindle Way, Crawley, West Sussex, RH10 1TT, UK. Tel: 01293 562700 Fax: 01293 562699 Email: paul@optima-systems.co.uk
Qualed Inc.	Nicole Schless, Georges Brigham	121 West Main Street, Milford, CT 06460, USA. Tel: +1 (203) 874-4334 Fax: +1 (203) 876-9083. Email: sales@qualedi.com; info@qualedi.com
RadSys Technologies AB	Randolph Schrab	Lovsångarv. 18, S-756 52 Uppsala, Sweden. Tel: +46 18 32 41 53 Fax: +46 708 1995 11 Email: randolph.schrab@radsys.se
Renaissance Data Systems	Ed Shaw	P.O. Box 511, Botsford, CT 06404, USA. Tel: +1 (203) 270-9729 Email: orders@aplbooks.com
Resources & Results	Frank Rhodes	2438 W. Northgate Dr., Irving, TX 75062, USA. Tel: +1 972 523 5117 Email: frank@decision-support.net
RE Time Tracker Oy	Richard Eiler	Mikonkatu 8 A, 2.krs, PL 363, 00101 Helsinki, Finland. Tel: +358 9-621 3300 Fax: +358 9-621 3378 Email: re@rett.fi
Right Seat Software, Inc.	Tom Atkins	1110 12th Street, Golden, CO 80401, USA. Tel: +1 303 278 2244. Fax: +1 303 278 6967. Email: info@voxproxy.com
The Rochester Group Inc.	Robert Miller	600 Park Avenue, Rochester, NY 14607-2926, USA. Tel: +1 (716) 271-1110. Fax: +1 (716) 271-1230
Rome/Italy S/G	Mario Sacco	Casella Postale 14343, 00149-Roma Trullo, Italy Email: mario.sacco@tin.it
SE APL Users Group	John Manges	413 Comanche Trail, Lawrenceville, GA 30044, USA Tel: +1 (770) 972-3755 Email: seapl@comcast.com
Shepp & Associates LLC	Andrew Shepp	1312 Washington Avenue, 6th Floor St. Louis MO 63103, USA Tel: +1 (314) 621-3272 Fax: +1 (314) 621-4267 Email: ashepp@compuserve.com
Snake Island Research Inc	Bob Bernecky	18 Fifth Street, Ward's Island, Toronto, Ontario M5J 2B9 Canada Tel: +1 (416) 203-0854 Fax: +1 (416) 203-6999 Email: bernecky@interlog.com
SOCAL (South California)	Roy Sykes Jr	Sykes Systems Inc, 4649 Willens Ave, Woodland Hills, CA 91364-3812 USA. Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Soliton Associates	Benoit Paquin	Soliton Denmark, Havsgårdsvej 4, 2900 Hellerup, Denmark Email: sales@soliton.com

SovAPL	Alexander Skomorokhov	PO Box 5061, Obninsk-5, Kaluga Region 249020, Russia
Russian Chapter of SIGAPL		Tel: +7(08439)47109 Fax: +1 (530) 6885510 Email: askom@obninsk.com
Strand Software Inc	Anne Faust	P.O. Box 330, Excelsior, MN 55331 USA
		Tel: +1 (952) 470-7345 Fax: +1 (952) 470-9202 Email: info@strandsoft.com
Rex Swain	Rex Swain	8 South Street, Washington, CT 06793 USA. Tel: +1 (860) 868-0131
		Fax: +1 (860) 868-9970 Email: rex@rexswain.com
SwedAPL	Christer Ulthielm	Novator Consulting Group AB, Svärdvägen 11C, S-182 33 Danderyd
		Sweden. Tel: +46 8 622 63 50 Fax: +46 8 622 63 51 CServe: 100341,404
Swiss APL User Group		Swiss APL User Group, CH-3001, Bern 1, Switzerland
		Email: sl@ifi.unizh.ch
Sykes Systems Inc	Roy Sykes Jr	4649 Willens Ave., Woodland Hills, CA 91364, USA
		Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Toronto SIG	Richard Procter	PO Box 55, Adelaide St. Post Office, Toronto Ontario M5C 2H8, Canada
		Email: info@torontoapl.org
Weighahead Systems	Philip Bulmer	Camberley House, 1 Portesbery Road, Camberley, GU15 3RB, UK.
		Tel +44 1276 20789 Email: sales@weighahead.com
Stephen Wynn		8 Clarence Gardens, Brighton, Sussex BN1 2EG, UK.
		Tel: 01273-327236 Email: centre@cwcom.net
Zark Incorporated	Gary A. Bergquist	23 Ketchbrook Lane, Ellington CT 06029, USA. Tel: +1 (860) 872-7806

FTP SITES

IBM APL2	ftp.software.ibm.com/ps/products/apl2
Waterloo Archive	archive.uwaterloo.ca/ftparch/languages/apl
APL-to-ASCII	archive.uwaterloo.ca/languages/apl/workspaces/aplascii

WORLD WIDE WEB SITES

ACM SigAPL	www.acm.org/sigapl/
Adapta Software	www.adapta.de/
Adaptable Systems	www.assuredsystems.com.au/
Adaytum Limited	www.adaytum.com/
AFAPL	www.afapl.asso.fr/ (Journal available on line)
APL2000	www.APL2000.com/
APL-385	www.demon.co.uk/apl385/
APL Journal, Germany	www.rhombos.de/apljourn.htm
APL Borealis Inc.	www.aplborealis.com
APL Systems IDC SL	www.apl2000.net
AUSCAN	www.interlog.com/~rjp/auscan/
Eke van Batenburg	www.bio.LeidenUniv.nl/~Batenburg/index.html
Capital PC User Group	http://cpcug.org/
Causeway	www.causeway.co.uk/
CODEWORK	www.codework-it.com/tangram/eng/
CoSy (Bob Armstrong)	CoSy.com/
Dinosoft Oy	www.dinosoft.fi/
Dittrich & Partner	www.dpc.de ; www.apl-online.de
DMOZ - Open Directory	http://dmoz.org/Computers/Programming/Languages/APL/
Dyadic Systems Ltd	www.dyadic.com/
DynArray	www.dynarray.com/
FinnAPL	www.pyr.fi/apl/
Godin London Inc	www.godin.com/
Houben (IQL)	www.apl.olap.club.tip.nl
Hoekstra Systems	www.HoekstraSystems.ltd.uk/

IBM APL2	www.ibm.com/software/ad/apl
Infostroy	www.infostroy.ru
Insight Systems ApS	www.insight.dk/
Japan APL Association	www.naska.co.jp/JAPLA/
JSoftware Inc	www.jsoftware.com/
KJK-tieto Oy	www.kjk-tieto.com
Lambent Technology	www.lambenttechnology.com
Lescasse Consulting	www.lescasse.com/
Lingo Allegro USA Inc	www.lingo.com/
Mackay Kinloch	http://mackaykinloch.ltd.uk/
MicroAPL Ltd	www.microapl.co.uk/apl
Milinta Inc	www.milinta.com
Oasis b.v.	www.oasis.nl/
Optima Systems Ltd	www.optima-systems.co.uk
Qualedi, Inc.	www.qualedi.com
Renaissance Data	www.aplbooks.com/
Resources & Results	www.decision-support.net
RE Time Tracker Oy	www.rett.fi/
Right Seat Software, Inc.	www.voxproxy.com
The Rochester Group Inc.	www.rochgrp.com/
Shepp & Associates	www.digitravel.com/
SigAPL	www.acm.org/sigapl/
Soliton	www.soliton.com/
Snake Island Research Inc.	www.snakeisland.com
Strand Software Inc.	www.strandsoft.com/
Rex Swain	www.rexswain.com/
Toronto SIG (for Toolkit)	www.torontoapl.org/
Jim Weigang	www.chilton.com/~jimw/
Weighahead Systems	www.weighahead.com

Vector Back Numbers

Back numbers of Vector are available from:

**British APL Association,
c/o Gill Smith,
Brook House, Gilling East,
YORK YO62 4JJ**

Price in UK: £10 per complete volume (4 issues);
£12 (overseas); £16 (airmail) including postage.

Please note that Vol.1 No.2 is now out of stock.

ZARK Newsletter Extracts

introduced by Jon Sandles

Here is another Zark newsletter extract. As always, don't hesitate to send in any solutions you may have that you may be inspired to investigate. There may be new more elegant ways of solving an old problem, which may also turn out to be faster. Please don't be afraid to try and shed new light on the problems being discussed in this column.

Utility Corner: Effective Date Searching

(The purpose of this column is to make you more productive by introducing you to utility functions. Think of utility functions as APL functions that have names instead of symbols. By expanding your function vocabulary, you'll be able to write APL code that's more concise, more efficient, and more readable.)

Suppose you have two numeric vectors of the same length, *IDS* and *PRICES*. *IDS* is a vector of distinct product ID codes for all available products; *PRICES* is a vector of corresponding process for these products. Given a subset list of product ID codes, say *LIST* you can easily determine their corresponding process:

PRICES[IDS⍷LIST]

Suppose prices change over time. Each price, for a given product, has an associated effective date. For a given product, there may be many different prices, each with a different effective date. To determine the price of a given product, you need to specify not only the product's ID code, but also the date as of when you want its price.

Here's an illustration:

<u>Product Id</u>	<u>Price</u>	<u>Effective Date</u>
204	2.65	1993/01/01
	2.75	1993/07/01
	2.89	1993/09/31
202	15.50	1993/01/01
	16.75	1993/10/15

The price of product 202 on 1993/09/25 is 15.50. The price of product 204 on 1993/10/15 is 2.89.

In last issue's Limbering Up column, you were asked to design and define a utility function that takes a list of product IDs and corresponding dates, and return the prices for those products on those dates. We were looking for two things: clean design, and program efficiency.

The responses we received consistently suggested storing the database of price and date information in a single APL matrix. If the product ID is known to be numeric, you can construct a three-column matrix that contains the necessary information:

```
204 265 1993 101
204 275 1993 701
204 289 1993 931
202 1550 1993 101
202 1675 1993 1015
```

However, if prices are larger than 32767 the matrix will go floating point anyway.

If the product ID may contain non-digits, the database can be constructed as a character matrix:

```
G204 26519930101
G204 27519930701
G204 28919930931
B202155019930101
B202167519931015
```

In this case, no blank columns are required between the fields of information. However, you may need to include a few extra blank columns in the price field to accommodate the largest possible price.

Having designed the database of effective price information as a single APL matrix, you can design an array of the target information along the same lines. For example, if you want the prices for product 202 on 1993/09/25 and 204 on 1993/10/15, the target array can be:

```
202 19930925
204 19931015
```

or

```
202 1993 925
204 1993 1015
```

or

```
820219930925
G20419931015
```

Given a database matrix and a target matrix, the syntax of a utility function suggests itself:

```
VPRICES←DB EFFECTIVE TARGET
```

The most straightforward and efficient solutions came from Jim Weigang and Dan Thompson. Jim assumed numeric matrices; Dan assumed character matrices. The key step in both solutions was to row-wise catenate the matrices and grade the result.

Here is a function that works for three-column numeric matrix databases, and that combines the best of Jim's and Dan's functions:

```

V R←DB EFFECTIVE TARGET;D;DI;E;G;M;N;T;TI;IO
[1] A Searches the 3-column num matrix DB
[2] A (ID price, eff. date [yyyymmdd])
[3] A for the rows that correspond to the
[4] A rows of the 2-column num matrix
[5] A TARGET(ID, date), and returns the
[6] A prices in effect on the target
[7] A date. For example, given DB:
[8] A      204 265 19930101
[9] A      204 275 19930701
[10] A      204 289 19930931
[11] A      202 1550 19930101
[12] A      202 1675 19931015
[13] A and TARGET:
[14] A      202 19930925
[15] A      204 19931015
[16] A the result is 1550 289. The result
[17] A contains 0s for rows of TARGET with
[18] A no matching row in DB. TARGET may
[19] A be a 2-element vector.
[20] A
[21] IO←1
[22] A Make TARGET a matrix if a vector:
[23] →(M←2=ppTARGET)pL1
[24] TARGET←(1,pTARGET)pTARGET
[25] A Squeeze out unneeded DB rows:
[26] L1←DB←(DB[;1]εTARGET[;1])÷DB

```

```

[27] A Make sure a DB row winds up first
[28] A after sorting:
[29] DB←~1,[1]DB
[30] A Vatenate and sort them to merge:
[31] G←A DB[;1 3],[1]TARGET
[32] N←1↑pDB A No. of recsin DB
[33] T←D+G≤N A Those from TARGET and DB
[34] A Indices into DB for each row of
[35] A TARGET (as sorted):
[36] DI←(D/G)[T/+D]
[37] A Corresponding indices into TARGET:
[38] TI←(T/G)-N
[39] A Flag those with a match:
[40] E←DB[DI;1]=TARGET[TI;1]
[41] A Construct result, one elt per row
[42] A of TARGET
[43] R←(1↑pTARGET)ρ0
[44] R[E/TI]←DB[E/DI;2]
[45] A Make R scalarif TARGET a vector:
[46] →Mp0
[47] R←R[1]

```

▽

Let's show how this function works, given the following arguments:

```

DB:      204  265 19930101
          204  275 19930701
          204  289 19930931
          202 1550 19930101
          202 1675 19931015
TARGET:  202 19930925
          204 19931015

```

This function allows *TARGET* to be a two-column matrix or a two-element vector. The result is a vector if the former, or a scalar if the latter. On lines [23] and [24], *TARGET* is reshaped to a one-row matrix if it is given as a two-element vector. By reshaping *TARGET*, you can treat it as a matrix for the remainder of the function.

On line [26], *EFFECTIVE* squeezes down the size of *DB* to retain only those rows that correspond to products with IDs in *TARGET*. If *DB* is large and *TARGET* is relatively small, this step can speed up the function considerably by reducing the number of rows to be sorted. In this example, *DB* keeps all its rows since both products are represented in *TARGET*.

On line [29], a row of $\bar{1}$ s is catenated to the top of *DB*:

```
DB:  -1  -1  -1
      204 265 19930101
      204 275 19930701
      204 289 19930931
      202 1550 19930101
      202 1675 19931015
```

This step is performed to guarantee that there is always a row of *DB* preceding each row of *TARGET*, once *DB* and *TARGET* have been merged. You'll see why below.

On line [31], the product numbers and dates from both *DB* and *TARGET* are catenated and graded. This is the workhorse line of the function. Though the rows of *DB* and *TARGET* are never reordered via the grade vector, let's show such a reordering to help you see what's happening:

			G	D	T	+ \ D
-1	-1	-1	1	1	0	1
202	1550	19930101	5	1	0	2
202		19930925	7	0	1	2
202	1675	19931015	6	1	0	3
204	265	19930101	2	1	0	4
204	275	19930701	3	1	0	5
204	289	19930931	4	1	0	6
204		19931015	8	0	1	6

Notice the rows are sorted by product code, and within product code by date. The two rows of *TARGET* have been automatically inserted directly below the corresponding rows of *DB* that contain the desired prices.

Lines [32] and [33] flag the rows of the sorted result that correspond to rows from *DB* (in *D*) and from *TARGET* (in *T*).

Lines [36] and [38] use the values from *G*, *D*, and *T* to construct vectors of indices of the target rows into the *DB* row above them (in *DI*) and into the original unsorted *TARGET* matrix (in *TI*):

```
DI:  5 4      TI:  1 2
```

For example, these values tell you that the row *DB*[5 ;] (202 1550 19930101) contains the corresponding price for row *TARGET*[1 ;] (202 19930925). Likewise, row *DB*[4 ;] (204 289 19930931) corresponds to row *TARGET*[2 ;] (204 19931015).

Line [40] compares the product codes in the matching rows to make sure *DB* does indeed contain an effective price for the corresponding row of *TARGET*. In this case, they do, so *E* is 1 1. However, if the product code in *TARGET* is not found in *DB*, or if the date in *TARGET* is earlier than the earliest effective date for that product in *DB*, *E* will contain 0s.

The remaining lines of the function construct the result *R* inserting the matching prices where they belong, and leaving 0s for rows of *TARGET* with no match. Finally, the result is converted to a scalar if *TARGET* was given as a two-element vector instead of a matrix.

As written, this function is general enough to allow for:

- rows of *DB* that are duplicates, or are not in sorted order;
- rows of *TARGET* that are duplicates, or have no matching product code in *DB*, or have no earlier date in *DB*.

The modifications needed to make *EFFECTIVE* work on character matrix arguments or on numeric matrices with multiple columns for dates are simple, and are left as an exercise.

Limbering Up: Exporting Numbers

(The purpose of this column is to work some flab off your APL midsection. Like muscles, your APL skills can atrophy if not exercised with adequate frequency and variety. This column presents a task for you to perform. Set aside a few minutes from your busy schedule and work the task. Mail in your solution and stay tuned for the results.)

APLers tend to live in their own little societies, secluded from the rest of the world and beholden to no one. However, reality being what it is, the need does occasionally arise to communicate with others. Sometimes, you want some data that others have, or they want some data that you have.

How do you move a copy of the data from them to you, or vice versa?

Suppose, for example, the data in question is a bunch of numbers, which you would view as a numeric matrix, and which they would view as a range of spreadsheet cells. They might suggest the numbers be transferred as a "comma-delimited text file". You need to construct a text (ASCII) file that contains the numbers in this format:

```
123,5000,60,90793
124,400,60,31593
125,39900,71,42293
```

If you want to send them the numbers, you must be able to convert your numeric matrix to a file in this format. If you want to receive numbers from them, you must be able to convert a file in this format to a numeric matrix.

Your task is to define utility functions that perform these two tasks. Call the functions *TOCOMMA* and *FROMCOMMA*. *TOCOMMA* is a monadic function whose right argument is a numeric matrix to be converted to a comma-delimited file, and whose result is a character vector. The character vector contains the formatted numbers (formatted with monadic *⌞*) separated by commas, and the rows separated by a carriage return and a line feed (*␣␣*). For example,

```

      NMAT←3 4p123 5000 60 90793 124 400 60 31593 125 39900 71 4
2293
      TOCOMMA NMAT
123,5000,60,90793

124,400,60,31593

125,39900,71,42293
```

(The lines appear double-spaced because of the inclusion of both carriage return and line feed characters. ASCII files typically contain both of these characters at the end of each line.)

FROMCOMMA does the inverse of *TOCOMMA*. Given a comma and carriage return/line feed delimited character vector as its argument, it returns the numeric matrix of the numbers within the character vector. For example,

```

      CVEC←TOCOMMA NMAT
      FROMCOMMA CVEC
123 5000 60 90793
124 400 60 31593
125 39900 71 42293
```

Obviously, *TOCOMMA* and *FROMCOMMA* do not solve the problem entirely. The character vector must still be written to or read from a file. In APL*PLUS PC or APL*PLUS II, this can be accomplished as follows:

Creating and writing a file:

```
'C:\FILENAME.TXT' ⎕NCREATE ~1
CVEC ⎕NAPPEND ~1
⎕NUNTIE ~1
```

Reading a file

```
'C:\FILENAME.TXT' ⎕NUNTIE ~1
X+⎕NREAD ~1 82, ⎕NSIZE ~1
⎕NUNTIE ~1
```

The functions should be as efficient (fast) as possible.

Send your solution to:

Vector Production
Brook House
Gilling East
York YO62 4JJ
UK email apl385@compuserve.com

The notable functions and their authors' names will be printed in the next issue of *Vector*. Good luck and happy limbering.

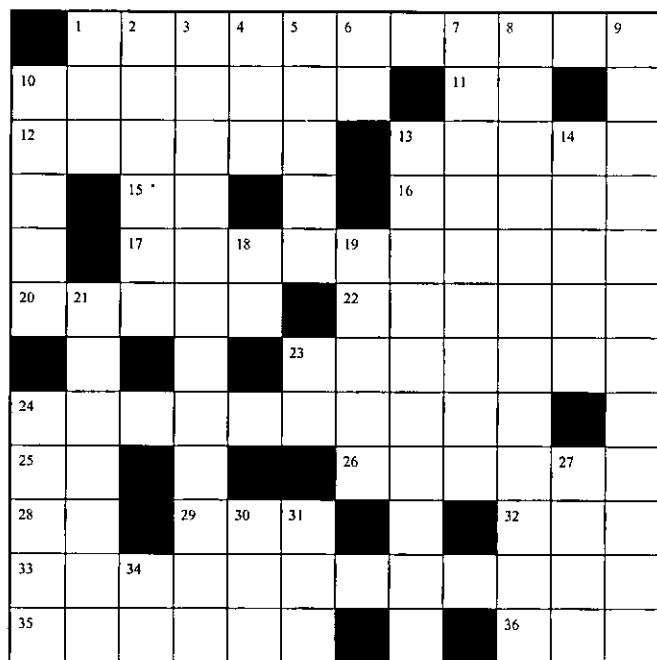
Reprinted with kind permission from Zark APL Tutor News, a quarterly publication of Zark Incorporated, 23 Ketchbrook Lane, Ellington, CT06029, USA.

New Crossword and Solution for 19.2

Solution to Crossword in 19.2

A	~	B			V	ι		1	?	N	
	0		(	<	N	R	)	ε	N	S	T
0	ε	~	2	9	↓	B		..			D
≠		T	5	ρ	Z			(		J	
□	I	O	▷	1	↑	0	ρ	<	A	B	S
N	^	P	A		V	=	▷	C	P		C
C	~		)		E	≡	N	V	L	▷	A
'		N	ι	<	C	V		)		,	L
V	,	¶	Q	φ	T			ε		/	A
E	M			N	S	T	≡	..	<	N	R
C		ρ	E	V		┘	N	N	M	S	
'	'	ρ	X		)	S	A	V	E		

Crossword



Across

1. Flag last of each group of equal values in sorted positive integer vector *PIV*.
10. Indices into *VEC* of the first *N* elements of vector *M*.
11. $\square$, the workspace's elbow room.
12. Set *S* to be the lesser of *R* or *RE*.
13. ____*A* to put the elements of *A* in the shape of *M*.
15. Product.
16. Add 2 to each element of *###ADV*.
17. A random integer from 1 to the length of *STRE/V*, for scalar *STRE*.
20. Format the elements of *RVO* to one decimal place.
22. Stick *D* and *OC* together and reshape to a matrix of shape *S*.
23. Magnitude of product of *AV* and *APT*.
24. Index into *NV* of the number following the first non-nine.
25. *R* times pi.
26. Shift the elements of vector *R* to the left *BQ* places and reshape to shape *E*.
28. *S###+ \###IO*, *Sp1* for scalar *S*.

29. Is execution suspended?
32. Stick *TL* onto...
33. The largest number of leading blanks in any row of *CM*.
35. $(1#####3) + . * (###1#####3)$, origin 1.
36. COBOL it's not.

Down

1. *PE* gets...
2. *ICR*, scaled by a random integer from 1 to *R*.
3. Index into *V* of the smallest number above 99.
4. ... unlike *NR*.
5. Select the first element of *E*, or of *V* if *E* is empty.
6. Vector of vectors from matrix *M* (in the original nested APL*PLUS).
7. $(-SD), (-2###SD), (-3###SD), \dots, (-E###SD)$, reshaped to length *IW*.
8. Onto bit matrix *VAMAT*, stick a new column flagging the elements of vector *A1QT* that match the scalar *A*.
9. Are both vectors *VECTOR* and *ML* empty?
10. All but the first *VS* elements of vector *V1*.
13. Expansion vector to put a blank after each character in the vector *V*.
14. Eliminate a stored workspace.
18. Empty vector, without using *⍒*.
19. Preserve the active workspace.
21. Format $(T+L)[RS/#####RS]$, for scalar *T*.
23. Absolute value of *N*.
24. Sin of whole radians up to the integer at or above *E*.
27. *LCP*, backwards.
30. Are both *S* and *E* true?
31. Array that returns vector *N* when compressed with bit vector *I*.
34. *E*.

J-ottings 35: How to Do Things with Words

by Norman Thomson

David Ness recently drew my attention to a fascinating little book called "How to Do Things with Words". This is an edited version of lectures given in 1955 at Harvard by an Oxford philosopher called John Austin, but the title is equally appropriate as a succinct answer to an enquirer seeking to discover what J is all about. The gist of the book is that there are some things in life which can only be done by making utterances, "I name this ship ...", "I baptise this baby ..." and so on. The pursuit of analogies with the grammars of natural languages has always been one of the driving forces of J, and it is not accidental that what in other languages are called "functions" are called "verbs" in J, while "constants" and "variables" are called "nouns".

Observers of the jsoftware forum will know about a new charged-for version of J called J5.01 which has just been introduced and which heralds J's maturity as a computer product widely used on many different platforms and within a great many environments. The increase in reported problems, some presumably more significant than others, is an inevitable part of this complexity, and demonstrates how the language interpreter has shrunk to a disproportionately small part of the total J structure. The topics in Strand Software's advanced course in New York last autumn were things like OCX, ODBC, sockets and so on. By contrast, the predecessor version called J4.06d with its near perfect interpreter remains downloadable and free to all, and in Delphic style, provides answers to questions about what happens when words are put together in combinations which Roger himself could not possibly have ever conceived. While it seems that an increasing number of computer specialists are now using J in conjunction with the many other products and adjuncts of modern computing systems and will thus sign up for J5.01, the perfection of J4.06d looks likely to remain a *de facto* standard for those whose quests are restricted to what are perhaps more enjoyable and less frustrating linguistic endeavours.

In its rigorous judgements of J competence, the J4.06d interpreter contrasts with the manner in which "perfect" English is not an absolute standard but an inherently unattainable ideal. When my German born travel agent answers my question "Is that Brigitte?" with the response "This is she", by every formal rule of English grammar this has to be judged totally correct, but not many native English speakers would instinctively consider it so. On the other hand Roger's

creation answers all possible questions of grammar and syntax (including millions which have not yet been asked) with total certainty and clarity. It is in effect a super-intelligence to which even the most accomplished J "speakers" must submit an implicit validity test with every J phrase which they utter or compose. Such speakers rarely react to an error report or unexpected result by presuming a bug is present, but rather accept that their instinctive and developing notions of the language do not yet quite measure up to the performance ideals of the unforgiving language examiner!

Much was made of APL as a notation for thought, and of the idea that, to quote A.N. Whitehead, "by relieving the brain of unnecessary work a good notation sets its users free to concentrate on more advanced problems thereby increasing the powers of the race". It might be supposed that this notion carried over into J, but this is not entirely correct. APL is first and foremost a shorthand, J only incidentally so. At an early age we develop fluency in the language we hear all around us, and we rapidly achieve *speech* as opposed to language skills. For example the child who says "throw Daddy" has acquired a *language* skill in combining words in a fashion which mimics the rhythm of the social interactions he hears in the adult speech surrounding him. The *speech* skill by which this utterance develops into "throw it to Daddy" comes at a later stage. Combining words in J and submitting them to an interpreter is like submitting "throw Daddy" to the full glare of perfect adult intelligence, leading to a possible unwanted response where a physically strong and highly spirited Mummy happens to be present!

This analogy must resonate clearly with anyone who has crafted and executed a J verb with a clear expectation of what it will perform. Surprise is frequently conveyed not by an error report, but by the delivery of *some* result, which, if carefully observed, can advance a general background goal of bringing the user's fluency in J closer to the perfection which only the super-intelligence can attain. In other words, reflective consideration of faulty utterances is the prime route towards the attainment of language mastery. It is all too easy to set such explorations aside without fulfilling good intentions to return and learn from the experience. Thus many J speakers never progress much beyond the "throw Daddy" stage, or at best graduate only as far as the language equivalent of infancy!

I said above that J, at least in the perceptions of some of us, is about doing things with words, and I return now to that theme in the context of the two types of rank which seem to be one of the principal stumbling blocks for initial language learners. The two types of rank are *noun rank* and *verb rank*. The former is a relatively simple idea to grasp, namely the result of applying $\#$ to an object to

give the tally of its shape, or, in a single word, its dimensionality. Shape itself has geometrical connotations, so that a scalar has rank 0, akin to a point in space, a list has rank 1 like a line, a matrix has rank 2 analogous to points in a plane, and so on.

Verb rank is somewhat more subtle, but has more than a passing association with the grammatical idea of "number", that is singular or plural. It is also helpful to consider rank alongside the concepts of *box* and *cell*, both of which are means of "singularising" things, the former in a much stronger way by disguising the nature of the contents from external agents such as verbs. The Object Oriented concept of encapsulation is highly relevant here, so that a box is a scalar whereas a cell is not, the key difference being the property of penetrability.

The majority of J verbs have rank of either zero or infinity. It is only for verbs which have special semantic requirements such as *shape* and *matrix inverse* that any other rank is associated with the verb. Basically, the "safe" assumption about the rank of an unknown verb is that it is infinite, which is why all but the most trivial user-defined verbs, are assigned infinite rank. (There are occasions where, with analysis, a "true" rank could in principle be worked out, but such computing effort would not only overload the interpreter for little practical gain, but would also cost greatly in performance.)

Simple addition at different rank levels involves thinking about cells with possibly complex list structures. Addition without explicit rank can penetrate list structures by applying the basic verb (that is the one to the left of the rank conjunction) at rank 0 :

```
x=.i.2 3 NB. a List of two 3-lists
y=:>:x NB. another list of two 3-lists
x;y;x+y
```

```
+-----+-----+-----+
|0 1 2|1 2 3|1 3 5|
|3 4 5|4 5 6|7 9 11|
+-----+-----+-----+
```

However, adding objects with different structures may result in shape incompatibility, for example in the following case

```

]z=.10*i.2 2 3 NB. two 2-lists of 3-lists
0 10 20
30 40 50
60 70 80
90 100 110
x+z NB. two 2-lists plus two 3-lists
|length error
| x +z

```

you cannot meaningfully add 3-lists to 2-lists! However, addition at rank 1 *is* valid

```

x+"1 z NB. add two lists to two lists of lists
0 11 22 NB. this is 0 1 2 + plane 0 of z ..
30 41 52 NB. ..with list 0 1 2 replicated.
63 74 85 NB. and this is 3 4 5 + plane 1 of z ..
93 104 115 NB. ..with list 3 4 5 replicated.

```

because it is an addition of two compatible 2-lists. Each of x's two lists is a 3-list, whereas for z, both its lists are 2-lists, each item of which is a 3-list, and it is at the level of the 3-lists that the actual addition takes place. Addition at rank 2 is also valid:

```

x+"2 z NB. add one list to two lists
0 11 22 NB. this is x + plane 0 of z ..
33 44 55
60 71 82 NB. .. and this is x + plane 1 of z
93 104 115

```

In this case, a single 2-cell is added to a list of 2-cells, and so in this case the left argument x is replicated in its entirety.

+ is one of a subset of verbs, mainly arithmetic, which "penetrate" indefinite layers of structure to operate at atomic (that is numeric or character) level, as in the case of $x+y$ above. In APL these are called scalar verbs, and their counterparts in J are said to possess rank 0. Where scalar verbs operate on conformable arguments as with $x+y$, increasing verb rank has no effect on the result. This is because, for example, $x+"1 y$ is the addition of two pairs of 1-cells and $x+"2 y$ is the addition of a single pair of 2-cells, and in each case penetration takes place down to atomic level. In J terms

```

(x+"1 y)-:x+y NB. rank 1 add matches rank 0 add
1
(x+"2 y)-:x+y NB. rank 2 add matches rank 0 add
1

```

Returning to *box*, whereas rank 0 verbs penetrate list structures, they do *not* penetrate boxed structures:

```
(<x)<z
|domain error
| (<x)    <z
```

In general, the permitted operations on boxed scalars are restricted to shaping and joining, since encapsulation means that the nature of their contents is not available to verbs. To repeat the warning given earlier, do not confuse cells and boxed scalars!

Applying adverbs to verbs with rank continues the same rigorous logic. For example applying *table* :

```
$x+/z      NB. All possible atomic pairs are added
2 3 2 2 3
$x+/"1/z   NB. A 2 by 2 table (items=3-lists)
2 2 2 3
$x+/"2/z   NB. 1x2 table(items=2-lists of 3-lists)
2 2 3
```

In the first case the shape of the result is the catenation of shapes. If this were not so it would be impossible to accommodate all possible number pairs. However, when two lists are added, addition takes place only between matching items, which means that in forming the second table the two 3s at the lowest shape level in each argument are merged. Similarly, in the last case it is the two 2 3s which are merged by addition to give a 2-list, each item of which has shape 2 3. If you want to confirm your understanding of all this, try to predict the results of $x+/"1 z$ and $x+/"2 z$. It helps to appreciate that "table addition" is not a scalar verb, that is, it has infinite rank. You can use verb rank to specify the level at which tables are to be formed so that, for example, $x+/"0 y$ is identical to $x+y$. However, no penetration takes place below the level specified by an explicit rank. Thus :

```
$x+/"1 z   NB. A 2 by 2 table of 3 by 3 tables
2 2 3 3
$x+/"2 z   NB. A 1 by 2 table of 2 by 3 tables
2 2 3 2 3
```

In the first case above the verb $+/$ operates item by item on two pairs of lists to produce a 2 by 2 structure of 3 by 3 matrices.

Now return to natural language. Verbs like "quit" and "bid" make no number distinction – "he quit, they quit" and so on – as opposed to verbs like "spell" – "he spells", "they spell" – which do. The concept of "plurality" with respect to verbs is

a little more subtle than it might at first appear. For example if I say (talking about a group of people) "they sang", it could be taken to mean that each individual in the group sang, that is, the singing was performed at rank 0. On the other hand, the statement "they sang" could equally be applied to, say, a crowd at a football match where it was almost certainly *not* the case that every single individual sang. On the other hand "they multiplied" (in the reproductive sense!) could not have been done by individuals, and so we have here a verb which whose rank is certainly greater than zero! Similarly, if an utterance like "they quit" meant that *all* of them did as a body, then we have an instance of infinite rank. At best English verbs allow the distinction between one and many, that is between singular and plural. Explicit verb rank in J allows all possible gradations in between, 0, 1, 2 and so on up to infinity, and further, it allows ranks to be specified separately for subject and objects, much as if English were to provide separate inflections for "he hitss her", "he hits them", "they hit them" and "they hitt him". With a commutative verb like + there is no difference in result between $x + "0 \ 1 \ y$ and $x + "1 \ 0 \ y$. However, the distinction is immediately apparent with a non-commutative verb :

$x - "0 \ 1 \ y$	NB. atoms of x minus rows of y
$\begin{array}{r} _1 \ _2 \ _3 \\ 0 \ _1 \ _2 \\ 1 \ 0 \ _1 \end{array}$	NB. 0 minus 1 2 3 NB. 1 minus 1 2 3 NB. 2 minus 1 2 3
$\begin{array}{r} _1 \ _2 \ _3 \\ 0 \ _1 \ _2 \\ 1 \ 0 \ _1 \end{array}$	NB. 3 minus 4 5 6 NB. 4 minus 4 5 6 NB. 5 minus 4 5 6
$x - "1 \ 0 \ y$	NB. rows of x minus atoms of y
$\begin{array}{r} _1 \ 0 \ 1 \\ _2 \ _1 \ 0 \\ _3 \ _2 \ _1 \end{array}$	NB. 0 1 2 minus 1 NB. 0 1 2 minus 2 NB. 0 1 2 minus 3
$\begin{array}{r} _1 \ 0 \ 1 \\ _2 \ _1 \ 0 \\ _3 \ _2 \ _1 \end{array}$	NB. 3 4 5 minus 4 NB. 3 4 5 minus 5 NB. 3 4 5 minus 6

In English we tolerate the absence of such inherent number precision, which makes explicitly ranked J verbs harder to grasp than English ones – but on the other hand what interesting and varied things one can do with them!

A Tutorial in J Programming: The Second-Order Josephus Sequence

*Howard A. Peelle (email: hapeelle@educ.umass.edu)
University of Massachusetts*

Abstract

An interesting sequence of non-decreasing positive integers serves as a focus for a tutorial on J programming. Six different approaches and accompanying algorithms are described, each with an overview of its purpose, relevant references, J code, syntactic structure and names of symbols, detailed explanation of the algorithm, examples of execution, and comments. The algorithms are compared for efficiency in speed and space. Concluding insights and programming challenges include a simple strategy for solving the Second-order Josephus problem.

Introduction

Consider the sequence

1 2 2 3 4 4 4 5 6 7 8 8 8 8 9 10 11 12 13 14 15 16 16 16 16 16 16 16 16

This sequence appears (with another leading 1) in [1] and is interesting because it lists answers to the "Second-order Josephus" problem (see below), recently posed by Boyko Bantchev in [2]. Related programs have been written by him and Eugene McDonnell in [2] and by Howard Peelle in [3].

The sequence is also instructive as a programming problem which invites different approaches and algorithms. Here we will present and compare several different J programs for producing this sequence – replete with detailed explanations of intermediate programming techniques. (For an introduction to programming in J, see [0] or appropriate books on the Jsoftware website.)

The Second-order Josephus Problem

The "Second-order Josephus" problem is derived from the basic "Josephus" problem ([4],[5],[6]) which is to determine the index of the survivor among N people in a circle where every second person is successively killed. For example, for N=9, numbers 2,4,6,8 (in index origin 1) are killed, then 1,5,9, then 7, leaving number 3 as the survivor.

The Second-order Josephus problem is to determine the index of the last survivor when those identified as survivors of the Josephus problem are successively killed. For example, for $N=9$, number 3 is killed, then (for $N=8$) 1 is killed, then similarly (for decreasing N) 9, 7, 5, 2, 8, 4, leaving number 6 as the last survivor.

The reader may wish to pause and write a program to solve this problem before reading on.

Utility Programs

To begin, here are some utility programs which will be needed.

Natural numbers

Purpose: Generate consecutive integers from 1 to input integer.

```
Natural =: }.@i.@>:
```

Symbols/Syntax: *Natural* is (Behead Atop Integers) Atop Increment. Note that the Atop conjunction (@) applies the entire verb train on its left after the verb immediately on its right.

Explanation: For input integer y , first add 1. Then generate a list of integers 0 to y . Finally, drop the first item and return the list of integers 1 to y .

Example: $>:9$ is 10. Then $i.10$ is 0 1 2 3 4 5 6 7 8 9. Finally, $}.0$ 1 2 3 4 5 6 7 8 9 is 1 2 3 4 5 6 7 8 9. Using the program:

```
Natural 9
1 2 3 4 5 6 7 8 9
```

Comment: A shorter definition is *Natural* =: >:@i. but it is slightly less efficient for large inputs.

Sort

Purpose: Sort a list of numbers into ascending order.

```
Sort =: /:~
```

Symbols/Syntax: *Sort* SortUp Reflex. Note that adverb Reflex (syntax *u~*) applies to a dyadic verb *u* with input y on both sides, that is, y *u* y .

Explanation: For an input list y , $/:\sim y$ is $y/:y$, which returns the items of y in the order specified by y – that is, the order which would sort y ascendingly.

Example: $/:\sim 1\ 2\ 3\ 4\ 5\ 6\ 7\ 2\ 4\ 4\ 8\ 8\ 8\ 8$ is $1\ 2\ 2\ 3\ 4\ 4\ 4\ 5\ 6\ 7\ 8\ 8\ 8\ 8$. Using the program:

```
Sort 1 2 3 4 5 6 7 2 4 4 8 8 8 8
1 2 2 3 4 4 4 5 6 7 8 8 8 8
```

Comment: An alternative definition is $\text{Sort} = :] / :]$ where $]$ is the input.

Two-Power

Purpose: Compute powers of 2.

```
Power =: 2&^
```

Symbols/Syntax: **Power** is 2 With Power. Note that the With conjunction ($n\&v$) uses a noun n on one side (here 2) and a verb v on the other side (here dyadic primitive Power $\wedge$) to create a monadic verb.

Explanation: 2 raised to the power of input y .

Example: $2\&^ 4$ is 16. Using the program:

```
Power 4
16
```

Comments: **Power** allows a list input: $\text{Power } 0\ 1\ 2\ 3$ is $1\ 2\ 4\ 8$.

An alternative definition is $\text{Power} =: 2 : \wedge]$ where $2 :$ is the constant verb **Two** and $]$ is the input.

Log

Purpose: Compute base-2 logarithms.

```
Log =: 2&^.
```

Symbols/Syntax: **Log** is 2 With Logarithm. Note that the With conjunction ($n\&v$) uses a noun n on one side (here 2) and a verb v on the other side (here dyadic primitive Logarithm $\wedge.$) to create a monadic verb.

Explanation: Base-2 logarithm of input y .

Example: $2 \wedge .16$ is 4. Using the program:

```

      Log 16
4

```

Comment: Log allows a list input: Log 1 2 4 8 is 0 1 2 3.

An alternative definition is `Log =: 2: ^. ]` where 2: is the constant verb Two and] is the input.

Each Atom

Purpose: Apply a verb (or program) to each atom of input.

```
EACH =: "0
```

Symbols/Syntax: EACH is Rank Zero. Note that EACH is an adverb created from the Rank conjunction (u"n) with the noun 0 on its right. Verb u will be input on the left of EACH.

Explanation: For u EACH y, verb u is applied to each atom of y.

Example: `i."0 (1 2 3 4)` is list `i.1`, list `i.2`, list `i.3`, list `i.4` arranged in a table, with 0 fill elements. Using the program:

```

      i. EACH 1 2 3 4
0 0 0 0
0 1 0 0
0 1 2 0
0 1 2 3

```

Comment: A different definition is needed for boxed nouns. (See next utility program.)

Each Box

Purpose: Apply a verb (or program) to each item of input in boxes.

```
each =: &.>
```

Symbols/Syntax: each is Under Open. Note that each is an adverb created from the Under conjunction (u&.v) with the Open verb (>) on its right. Verb u will be input on the left of each.

Explanation: For u each y, verb u is applied to each item of Open y and then each result is Boxed (the inverse of Open).

Example: $i. \&. > 1\ 2\ 3\ 4$ is boxed list $i. 1$, boxed list $i. 2$, boxed list $i. 3$, boxed list $i. 4$ arranged in a list of boxes. Using the program:

i. each 1 2 3 4

0	0	1	0	1	2	0	1	2	3
---	---	---	---	---	---	---	---	---	---

Comment: In the result above there are four items, each containing a list in a box.

J Program for the Josephus Problem

Here is a program (verb) in J to compute the survivor number for the Josephus problem using a well-known algorithm ([4],[5]):

S =: 1&|. &. #:

Symbols/Syntax: S is (1 With Rotate) Under AntiBase. Note that the Under conjunction ($u \&. v$) executes verb v, then verb u, and finally the inverse of v.

Explanation: Represent integer input y in binary digits. Then rotate one bit from the front to the back. Then return its base-2 value.

Examples: #:9 is 1 0 0 1. Then $1 \&|. 1\ 0\ 0\ 1$ is 0 0 1 1. Finally, #: 0 0 1 1 is 3. Using the program, for 9 people in a circle, the survivor is in position 3:

S 9

3

Comments: The reader should be cautioned that the result is in origin 1 while J uses origin 0. And, unfortunately, this neat verb does not generalize to the original Josephus problem (stated in [4]) using an interval of three nor to other intervals.

Programs for the Second-order Josephus Problem

Now, six different approaches and accompanying algorithms will be described for solving the Second-order Josephus problem.

1. Simulation Approach

The problem can be solved by simulation – that is, by starting with a list of integer indices and successively eliminating the index of the Josephus survivor (computed by program S above).

In [2] Bantchev provided a verb $E =: ((<:@\{.\}),.)\sim S@#$ to eliminate the ($S \#y$)-th item from any list y . This verb could be rewritten as:

$$E =: ({}):@{. , }.\sim S@#$$

Symbols/Syntax: E is (((Curtail Atop Take) Append Drop) Commute) (S Atop Tally). Note that E is a “hook”($g h$) which applies monadic verb h to the input and then dyadic verb g to the input and that result; that is $y g h y$ here. The adverb $\sim$ is used to switch the inputs of g so that $S@#$ becomes the left input and y the right input. Note also that the verb inside the parentheses is a “fork”($f g h$) which applies the central verb g to the results of verb f and verb h ; that is, $(f y) g (h y)$ here.

Explanation: Remove the last item of the first $S@#$ items taken from list y . Then append it to the list of $S@#$ items dropped from y .

Example: For an input list $y =: 1 2 3 4 5 6 7 8 9$, $\#y$ is 9 and $S \#y$ is 3 (as shown above). $3\{.y$ is 1 2 3 and $\}.y$ is 4 5 6 7 8 9. Appending $({}):3\{.y), (3\}.y$ gives the result 1 2 4 5 6 7 8 9. Using the program:

```
E Natural 9
1 2 4 5 6 7 8 9
```

Comment: Note that this verb uses S which uses origin 1 indexing.

Peelle [3] previously offered a more succinct verb to eliminate item $S \#y$:

$$E =: -. <:@S@# \{]$$

Symbols/Syntax: E is input Less (((Decrement Atop S) Atop Tally) From Input). Note that E is a hook ($g h$) where h is a fork ($f1 g1 h1$), so execution is $y -. ((<:S\#y) \{ y)$.

Explanation: Remove from input list y the item in the (origin 0) position one less than j of the length of y .

Example: For input list $y = 1\ 2\ 3\ 4\ 5\ 6\ 7\ 8\ 9$, # y is 9, then $S\ 9$ is 3, and $<:3$ is 2. Then $2\{y$ is 3. Then $1\ 2\ 3\ 4\ 5\ 6\ 7\ 8\ 9\ -. \ 3$ is the result, as shown by using the program:

```

      E Natural 9
1 2 4 5 6 7 8 9

```

Comment: Input y should be restricted to unique items (as in the example here).

Repeating E simulates the Second-order Josephus problem. For example:

```

      E Natural 9
1 2 4 5 6 7 8 9
      E E Natural 9
2 4 5 6 7 8 9
      E E E Natural 9
2 4 5 6 7 8
      E E E E Natural 9
2 4 5 6 8
      E E E E E Natural 9
2 4 6 8
      E E E E E E Natural 9
4 6 8
      E E E E E E E Natural 9
4 6
      E E E E E E E E Natural 9
6

```

This can be accomplished in one expression:

```

      E^:8 Natural 9
6

```

Symbols/Syntax: (E Power 8) (Natural 9). Note that the Power conjunction ($u^:n$) here uses verb E on the left and noun 8 on the right. *Natural 9* is the input for $E^:8$. Note that the input could be entered directly, as in $E^:8\ (1\ 2\ 3\ 4\ 5\ 6\ 7\ 8\ 9)$ or using Right (J) to separate 8 from the input numbers, as in $E^:8\ J\ 1\ 2\ 3\ 4\ 5\ 6\ 7\ 8\ 9$.

Explanation: E is repeated 8 times on its own result, starting with the natural integers 1 to 9 input. Note that this illustrates how to do iteration in J .

Example: For input $y=9$, *Natural 9* is $1\ 2\ 3\ 4\ 5\ 6\ 7\ 8\ 9$, and $E^:8\ \text{Natural } 9$ is 6 (as shown above).

Comment: This could be embodied in a master program.

Bantchev [2] offered a master verb $S2 =: E^{\wedge} : (< : @ \#) @ (> : @ i .)$. An alternative definition, using a gerund, is:

$S2a =: E^{\wedge} : (< : \text{` Natural})$

Symbols/Syntax: $S2a$ is $E^{\wedge}$ Power (Decrement Tie Natural). Note that a gerund is a boxed noun containing verbs. Here, the gerund produced by ``` allows the input for $\wedge$ to be entered along with the input y for the derived verb $E^{\wedge} : < :$.

Explanation: Repeat E one less number of times than input y , starting with input integers 1 to y .

Example: For input 9, $< : 9$ is 8 and `Natural 9` is 1 2 3 4 5 6 7 8 9. $E^{\wedge} : 8$ (1 3 2 3 4 5 6 7 8 9) is 6 (as shown earlier). Using the program for a list of inputs produces the Second-order Josephus sequence:

```
,S2a EACH Natural 31
1 2 2 3 4 4 4 5 6 7 8 8 8 8 9 10 11 12 13 14 15 16 16 16 16
16 16 16 16 16
```

Comments: The utility program `EACH` is required to apply $S2a$ to each item of the input list separately. `Ravel (,)` is used to display the resulting array of 1-item lists as one list.

2. Sorting Approach

The Second-order Josephus sequence can be produced directly – that is, without resorting to simulation – by sorting indices together with copies of powers of 2.

McDonnell [2] offered a “high-speed” verb $hsJ2 =: 13 :$
`'/:~(>:i.+ /2^i.y.), ; (#each+:)2^i.y.'` which generates the sequence without the last item. Here is a revised explicit definition to produce a full sequence:

$J2x =: 13 : '/:~(\text{Natural } 2^{\wedge} y.), (\# +:) 2^{\wedge} i.y.'$

Symbols/Syntax: $J2x$ is verb Define (SortUp Commute) ((Natural (Two Power input)) Append ((Copy Double) (Two Power (Integers input)))). Note that `13 :` indicates a verb is being defined explicitly (using input $y.$) for conversion to tacit form, and its monadic definition is given in quotes on the right.

Explanation: The input is the power of 2 needed to produce the sequence. Overall, the program sorts into ascending order the result of appending two lists. One list

is the Natural integers from 1 to input y. The other list is c copies of double c, where c is a list of powers of two from 1 to 2 raised to one less than input y.

Example: For input 4, i.4 is 0 1 2 3, and 2^0 1 2 3 is 1 2 4 8, and +:1 2 4 8 is 2 4 8 16, and 1 2 4 8 # 2 4 8 16 is p=:2 4 4 8 8 8 16 16 16 16. Then 2^4 is 16, and Natural 16 is n=:1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16. Then n,p is r=:1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 16 16 16 16 16 16 16 16. Finally /:-r is the result below using the program:

```
J2x 4
1 2 2 3 4 4 4 5 6 7 8 8 8 8 8 9 10 11 12 13 14 15 16 16 16 16
16 16 16 16 16
```

Comment: An explicit definition for a verb can be entered without quotes on successive lines (if needed) using 3 : 0 with a final right parenthesis to close the definition.

Peelle [3] offered an equivalent program using tacit definition:

```
J2t =: Sort @ (Natural@Power , Copy@Power@i.)
Copy =: # +:
```

Symbols/Syntax: J2t is Sort Atop ((Natural Atop Power) Append ((Copy Atop Power) Atop Integers)). Copy is input Copy Double. (See Utility Programs earlier for definitions of Natural, Power and Sort.)

Explanation: Input y is the power of 2 needed to produce the sequence. One list is natural numbers 1 to 2^y . Another list is powers of 2 ($2^i \cdot y$) copies of doubled powers of 2. Sort the two lists into ascending order.

Examples: For an input of 4, i.4 is 0 1 2 3. Then Power 0 1 2 3 is 1 2 4 8. Then Copy 1 2 4 8 is 1 2 4 8 # 2 4 8 16 which is c=:2 4 4 8 8 8 16 16 16 16 16 16 16. Then Power 4 is 16. Natural 16 is n=:1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16. Then n,c is r=:1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 2 4 4 8 8 8 16 16 16 16 16 16 16 16. Finally, the result is Sort r as shown in the result below using the program:

```
J2t 4
1 2 2 3 4 4 4 5 6 7 8 8 8 8 8 9 10 11 12 13 14 15 16 16 16 16
16 16 16 16 16
```

Comment: Tacit definition requires specifying verbs without their inputs, which are given upon execution. This is often called "(pure) functional programming" (see [0]).

3. Replication Approach

Peelle [3] offered a similar program which replicates indices without sorting:

```
J2r =: >:@Copies # Natural@Power
      Copies =: Power@i. In} Power # 0:
      In =: <:@+:@[
```

Symbols/Syntax: J2r is (Increment Atop Copies) Copy (Natural Atop Power). Copies is (Power Atop Integers) (In Amend) (Power Copy Zero). In is (Decrement Atop Double) Atop Left input.

Explanation: Find the number of copies (Copies y) needed and copy each natural number (Natural Power y). Copies produces a list of 0s with powers of 2 substituted in positions of twice the powers of 2.

Example: For input 4, Power 4 is 16 and Natural 16 is 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16. Then (for Copies 4) Power 4 is 16, and 16 # 0 is 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0. Then i.4 is 0 1 2 3 and Power 0 1 2 3 is 1 2 4 8. Then 1 2 4 8 In 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 is 1 3 7 15, so 1 2 4 8 (1 3 7 15) 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 is 0 1 0 2 0 0 0 4 0 0 0 0 0 0 0 8. Finally, use one plus this list to copy items from Natural 16, as results from the program below:

```
J2r 4
1 2 2 3 4 4 4 5 6 7 8 8 8 8 8 9 10 11 12 13 14 15 16 16 16 16
16 16 16 16 16
```

Comments: A variation is J2r1=:Copies1 # Natural@Power where Copies1=:>:@Power@i. In1} Power # 1: and In1=:<:@+:@<:@[(which is slightly less efficient until input is very large).

4. Bits Approach

Peelle [3] also offered a different program to produce the Second-order Josephus sequence by accumulating sums of ones and zeros:

```
J2b =: +/\ @ Bits
      Bits =: 1: , ;@:(Take each)@Power@i.
      Take =: +: {. (# 1:)
```

Symbols/Syntax: J2b is ((Plus Insert) Prefix) Atop Bits. Bits is One Append (((Raze At (Take each)) Atop Power) Atop Integers). Take is Double Take (input Copy One). Note that Bits is defined as a fork (f g h) which is executed as (f y) g (h y). Take is a fork (f1 g1 h1) where h1 is a hook (g2 h2), which is executed together as (f1 y) g1 (y g2 (h2 y)).

Explanation: Accumulate 1s from sub-program Bits which produces a 1 appended to a list of 1s and 0s from boxed lists resulting from (over)taking double the power of 2 copies of 1s and the rest 0s for each integer from 0 to input y minus 1.

Example: For input 4, i.4 is 0 1 2 3, and Power 0 1 2 3 is 1 2 4 8. Take each 1 2 4 8 is boxed list 1 0, boxed list 1 1 0 0, boxed list 1 1 1 1 0 0 0 0, boxed list 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0. Razing (;) the list of these lists is r=:1 0 1 1 0 0 1 1 1 0 0 0 0 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0. Then b=:1, r and cumulative sums +/\b is the result below using the program:

```
J2b 4
1 2 2 3 4 4 4 5 6 7 8 8 8 8 8 9 10 11 12 13 14 15 16 16 16 16
16 16 16 16 16
```

Comments: A variation is: J2b1=:+/\@Bits1=:1:,:@:(Pairs each)@Power@i. where Pairs=:(# 1:),(# 0:).

Another possible variation is: J2b2=:+/\@Bits2=:1:,:@:(Boxes each)@Power@i. where Boxes=:> i.@+:.

5. Base-2 Encoding Approach

Bantchev [2] offered an algorithm using base-2 encoding-decoding (for inputs above 1): S2=:(>:@(1&,@({.+.}.).)@}.&.#:))"0. An alternative algorithm which accepts an input of 1 is:

```
S2b =: Shift &. #:
      Shift =: -@Second |. One + Half Min Second
      Second =: {.@Rotate1=:1&|.
      One =: -@# {. 1:
      Half =: # {. 0: , 1: , }.@}.
      Min =: *, -.
```

Symbols/Syntax: S2b is Shift Under Antibase2. Note that the conjunction &. executes the verb on its right, then the verb on its left, then the inverse of the right verb (here #. Base2). Shift is (Negate Atop Second) Rotate (One + (Half Min Second)). Note that (Half Min Second) is a fork whose result is used within another fork. Second is Head Atop Rotate1 which is 1 With Rotate. Note that

defining *Rotate1* in-line obviates parentheses (*1&|.*). One is (Negate Atop Tally) Take One – a simple fork. *Half* is Tally Take (Zero Append (One Append (Behead Atop Behead))) – a fork within a fork within a fork. *Min* is input And Not – a simple hook.

Explanation: Convert the input to a list of binary digits, apply *Shift*, and then convert back to base-10 value. If the second binary digit is a 1, *Shift* returns a 1 followed by all zeros (a power of two). Otherwise, if the second bit is 0, *Shift* returns 1 (with leading zeros) plus the given bits with a leading 0 and a 1 in the second digit (that is, less 'half'). This is accomplished effectively in *Min* by 'anding' (***.) the given bits with not (*-.*) the second bit.

Example: For an input 9, #: 9 is 1 0 0 1. *Shift* 1 0 0 1 is 0 1 0 2. And #: 0 1 0 2 is 6. Details of *Shift* 1 0 0 1 are: Second 1 0 0 1 is 0. *Half* 1 0 0 1 is 0 1 0 1. 0 1 0 1 *Min* 0 is 0 1 0 1. One 1 0 0 1 is 0 0 0 1. 0 0 0 1 + 0 1 0 1 is 0 1 0 2 (which evaluates correctly in base 2). Using the program for a list of natural numbers produces the sequence:

```
S2b EACH Natural 31
1 2 2 3 4 4 4 5 6 7 8 8 8 8 9 10 11 12 13 14 15 16 16 16 16
16 16 16 16 16
```

Comments: *EACH* is required to execute *S2b* separately for each natural number input.

An alternative definition which converts bits to base value before finding the minimum is more efficient: *S2b1* =: (*Min1 Pow1*)@#: where *Pow1* =: # { . 1: and *Min1* =:] <. &#. *Shift1* =: One1 + (-*Half1*) and *One1* =: 1: |.] and *Half1* =: _1: |.] .

6. Log Approach

Peelle [3] offered an alternative program using base-2 logarithms:

```
S2p =: (] <. 1: + (- -:)) Pow
      Pow =: <.&.Log
      Log =: 2&^.
```

Symbols/Syntax: *S2p* is (Right input Minimum (One Plus (input Minus Half)) Pow. *Pow* is Floor Under Log. *Log* is 2 With Logarithm. Note that *S2p* is a hook (g h) where g is a train of forks (f1 g1 h1) where h1 is a fork (f2 g2 h2) with h2 a hook (g3 h3) so that execution is (*Pow y*) <. (1 + (*y* - -: (*Pow y*))).

Explanation: First find the largest power of 2 less than input y. Call it pow. Then calculate 1 plus the difference of y and pow. Call it d. Compare pow and d. Return whichever is smaller.

Examples: For input 9, Log 9 is 3.16993 and Pow 3.16993 is 8. 9--:8 is 5, and 1+5 is 6. Finally, 8<.6 is 6. Using the program for a list of inputs produces the Second-order Josephus sequence:

```
S2p Natural 31
1 2 2 3 4 4 4 5 6 7 8 8 8 8 8 9 10 11 12 13 14 15 16 16 16 16
16 16 16 16 16
```

Comments: EACH is not needed because S2p applies to each atom in a list.

A less efficient but perhaps more readable version is S2p1=:Pow <. 1:+(- -:@Pow).

Comparisons of Programs

Now the featured programs will be compared for efficiency in speed and space. First, the following utility programs are needed:

Speed

Purpose: Measure execution time.

```
Time =: 6!:2
```

Symbols/Syntax: Time is 6 Foreign 2. Note that this verb may be used monadically for one execution of an expression (on the right) or dyadically for a number of repeated executions (on the left).

Explanation: Average number of seconds used in executing an expression.

Examples:

```
Time 'J2b 12'
0.00097528
```

```
1000 Time 'J2b 12'
0.00069833
```

Comment: For increased accuracy, increase the left input to Time (which will take longer to return the result).

Space

Purpose: Measure execution space.

`Space =: 7!:2`

Symbols/Syntax: `Space` is 7 Foreign 2.

Explanation: Number of bytes used in executing an expression.

Example:

```
Space 'J2b 12'
197440
```

Fixed Programs

Purpose: To replace names of sub-programs with their definitions (for better efficiency).

`Fix =: f.`

Symbols/Syntax: `Fix` is an adverb which requires a verb input (on its left).

Explanation: `u f.` substitutes primitives in verb `u`.

Example:

`J2b =: J2b f.`

Comment: By choosing the same name here, `J2b` is reassigned. To compare fixed vs. unfixed versions, use a different name.

Note that a fixed program should speed up. For example, now `1000 Time 'J2b 12'` is 0.00068846.

Benchmarks

Benchmarks were obtained for fixed programs here using J5.01a running Windows 95 on a Pentium II PC with 256M RAM. The benchmarks are compiled below into tables – one table of average time and space for the “J2” family of programs requiring an input of the power of 2 needed to produce the sequence, and one table for the “S2” family of programs using a list of inputs needed to produce the same sequence:

Input Program\	Time (seconds)			Space (bytes)		
	4	12	20	4	12	20
hsJ2 *	0.0000557	0.01378	2.058	4096	1181696	168298496
J2x	0.0000411	0.01275	2.019	3200	1082624	143131904
J2t	0.0000396	0.01252	1.984	2944	1082176	143131456
J2r	0.0000501	0.00278	1.017	2048	328512	83886912
J2b	0.0000485	0.00069	0.348	2880	197440	50332480

* Last item omitted in result.

Input Program\	Time (seconds)			Space (bytes)		
	Natural 31	Natural 4095	Natural 1048575	Natural 31	Natural 4095	Natural 1048575
,S2a EACH	0.0102841	860.1	(days)	4352	215552	(huge)
S2b EACH	0.0015516	0.2084	55.94	3840	69312	16780992
S2 **	0.0003800	0.0485	12.63	3072	100608	25168128
S2p	0.0000635	0.0039	1.45	3840	296448	75499008

** Executed without first item (1) of input.

Comparing these benchmarks indicates that J2b (bits approach) is the fastest program for large inputs. S2b (Base-2 encoding approach) is the most economical in space for large inputs. Overall, the programs in the J2 family are generally faster but increasingly inflexible (always returning nearly a power of 2 number of items); whereas the programs in the S2 family generally consume less space and are completely flexible (allowing any list of inputs).

Note that all J2 programs except hsJ2 work correctly for an input of 0 and that all S2 programs except S2 work correctly for an input of 1 to produce the first item in the Second-order Josephus sequence.

Note also that S2b is the only program which correctly handles the degenerate case of an input of 0, producing the additional leading 1 in the sequence (as in [1]).

Concluding Insights and Programming Challenges

Defining and comparing programs for the Second-order Josephus problem prompts several insights. First, logarithms (in S2p) appear much faster than encoding-then-decoding (using u&.#: in S2). Secondly, logarithms may be more productive than base-representations in seeking solutions to the General Josephus problem (which involve killing in intervals of k). Indeed, the work here led to an improved definition of a generalized program (from Peelle [7]):

```
J_1 =: K * (N - Mod * K PoW N % Mod) % K-1:.
```

```
K =: [
N =: ]
Mod =: 1: + |&<:
PoW =: K ^ <.@^.
```

Further, the reader might consider generalization of Second-order Josephus to interval k, which produces intriguing non-decreasing integer sequences (perhaps difficult to express in closed form). And consider the Third-order Josephus problem. Fourth-order? Nth order? (Not difficult!)

Finally, the programs here lead to a simple strategy for surviving the Second-order Josephus problem for N people:

Calculate p – the largest whole power of 2 less than or equal to N. Start by standing at half p. Move up one more. Now move one further for each person over p, but only up to number p.

For example: If N=9 people, p is 8, so start at 4. Add 1 up to 5. 9-8 is 1, so move 1 further. Stand at 6 (which is less than p).

Another example: If N=30 people, p is 16, so start at 8. Add 1 to get 9. 30-16 is 14, so move from 9 up 14 (but stop at p). Stand at 16.

In closing, challenges for the reader are to program this strategy, to generalize the programs here for any interval, and to seek other approaches to the Second-order Josephus problem.

References

- [0] Peelle, H.A. "Introduction to Functional Programming in J" (in four parts), *Vector*, Vol. 13, Nos. 1 – 4, 1996–97
 - [1] Sloane, N.J.A. Sequence A006165, *Online Encyclopedia of Integer Sequences*, <http://www.research.att.com/~njas/sequences>
 - [2] McDonnell, E.E. "At Play with J", *Vector*, Vol. 18, No. 4, 1992
 - [3] Peelle, H.A. "Technical Correspondence", *Vector*, Vol. 19, No. 2, 2002
 - [4] Graham, Knuth & Patashnik, *Concrete Mathematics*, Addison-Wesley, Reading, MA 1994
 - [5] Hui, R.K.W. "Three Combinatoric Puzzles", *Vector*, Vol. 9, No. 2, 1992
 - [6] Peelle, H. & Lipp, A. "Problem Solving by Programming: The Josephus Problem (Revisited)", *Journal of Computer Science Education*, Vol. 12, No. 1, Fall 1997
 - [7] Peelle, H.A. "Josephus Generalized in J", *Vector*, Vol. 14, No. 2, 1997
-

Renaissance Data Systems

www.aplbooks.com

Your source for all books on APL & J

Now accepting international payments thru PayPal

P.O. Box 511, Botsford, CT 06404

orders@aplbooks.com

Appendix

Collection of J programs for Second-order Josephus sequence:

NB. Utility programs

```
Natural =: }.@i.@>:
Sort =: /:~
Power =: 2&^
Log =: 2&^.
EACH =: "0
each =: &.>
```

NB. Basic Josephus solution

```
S =: 1&|.&.#: NB. Origin 1
```

NB. Simulation Approach

```
E =: ((<:@[ {. ]) , .)~ S@# NB. Bantchev
E =: (}:@{. , .)~ S@#
E =: -. <:@S@# { ]
S2 =: E ^: (<:@#) @ (>:@i.) NB. Bantchev
S2a =: E ^: (<: ` Natural)
```

NB. Sorting Approach

```
hsJ2 =: 13 : '/:~(>:i.+/2^i.y.),;(#each+:)2^i.y.' NB. McDonnell
J2x =: 13 : '/:~(Natural 2^y.),(#+:)2^i.y.'
J2t =: Sort @ (Natural@Power , Copy@Power@i.)
Copy =: # +:
```

NB. Replication Approach

```
J2r =: >:@Copies # Natural@Power
Copies =: Power@i. In} Power # 0:
In =: <:@+:@[
J2r1 =: Copies1 # Natural@Power
Copies1 =: >:@Power@i. In1} Power # 1:
In1 =: <:@+:@<:@[
```

NB. Bits Approach

```

J2b =: +/\ @ Bits =: 1: , ;@:(Take each)@Power@i.
    Take =: +: { . (# 1:)
J2b1 =: +/\ @ Bits1 =: 1: , ;@:(Pairs each)@Power@i.
    Pairs =: (# 1:),(# 0:)
J2b2 =: +/\ @ Bits2 =: 1: , ;@:(Boxes each)@Power@i.
    Boxes =: > i.@+:

```

NB. Base-2 Encoding Approach

```

S2 =: (>:@(1&,@({.+}.).)@}.&.#:))"0      NB. Bantchev

S2b =: Shift &. #:
    Shift =: -@Second |. One + Half Min Second
    Second =: {.@Rotate1=:1&|.
    One =: -@# { . 1:
    Half =: # { . 0: , 1: , }.@}.
    Min =: *. -.

S2b1 =: (Min1 Pow1) @ #:
    Pow1 =: # { . 1:
    Min1 =: ] <.&#. Shift1 =: One1 + (-Half1)
    One1 =: 1: |. ]
    Half1 =: _1: |. ]

```

NB. Log Approach

```

S2p =: (] <. 1: + (- -:)) Pow
    Pow =: <.&.Log
S2p1 =: Pow <. 1: + (- -:@Pow)

```

NB. Benchmark Utility Programs

```

Time =: 6!:2
Space =: 7!:2
Fix =: f.

```

Howard A. Peelle, Professor of Mathematics and Science Education at the University of Massachusetts, has received a Fulbright Scholar Award to join the Institute for Mathematical Research at the University of Putra Malaysia in Kuala Lumpur where he will use J in teaching mathematics teachers during spring 2003.

CONFERENCE REPORT APL2000 AT NAPLES

CONFERENCE REPORT – APL2000

Notes and Pictures

Adrian Smith

66

APL and Web Services

FM Waid

71

All the photographs are by Adrian Smith.

APL2000 User Conference 2002 Nov 10th-13th at Naples, Florida

reported by Adrian Smith

Location, Location

There is very little one could say against Naples (just at the moment I can't think of anything) and an awful lot in its favour.



It was warm, but not hot. It was (mostly) dry but with a sea-breeze during the afternoon. The sun set over the sea. This being Florida, the beach is publicly owned, so you could hike about 2 miles North, and South as far as you wanted. Naples itself is about 3 miles further down the coast, but it is comfortable walking in the cool of the evening, and the choice of restaurants is excellent (but don't be too late – they start closing up around 8:30).

The Naples Beach Hotel was only a bit posh, and the conference room-rate was extremely reasonable. The above shot shows the bar (the day's sessions always finished in time to watch the sunset) and the beach. It is now a tradition of APL2000 conferences that breakfast is provided, which (a) neatly circumvents the one obvious piece of rip-off pricing and (b) is a great place to meet and natter in

the cool of the early morning. I can report that Causeway made a handsome profit on the trip, and the ample networking opportunities had a lot to do with it.

The meeting rooms all opened out onto the balcony which fronted onto the 'championship quality' golf course, which looked a bit like this in the early morning:



Tempted? You should be! Actually the really good option was a very well-set-up practice ground with buckets of balls at \$2 a time (clubs free). An excellent way to work off the effects of a sedentary morning was to take a bucket of balls and a 7-iron and belt them satisfyingly into the distance.

The Content

Training days

The training days (Sunday and Wednesday) were dominated by a heavyweight introduction to APL and WebServices. You can read a little more about this in FM Waid's article which follows this report on page 71 – Jonathan and I went off to "Corkscrew Swamp" to look for alligators and cypress trees so we will be catching up on this later. It is a particularly interesting new technology, and may well be responsible for many of the customer 'win-backs' which APL2000 are reporting. Probably it will be the hot technology of 2003, and it is particularly relevant to APLers working with 'large organisations', which just could mean *you*. Don't ignore it, it won't be going away.

First Conference Day

Sonia booked us all in with her usual efficiency and Eric Baelen had picked himself up from the previous week's Cognos Finance conference sufficiently to get things started with his usual mix of enthusiasm and anecdote. Unfortunately, I was scheduled to present RainPro and NewLeaf opposite Patrick Parks on the 'inner secrets' of the new WebServices capability, so we will just have to wait for Patrick to send us something to publish here. There was also an excellent introduction to making an application multilingual, which is well-documented in the published papers. Maybe we will re-run this in a future Vector?

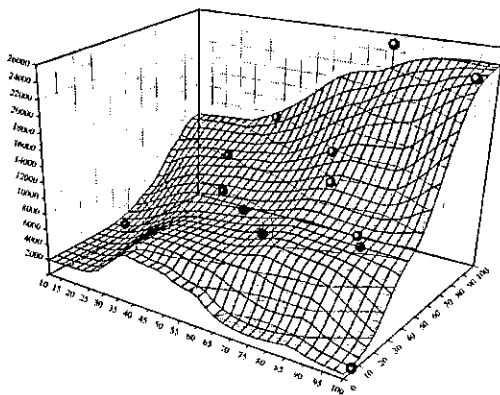
Jonathan Manktelow ran two sessions on getting the best out of the new 'APLDraw' OCX control which is shipping with APL+Win version 5. This is a full interface to GDI+ (native on XP and freely redistributable otherwise) which allows APLers to create all the standard image formats (JPEG, GIF, PNG etc) and exploit all the nice effects (like transparency and gradient fills) that come free with the system. It is a huge step forward from the old 'Draw' method and allows APL programmers to create genuinely leading-edge graphics with high efficiency.

APL

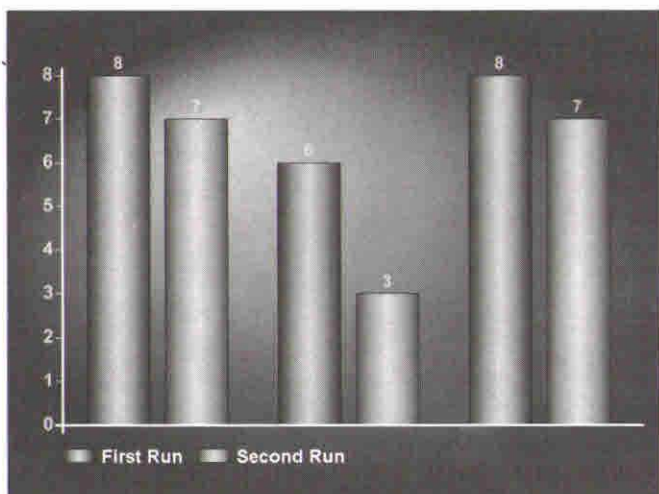
This is the sort of thing you can now do rather easily with APL (that shadow is done with a variable opacity black, so it correctly modifies whatever is beneath), and from my point of view, it offers me some really exciting new possibilities

for RainPro charts. As an example, here is a fitted 3D surface, which is partially transparent so that you can still see the data-points which are below it:

*Fitted Trend Surface
with Contours and Residuals*



... or if you just want to take on PowerPoint at its own game, you can do your barcharts like this now:



OK, it looks better in colour, but you get the idea. I *think* I have got everything I need running with the new OCX as `csView PG`, so look forward to an upgrade of RainPro alongside the shipping version of APL+Win V5. It looks better, runs faster (about 3 times faster on average) and has simplified my viewer code enormously. Enough said.

Eric Baelen on "Making IE work for You" was a thorough introduction to using the WebBrowser object as a component of your application. This straightened out several issues for me (GraPL Desktop is about to gain a "Preview as SVG" panel as a consequence) and introduced some good new ideas, for example using a webform (bog standard HTML) to log errors and 'Submit' the error messages (with the user's comments) directly to your server over the internet.

And so to the beachside barbeque and entertainment. We were not disappointed, in fact young Parks was so good on the piano that I bought 3 of his CDs.

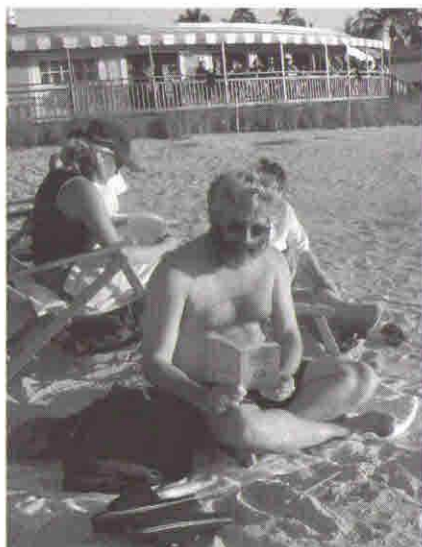
Second Conference Day

Eric Lescasse gave us a serious hit on the APL grid object. He showed a time-tabling application he had built for TF1 (the major French TV channel) which used the grid in any way you could think of, and several others besides. It was quite interesting to see one of his early panels from the initial 'concept' design which

predated the grid, and managed most of the same tricks in raw Gui APL code. One day, he may re-code it to use the real grid, but I think he has more urgent calls on his time.

I missed out on the rest of the morning, but the papers on the enhancements to the session manager (**Mark Osborne**) and the Windows registry (**Brian Chizever**) were well-covered in the written proceedings. Likewise after lunch, we had **Marvin Renich** covering some undocumented `WCALL` options and INI-file settings which may be generally useful. I gave a brief introduction to Genetic Algorithms (and solved at least one delegate's 'hard' problem along the way) and we ended the afternoon with an excellent talk from **Rick Woll** of Allstate Insurance on the power of the WebServer concept in getting APL accepted within a corporate culture. I was particularly impressed with the simple, elegant, consistent design of the pages he showed. As he pointed out in his talk, the lady who builds the 'website' has little or no knowledge of APL, and he really doesn't care about the input documents. He can focus on writing the APL code to convert an incoming set of parameters into an output set of values and associated text. The result is a clean, simple, maintainable APL 'backend' which is functionally well separated from the equally clean, simple HTML front end. Maybe there is a lesson here for the rest of us?!

Summary



Well worth the trip, if only to escape the English November for a week or so. We flew BA to Tampa and drove down the coast, which was a most enjoyable way to make the journey. The Cannons (pictured) took the Virgin route via Orlando which is particularly good value if you book early and buy the whole 'package' deal.

If APL2000 do it all over again next year, we will be there, war and weather permitting. Maybe this time I will get my 3-wood straightened out, if I work at it.

APL and Web Services

by F. Milton Waid (FMilton@Openhere.com)

There has never been a technology that better complements the capabilities of APL than Web Services.

Historically, APL has consistently been capable of performing complex calculations more quickly than any other language.

Web Services provides a seamless mechanism for publishing these capabilities both to other servers and to clients.

During the last year, it became common for APL to have "win backs" where complex calculations are involved. Both in the US and in Europe, we have clients who spent the last several years replicating APL applications in VB and other

Web Services, matched with APL on the server, provide a compelling combination for publishing results generated from APL calculation engines

.Net languages, only to discover that the response time, which was seconds in APL, is now 20 or 30 minutes in the new application. Because these applications must deliver a result to a Web browser, response times in minutes are intolerable. Web Services, matched with APL on the

server, provide a compelling combination for publishing results generated from APL calculation engines.

However, with any technology solution, no matter how compelling, in the end it is the delivery of that solution that makes an application a success. Connecting an APL application running on the server with a client is the future of computing.

Understanding the client and how it connects to the server is critical. That means understanding what technology has been deployed, the depth of that deployment, and what technology is on the horizon.

If the only goal is to serve a single discrete page, which is either static or includes a simple database query, then any server methodology can be successful.

However, in most cases, that single page actually needs to be an integral part of a larger task or application. It is the creation of applications which use Web Services that make APL unbeatable.

What defines the computing client?

Looking at today's computing client, we discover it has become almost monolithic in structure, if not monopolistic in practice.

According to Media Metrix, Nielsen, IDC and OneStat.com ...

- IE has 95% of the browser market
- MS Office has 93% of the business application market
- Windows has 94% of the desktop OS market
- Netscape, Mozilla have less than 3% of browser market
- Opera has .9% of browser market

These market share statistics mean that an application which works in the IE browser will run on a larger share of desktops than any other application in the history of computing.

The IE browser is not the browser of a few years ago. No mere display device for HTML formatted pages, it has become a platform on which applications can be deployed. This includes well-formed XML/HTML document structures, accessible at every level. Additionally, the IE browser includes both Jscript and VBScript languages. Using these languages makes it possible to create complete applications which run on the client without requiring the user to download or install anything. Jscript and VBScript also provide complete access to all of the safe OCX controls that already exist on the client machine.

The technologist's fear of the application

The most common trend we discover in IT, whether it be in corporations, or home users, is an extreme reluctance to install software. 80% of all users are still using modems. What do users and IT departments want? Not to download software, and not to install new software. Most corporations now have "zero footprint" requirements.

Even a few years ago, it was hard to foresee this. But in retrospect, it looks obvious. Here's why.

Starting with Windows 95, Microsoft decided that it would be a great idea to gather up all the .INI files on a user's system and consolidate them into one huge database, which they called the Registry. Of course, if this file ever becomes corrupted, the operating system not only fails to start, but can only be fixed by reinstalling Windows.

Another spectacular decision by Microsoft was to take all the DLLs on a machine and put them into one system directory, so all shared code could be used by any program on the computer, which seemed like a good idea at the time to some people.

But what happens if, while a program is installing, it overwrites an older DLL with a "newer" version? The DLL is often less than perfectly backward compatible, and programs that relied on the old version will fail even to start. The result is predictable. As developers produce applications, and write information into the massive Registry, troubles develop.

Users have become very reluctant to install new applications, after having a newly installed software package cause other working applications to malfunction. IT departments have also become unwilling to install new applications for the same reason, not having at their disposal the time or resources to debug the problems which occur from installing new software.

The result of this is that users will no longer install new software and corporations demand "zero footprints". This together with IE's 95% possession of the browser market, produces an environment in which users and corporations want applications with no "installed" components, and which "run in the browser".

Since it has been established that a successful application must now be created in the browser, the next question is what tools are available to us at this time, and will be for the next three to five years?

What languages are on the client today?

- Jscript, the language formerly known as JavaScript
- Vbscript

So, for at least the next three years, if we are going to effectively get to the client, these are the tools we have to work with.

Now that the client situation is well defined, the next question becomes, what do we need on the server side, and how do we communicate between the client and the server, and between servers?

What questions should I be asking about Web Services?

- Why should I care?
- Who is going to use this?

- Does this tie me to Microsoft?
- Is it fast?
- How much hardware do I need?

Who is deploying Web Services today?

- Amazon has deployed its first web service for affiliates using XML over HTTP, with SOAP to be added next quarter
- Google Web APIs, which include the following:
 - A WSDL file that formally describes the service.
 - Example SOAP messages and responses.
 - Sample code application that uses the above Web service.
 - Very simple to deploy because of Web Services

There are over 10,000 applications which now include the Google Web API using Web Services. This allows a developer to include access to Google search results in their application. The communication with Google is done using SOAP over HTTP with the result being in XML format. This is an excellent resource to begin to understand Web Services. And, it makes an excellent addition to many applications.

How about security?

Microsoft, IBM and VeriSign have teamed to create security specifications for Web Services, a move analysts say will help drive adoption of the hyped but still emerging technology.

Web Services without Microsoft

- Web Services has taken on a life of its own.
- Independent of vendor or platform
- WS-I providing leadership on technology
- W3C providing standards validation
- Many vendors implementing

With all of this technology, the question on everyone's lips, is how does this apply to me as an APL developer? How can I make this work for me? What tools do I

have to make to take advantage of these developments and still write code in APL?

APL and Web Services

To make APL a success in this server/server or server/client environment requires that APL be capable of multithreading, seamlessly integrating with Web Services, and also being able to run in IE without downloading any code. Yet, with all of these requirements, we also need the ability to place existing APL applications into this new environment without bankrupting our businesses rewriting code that has already been debugged and optimized.

With this background, APL2000 has been working to create a solution which makes it possible to publish existing APL applications to the IE client using Web Services with few or no changes.

The ideal application would of course be stateless, that is carrying all the information about the present state of the application with each transaction. However, that has hardly been the requirement for application development over the last 20 years. So, we discover that existing applications rely on state, global variables and system resources which have been initialized for a particular client.

Deploying legacy code in this new environment requires that a Web Services server understand a one-to-one mapping of client to workspace. Additionally, it must support many-to-one and one-to-many as applications evolve. Next, it logically follows that the server, because of finite resources, must deprecate a workspace which has not been used for some amount of time, and reinstantiate that workspace if a request occurs within a specified period.

In line with this server effort, the APL interpreter also needs dynamic memory allocation. This solves both resource issues on the server, and WS-full concerns where server applications are running basically unattended.

Of course, all of this would be futile if APL could not create responses as quickly as Microsoft's IIS server. The APL2000 WebServer not only multithreads APLs but also outperforms the IIS server, even on simple requests. When the calculation involves any type of mathematical analysis, APL outperforms IIS by orders of magnitude.

Speed and Reliability

Because we knew that Microsoft had seriously biased the Windows 2000, NT and XP operating systems to perform well with their IIS server, we were assuming

that we might be able to perform at about 80% of the speed of IIS. So, we were truly ecstatic when we were able to serve up 14,623 responses to only 12,498 for Microsoft's IIS. This meant that Microsoft's IIS/ASP was only 85% as fast as APL+Web Services.

In addition there is a particularly critical piece of data which is contained in the test results. In the case of APL+Web Services, we had no socket errors and no HTTP errors. However, the Microsoft IIS solution had 743 HTTP errors and 56 socket errors, or a total of 799 failed responses. That amounts to an error rate of over 6%. From the viewpoint of the user, it means that only 94 out of every 100 pages requested are returned, the other 6 are error pages. Even if the users were willing to tolerate having 6 out of every 100 pages fail, it also means that the application would have to be able to function with only 94 out of 100 pages successfully returned, which becomes fairly difficult to design.

Maybe Microsoft looks at this as a feature? The Microsoft solution is to suggest that you buy more copies of their software and install it on more servers. This also means that you will want to buy their clustering software to efficiently manage this group of servers.

So, for a case where you have an application being accessed by 10 simultaneous users, you either buy more software from Microsoft or experience 6% errors. The reason for this is that Microsoft has built their IIS server on the concept that developers want to deploy single pages, not applications. So for each dynamic page that is returned, IIS must start and stop whatever software is required to create the page.

In our case, we were returning an arraysamp.htm page in the Microsoft simple sample group folders. When we performed the same side-by-side comparison on pages that involved mathematical calculations, APL+Web Services outperformed IIS by a factor of 10 or more, depending on the complexity of the calculation.

For those cases where an application is being deployed over the Internet, APL+Web Services is the only choice. IIS/ASP works primarily only for cases where a single page of data is being returned, not as part of an overall application environment.

One of the concerns, over time, for APL developers has been whether the results calculated by APL could be accessed rapidly. With APL+Web Services, requests can be fulfilled at over 250 per second. This means that an APL application is just as accessible over the web using Web Services as it is when directly connected to the interface.

When an application needs a standalone web service/web server, APL+Web Services provides the answer, and does a better job of publishing an application to the web than anything Microsoft has to offer.

APL and the IE client

Since we have IE as the default interface on the client, we needed a consistent method for using this resource. Combining this with a desire to make publishing legacy APL applications as simple as possible led to the creation of APL+JS. This is simply the APL interpreter ported to Jscript.

Typically when a browser interface is created for an application, it involves the creation of *ad hoc* HTML pages, linked to other HTML pages, which include snippets of Jscript on each page. Maintaining this collage of pages and code becomes exponentially more difficult as the user interface grows.

Using APL+JS means that your existing application code is simply published to the web, without having to create any HTML pages or learn any Jscript. This is great, if you want to concentrate on writing APL code. It also provides the ability for the developer to create a standalone application, maintain that application, and then make that same application available over the Internet using Web Services with a minimal of effort.

The IE browser has the Web Services methodology built in, which means that all communication between the user interface, and APL on the server, uses Web Services.

This means that an application published to the web is completely compliant with what an IT department expects in terms of protocols and security. Thus the APL developer can concentrate on their application, not on spending time learning HTTP, HTML, XML, SOAP, etc, etc, etc....

So, if a client asks if the APL application will work in their office, there is only one question which has to be asked. "Can your people get to Yahoo?" If the answer is yes, then the application will work in their IE client.

Of course, speed is always an issue, and tests using the Microsoft Application Test Center show that using the APL2000 Server Suite with APL+Web Services supports over 250 transactions per second. Best of all, there is no requirement that the server side be a Microsoft OS. The server side can also be Linux or Unix, with APL2000 running on the server.

Where do we go from here?

It is difficult to guess which way Microsoft will jump. However, no matter which turn technology development may take, it is clear that managed code, running in a client, connecting to a server using Web Services will be at the core of the solution.

This means APL+Web Services and APL+JS are methodologies which create a bridge to the future today, and will be the workhorses of tomorrow. Best of all, APL now has a chance to once again shine in computing solutions, in a way we have not seen since the mid 1960s.

Appendix: Basic terms which define Web Services

XML (Extensible Markup Language): Language used to exchange data between computers over the Internet.

SOAP (Simple Object Access Protocol): Describes how Web Services communicate over the Internet.

WSDL (Web Services Description Language): Describes Web Services and how to access them.

UDDI (Universal Description, Discovery and Integration): Lets businesses register, advertise and find Web Services in a directory.

WS-I (Web Services Interoperability Organization) Founded by Microsoft and IBM in February 2002

Trip Report: One Way From APL to J and K

*by David Ness (edited extract from his article at
<http://mywebpages.comcast.net/dness/notes/sjt.htm>)*

This trip from APL to J/K has been productive, for me at least. The purpose of this note is to describe some of the factors that led to taking this trip. And to document some notions about the efficacy of having done so.

J and K

A few words about treating J and K together may be in order. Both languages were important to me as I gradually moved away from using APL as an important part of my programming repertoire.

Compare

J and K are quite different languages, but they share enough characteristics to make it not completely foolish to talk about them together. Their similarities are at the conceptual, not programming language, level. Their differences are structural and (most probably) in deep structure implementation. In what follows we'll use J/K when the arguments apply equally to either. Most of the time, and for the purpose of being used as an APL follow-on, either would serve our purpose well.

Elsewhere

The issue of what differentiates J from K has been treated in some depth elsewhere. The purpose of the discussion here is not to emphasize their characteristics, but rather to suggest why these languages have replaced APL in my implementation repertoire.

Attraction

There was nothing particular about J/K that caused me to be attracted more to one than to the other. As should be clear from the discussion below, I have used both of these languages in circumstances where I might have previously used APL. It may be relevant that I first learned J, but since I now probably do more K programming, I think of myself as having equivalent skill in the two languages.

J/K represent reasonable alternatives to APL. While it is slightly unfair to treat them together, I have found them a productive alternative. And for the purposes of this discussion treating them together simplifies discussion.

Personal Background

Insofar as it is relevant to this discussion some aspects of my background may have an impact on the way the circumstances have an effect on the choices that I have made. So in the interests of disclosure, some short items may be relevant.

- **Taught APL.** I have taught APL to some thousands of students over more than a decade of teaching introductory computer courses. Most of this has been to BA/SB students in Economics and to MBAs. So I have seen lots of different people work with, struggle and learn APL. And a lot of the acceptance of APL in the financial communities of the early 1970s was due to students that came out of these, or similar, programs.
- **Lots of Languages.** I know, and regularly use, lots of languages. I suppose my principal implementations were generally done in C, but early work was in Fortran, and more recent work has been in perl. I write a fair amount of code and more than a little descriptive and expository text. Many of the problems I analyze have a quantitative aspect, although few of them involve any profound or deep mathematics. I know lots of computer languages, starting with several assembler languages in the 1950s, and now use lots of more 'modern' tools.
- **Long Consulting Experience.** I have spent a long time consulting for a variety of organizations, both large and small. In the course of working on these assignments, various programming tasks were almost always at least a part of the problem. In addition some of the assignments were working on electronic document creation and dissemination at large financial houses, thus mixing editorial, analytical and technical problems in about equal measure.

First Encounters

My first encounters with J/K started about a decade ago (J, principally, as K came along somewhat later). The earliest versions of J that made it into public circulation date from about 1990, and my regular use of J began a couple of years later, around 1992. K was made public a good deal later, but now we have had several years of K usage as well.

Old APLs

For the past several years I have been using J/K, so I have little experience with the 'modern' APLs. The last APL I bought for a corporate client was probably in the middle 1990s, and it was an adequate APL, though I found it somewhat clumsy to use. Since finding J/K I haven't given much serious thought to the capabilities of current APLs, although I have continued to follow `comp.lang.apl`, so I am reasonably certain that nothing has happened that would cause me to investigate it seriously again.

What I don't do

Contrary to many APL users, I don't do a lot of matrix mathematics and I don't do a lot of statistics. Of course some of my problems will make use of the statistical capabilities of these languages, but most of my uses are what might well be described as 'non-mathematical'. However, even if the problems are non-mathematical, they are, nevertheless, quite real.

My work may make me more interested in applications that are not intrinsically formulated as matrix problems. I tend to process a lot of text and do a fair amount of work with long integers. So I cannot claim to be a normal APL user. However, my work load is quite typical of that of a reasonable number of people who do some analysis and write a wide range of reports that often involve some quantification. It is probably appropriate to note that much of this work is better directly supported by the facilities in J/K than is the case in APL.

The Pull to J/K

Lessons learned from Knuth (and others) and which may be a deeply ingrained part of MIT undergraduate education, suggest that it is usually worthwhile to keep a sharp eye out for what the 'smart guys' are doing. As one might expect there are 'LISP smart guys', 'perl smart guys', 'python smart guys', etc.. Among the APL 'smart guys' Iverson plays a special role as 'inventor' as well.

- **People.** I was using APL fairly actively in the early 1990s. I guess the first clue I got about J's existence came from noticing that Iverson had been working on a language called J. If I recall correctly, the source to this little language was actually available, and I was particularly charmed by the fact that it would run on my HP200Lx handheld (as would APL – in the Manugistics version).

Iverson is certainly one of the 'smart guys' that it is always worth paying attention to. Although I didn't know it at the time, Roger Hui is also in that league, and the combination – with help from lots of talented others, no doubt – produces a very effective piece of software. It is also, given its authors, an intellectually cohesive piece of software, and this shows up regularly, often in important circumstances.

- **Typography.** Another nattering problem with APL has always been the typography. I still have some code – probably written in the early 1970s – that allows me to (with tedious and trying exchange of typeballs) put APL code into typeset documents. Within the past month there was another of the seemingly interminable discussions of APL and typography on comp.lang.apl. I figured, I guess, that if Iverson decided that his ancient typography wasn't worth saving, I probably wouldn't profit much to spend time second guessing him, so following his lead made sense.

- **Flow.** APL never seemed, to me at least, to represent the flow of control very comfortably. While I understand that now there may be much better representations for this in modern APLs, I never cared much for right arrows or line numbers or for the odd machinations that always reminded me most of my machine language days where we would program jump tables and mistakenly think that we had done something profound.
- **Former Students.** While I was beginning to play with J I started to hear a lot about K, principally from some of my Wharton students who were spending a lot of their time working on some of those derivative financial instruments that would later check their careers. I started to hear about demos where a few keystrokes and almost no time were required to do things that took ages not only in the Oracles and SQLs but also even in the APLs that were then still commonplace on Wall Street.
- **Whitney.** A name which kept popping up in these contexts was that of Arthur Whitney. After a bit his links to A+ and to Iverson and Hui made clear that he was another of the 'smart guys' that was worth paying attention to. Around this time comp.lang.apl was full of debate and discussion about J, and Stevan Apter (another of the real smart guys) started to talk about K with some frequency.

A problem for the rest of us, at that time, was the fact that nothing (that I knew of anyway) had been published about K. So these discussions were not always very informative, often ending with some startling revelation from Apter which was nevertheless unprovable, as few of us had any access to K. And the snippets of code that were exposed were generally undocumented, and given the rebarbative appearance of K, often woefully obscure.

This all changed when Whitney decided to release an 'evaluation copy' of K that was functional enough for most of us to use to develop an understanding of the language and to handle the kinds of problems that had a personal, not corporate, scale. This came along with documentation that was very effective in explaining the elements of K once one learned how to read the terse writing that described the language.

- **Activity.** As the J/K people began to leave comp.lang.apl traffic on that newsgroup dropped precipitously. For many of us the signal/noise ratio collapsed, and discussions of highly intellectual subjects like 'Are APL people disproportionately hirsute?' started to occupy a significant proportion of the traffic.

- **Keyboard.** That the APL keyboard is a nuisance is hard to argue. Even after nearly 30 years, my fingers still know where the proper keys are for the APL functions. But that's not of much use to me, and it is a positive hindrance to anyone whose fingers aren't similarly programmed. When I was still writing APL code I even learned how to read the incredibly odd expressions that resulted from the various renderings of the APL glyphs. But this is all an extra burden with very little payoff for me. And indeed it has no payoff whatsoever that I can see to any 'new' person who might encounter the language.

Neither J nor K impose such a burden on the learner, although they do, I suppose, impose some burden on the 'old-APLer'. But then there are fewer and fewer of them to worry about every day.

- **Off-Line Editing.** Special character sets are also a real nuisance to edit outside their 'home' environment. It is generally not an easy proposition to hand an APL function to a conventional editor. With J and with K this is an everyday occurrence. While J has its own IDE, K does not, and both of the languages are very easy to handle in any conventional ASCII-oriented editing environment.
- **Off-Line Everything Else.** And then there's the problem of everything else that one might want to do outside of the specific implementation environment of a particular programming language. For example, I regularly have K code that writes perl and perl code that writes K. They're all ASCII based, so none of this seems either unusual or difficult. To illustrate with a specific example, I happen not to like K's regular expression handling much, but that doesn't cause me much any real difficulty as I can just use the kind of 'bridge' discussed to accomplish the task of making this easy.
- **Character processing.** I have never much liked the way that APL handled strings as variable length entities. J/K (particularly K) handles strings much more easily and naturally. Since strings, and text files, are a very important form of storage and an important domain for computer manipulation of data, this is particularly important to me.
- **List Structures.** Some problems just don't fit into a rectangular matrix paradigm very naturally. In such cases the list structure orientation of K is particularly effective. In the view of K, matrices are special cases of the more general concept of list of lists. In general I'd say it's easier for K to adapt to the special case of matrices – they are regarded as rectangular lists – than it is for J, or APL for that matter, to treat ragged lists – by boxing them in homogenous structures.

- **Cost.** Cost is also a consideration. Modern versions of APL can cost from \$500 up to several thousand dollars. It is perhaps interesting that most of the vendors have gone to an 'ask for pricing' basis – something that usually indicates negotiable prices (and weak sales?). J has just announced a strange policy that essentially suggests users pay somewhere between zero and \$800 to use J. The 'normal' fee is pegged at \$100 / year. K pricing is generally free for the 'evaluation' version, something perfectly adequate for most individual use. IBM has an odd APL pricing policy which was, last time I looked, something like \$150 for an OS/2 version and about 10 times that for a Windows version.

In summary, for my purposes J and K don't cost anything and I have been unable to justify (to myself) paying for any upgrades to my (now) ancient APL. So when I use APL it is either the old Manugistics version or a nearly decade old copy of APL+Win.

This is a long list of things that have interfered with my use of APL in an even wider set of circumstances than seemed to be natural. These difficulties have, for me at least, now been pretty well resolved by using J/K. And the transition hasn't been very difficult.

Keep the Baby

APL has taught us lots of important lessons. It's important that we don't throw them out if we are going to move to something else.

- **Terseness.** Perhaps the most important of these lessons is the value of terseness. When a computer language is terse, you can see a lot at one glimpse. APL is admirably terse. So are J/K, thus preserving the value of this important aspect of the original language.
- **Clarity.** One of the things that moved Iverson in the direction of defining APL was clarity and lack of ambiguity. Again both J/K continue in this tradition.
- **Interpretation.** Interpretive languages are so powerful, and in the case of languages that are terse as J/K are, this does not require that we sacrifice the power and expressiveness of being able to write code 'on the fly'. Interpretation is a technique that seems to allow a straightforward expression of matters that would otherwise be very complex.
- **Interactive.** The ability to sit at the console and 'play' with problem formulations and solutions is also immensely powerful, again particularly when the languages involved are terse. Terseness is attractive when you are actually typing commands on the console directly. And a little experimentation often proves to be a key in getting to an effective view of a problem domain, as well as an important step in often avoiding hours of debugging effort that might otherwise be required.

- **Non-Iteration.** All of the languages coerce one into thinking non-iteratively. While J/K can be used with conventional loops, and possess loop-like constructs, they generally coerce you into thinking of problems in some array or list oriented form. For many problems this is a particularly enriching experience.
- **Intellectual Cohesion.** APL, J and K are all remarkable for their intellectual cohesion. Perhaps this is a direct consequence of Iverson's leadership, or perhaps it has wider causes. I don't know. What I do know, however, is that almost everything that ought to work actually does work and that is something that doesn't happen in environments that lack a fundamental underlying cohesion.
- **Simple Rules.** These languages also do amazingly complex tasks with equally amazingly simple rules. The fact that the rules are so simple makes understanding the overall structure easy, although it seems, for me at least, to be a lifelong study to understand the elaborate consequences that are obtainable with even a very simple set of rules.

A Negative

There is also one important negative that has also been inherited from APL. There is considerable evidence that lots of people find languages of this type hard to get their heads around. This is no surprise. Some people find rigorous structures easy to understand and use, and others can work a lifetime and not find them anything other than a hindrance. J/K shares with APL the fact that it is hard to think about, but very effective to use, if you can get your head around it.

It is fairly obvious that APL provided an extremely interesting and provocative view of an important subset of computer problems. For a while some of us were naive enough to think it was a better way rather than a different way. That idea, although it seemed to gain some currency for a few years, gradually withered away in the face of mounting evidence that it wasn't better but rather just different. However, there is enough value in just being different that should be exploited by people capable of doing so.

Programming Language

For me the most important use of a programming language is how well it functions as a practical matter solving real problems.

- **Conventional.** J is a fairly conventional programming language in many normal respects, and as such seems, right out of the box, to be more comfortable to me than APL. Representation issues aside (they argue against APL) the control structure of J seems quite normal to a C programmer. While some things (implicit array iteration, for example) are quite different, normal

programming structures look like normal programming structures. They are easy to recognize and not troubling. With K this is less so, as control structures happen to look different than we might expect, but once one 'gets the hang' of it, they read quite normally, too.

- **Efficacy.** Both J and K are efficacious. Somewhat to my surprise this is particularly true of K. At first I found K hard to read and write, but now it has become a rather commonplace activity for me, one that I face with no more trepidation that I do facing a task with any of my other programming tools.
- **Library.** J has a large library of functions that are available to users. Over time lots of packages that deal with things like 'fractal mathematics' or 'concrete mathematics' have been developed and are available for use. K has much less of an available library, but its functions are more financially oriented, and this is an important area. I find J/K easier to use to represent 'conventional' programming structures than was the case with APL. As a result, they serve the function of being a 'programming language' better for me.
- **Mind Bend.** These languages are more than just programming languages. They also influence the way we think about problems. As such, if they are successful, they open new ways for (at least some of) us to tackle the problems we have at hand.
- **New View.** All of the languages in this class can generally be regarded as instructive from the standpoint of having some possibility of giving us a new view of problems. The whole notion of looking at things other than iteratively can be enormously productive, and all of these languages are conducive to that.
- **Learning.** However, I find K particularly startling with respect to giving me new views of problem solutions. And I say this without even using K in what is arguably its most significant problem domain, namely that of data base management.
- **Terseness.** There is also an almost 'mystical' aspect to terseness that is hard to describe. When you have terse code, and see simplifications that make it even more terse, it seems that you are often getting closer to the core of the problem. Thus shorter is, often, not only faster, but truly better. I find my short fragments survive environmental change even better than long elaborate pieces of code. I suppose this is because the more we choose a solution structure that parallels the structure of the core of the problem, the more likely we are able to adapt to circumstances as they evolve in reality.

If you have the kind of mind that can make effective use of any of these languages, then the languages themselves can help you shape and structure your thoughts in ways that lead to them being tidy and effective. In my experience not everyone can, or wants to, do this. But those who can and do find using the languages, in places where they naturally apply, to both help themselves express

their understanding of the situation, and to help them modify and make more elegant that understanding. In my experience, this is less often the case with other computer languages, even when they are very helpful in solving practical problems.

Community

Community size can be important. Unless a language is being used only to maintain legacy systems, new jobs are likely to be created in some proportion to community size. Similarly, the more 'others' there are who use a language, the more likely their work may be able to help us. So size counts.

Conferences

There is little to distinguish among the languages with regard to conferences. First, APL, J and K people tend to go to the same conferences (if any). Second, attendance at these conferences appears to be slowly declining over time. Since I don't go to conferences this has no particular impact on my decisions, other than if there appeared to be a major difference between the languages in this regard. Since there isn't, as of now, any such detectable difference, conferences are not important in making the decision to move away from APL. Conferences also seem to be of declining importance at least partially due to the growth of the influence of the Internet and to the general decline in travel. Modern communications technology obviates the need to physically gather the clan. In addition the publication of active results in conference proceedings has now been effectively made obsolete by the availability of truly current research documents on the Internet.

Newsgroups

Newsgroups are another story. When I started to follow newsgroups on the Internet about a decade ago, `comp.lang.apl` had a reasonable amount of traffic with a respectable signal/noise ratio. Then, in the early to middle 1990s, when the J group split off, traffic on `comp.lang.apl` began to decline. When Whitney brought up `kx.com`'s website and set up a list server to handle K related mail, traffic on `comp.lang.apl` declined even further.

At the present time, discussion on the J forum is quite robust. The K list server has bouts of high activity that punctuate slow periods. In both cases the signal/noise ratio remains good to very good. There is little traffic on `comp.lang.apl` anymore, and what little there is often of no substantial technical content – ranging from discussions like *How can we prop up APL?* to discussions of why managers are clearly ignorant for not seeing that 30 years of lack of success in building APL communities in organizations is really a harbinger of a wonderful future for APL.

Membership

The only professional society that has any bearing on this business seems to be SigAPL, and its membership has been in a long slow slide for many years. There are essentially no corporate members anymore, and regular memberships seem to generally decline at a rate of 10-15% per year. I know of no corresponding professional societies that might usefully contribute any 'membership statistics' to let me make the same evaluations about J and K, though, so again this doesn't provide, in and of itself, any reason for choosing one over the other.

Education

I have not made a careful study of the use of any of the languages in educational settings. However, I have at least some residual experience with the use of APL in educational institutions in the US, but no real experience elsewhere in the World. So my view is limited at best.

It does seem to me to be safe to make a few observations, though. First, there was a period of time when APL was taught at several of what we might call elite institutions in the US. For the most part this was in the late 1960s and early 1970s. With the arrival of the spreadsheet, most of these efforts became increasingly moribund, and as far as I know, virtually all such usage had ended by 1990.

A number of smaller, less well-known (and therefore less influential) institutions have since taught some small amount of APL and a little J. One institution in the UK appears to teach a little K. However, taken across all institutions in all countries, the amount of any of these languages that is being formally taught is extremely small, and it is unlikely that the languages will be able to maintain any substantial influence because of schools knowledge.

Again, this appears to be generally equally true for all of the languages, so there is no differential effect here that would cause anyone to choose one over the other because of the availability of school trained people who understand the language.

Although it is very difficult to get one's hands on any real 'figures', I believe that the APL community is decreasing in size and that both the J and K communities are growing. Practically speaking, however, I should make the point that it is my impression that the growth in J and K just about explains the decline in APL, so net I see very little overall movement with respect to the whole problem domain. Indeed over the last 40 years or so, I see little indication that the size of this community, taken in total, has changed in size much at all.

Documentation

No one doubts the importance of good documentation. The only problem is that what one person regards as good, some other person will regard as horrible. If you accept that argument and think about its implications, then there may be a dominant reason to produce lots of different kinds of documentation in the hope that it might support different modes of learning in people.

- **Books.** There are few books of much interest for any of these languages. It amazes me to see people still refer to the APL text I used when teaching it in the 1970s as 'best book around' for teaching APL today. Even if you go to the bookstore with the largest collection of computer books, you are unlikely to see any books about any of the languages discussed here. So there's not much to choose between on this measure.
- **Drafts.** There are some drafts of books about J in circulation in the J User's Group. This is some sign that there are people willing to write about J. These books seem to be quite good in their own problem areas, but whether they will end up by serving much of a market or not still very much remains to be seen. However, at least there is some activity and this very activity provokes interest and discussion.
- **Journals.** I don't know of any Journals that support either J or K, so here APL has a clear advantage, if the journals that exist happen to serve your interests. I found over the years that I used journals as a means of useful communication, their impact and role was declining, and with the arrival of the Internet I now generally find them obsolete. But your interests and experiences may differ. In any event, if Journals matter to you, then that would probably suggest points in favour of APL.
- **Papers.** The use of any of the languages in academic papers is limited. It may also be rather 'topic specific'. For example, most of the research work done with the huge data bases that are commonly processed by K would generally be in the financial area, while most of the APL papers might have some focus on actuarial and insurance issues. For my purposes, none of the languages are heavily enough used in papers for it to influence my choice of working language.
- **Conventional Documentation.** I'd say that I personally prefer J/K's conventional documentation (particularly that of K) to any APL I know, but that may be largely a function of the kinds of problems I tend to work on.
- **On-Line.** J/K has on line documentation that is far superior, for me at least, to any APL documentation I ever encountered. But documentation is always a matter of taste, and your tastes may not be the same as mine.

Documentation obeys several well known laws. For example, there is never enough of it. There is also too much of it. It is never detailed enough, except when it is too detailed. It is never up-to-date except when it is written to incorporate features that have not yet been put into the software. In short, documentation is the thing that we blame for whatever other problems we are actually having using a system. Of the systems discussed here, I personally prefer K's documentation to J's, but only a little. I far prefer J/K's documentation to that of any APL I know. But I could well understand if others would feel differently. For example, J's documentation is quite idiosyncratic, wonderful once you 'catch on' but often quite useless until you do. Thus it is hard on new users, but a real blessing to those of us who call upon it daily.

New Problems

Up to this point we have been dealing with the issues of J and K as an alternative for handling tasks that we might have otherwise handled in APL. There is a whole additional class of other considerations, namely tasks which we wouldn't have tackled in APL, but which prove to be quite feasible in J/K. Here are some.

- **Data Base.** K is particularly rich in data base functionality. Indeed, managing large (billion line) data bases is the primary domain of application, and the speed with which it can do so is already of nearly legend status.
- **Net Connection.** Connections to the net are also an area of increasing importance and networks become an everyday part of the structure of both organizations and with respect to inter-personal communication. J and K both allow the easy management of TCP/IP message stacks and make the transmission and reception of messages across the Internet quite an easy thing. In addition, K's way of handling this problem often results in complex systems which can be easily debugged on a LAN, and then with a very simple change of command parameter suddenly turn into servers available on the net. Some of this code is startlingly simple.
- **Deployment.** Deployment is also an issue of importance to anyone who has systems that need to be installed at lots of sites. Not only can the installation prove difficult, but the maintenance of such systems can easily become quite a nightmare. K offers some truly incredible facilities for deploying operating code. Indeed, the distribution of the whole system itself is a system administrator's dream, as K only requires a 200K executable that is split into a couple of modules.

J and K both have their place even if they are not being used to replace APL. The fact that they can serve these purposes in addition to replacing APL is just an added advantage.

Conclusions

There are some lessons. Like all lessons, their significance changes, but their core is now quite stable, and I don't think that I've seen reason to materially change these judgments for at least several years now.

Usage

By using J and K, I virtually Never Use APL anymore, and in the (increasingly rare) circumstances that I actually encounter some old piece of APL I now regularly re-write it into one of the newer languages. As time passes, and I have less and less legacy code this becomes an increasingly rare process.

String APL

K has become a sort of 'string APL' for me. With only a few keystrokes I am able to pick up a file, read it into memory, manipulate it in some significant way and then write out the result. About the only 'serious' string facility that I miss in K is regular expression handling. K's capabilities in this dimension are not very sophisticated, but K makes it easy to call on other facilities that might be present in the environment to perform such functions.

Integer APL

I find J to be particularly effective for handling integer problems. K, by design, is not very good at integer problems. Integers are thought of principally as 'pointers' to storage blocks. They are not much used to *compute* with. in J, on the other hand, the algorithms through which the handling of such numbers are both rich and effective, and the integration of *extended precision integers* with other data types is well thought out and easy to use.

Rich Number Types

J, in particular, is very rich in numerical data types and in functions that support basic mathematical constructs. For example, not only is there full support for complex variables, there is support for rational numbers, factoring, primes, polynomials, etc.

New Users

I would unhesitatingly point new users towards either J or K rather than towards APL. My principal quick arguments would have to do with typography, keyboard, cost, and the availability of an active and interested support community on the Internet.

Old Users

I really don't know what advice to give old users. Actually, that's not true. If pressed my advice would be to abandon APL and invest the time and energy required to learn whichever of J or K seemed to offer a natural structure that was best suited to the kinds of problems that an old user regularly had to solve. I would never presume to think that someone else would learn things with the same difficulty or ease that I do, anyway, but as testimonial experience I can say that I could use J in about a week and K in about two weeks. But, of course, such early usage was crude, cumbersome and caused substantial discomfort and 'pain'. However, in both cases, productivity came fast.

Future Prospects

As to future prospects, most of the work I know for APL programmers relates to legacy systems. Since there are no real J/K legacy systems, most J/K work is at least new work. For some this would likely be more interesting.

Summary

The trip from APL to J/K has been a journey well worth taking. I have surely learned as much from this trip as I did when I encountered APL itself for the first time in the 1960s. In the process of learning a lot, I also find myself in possession of some programming tools that I use almost daily in practical ways that range from quick simulation models to HTML screen scraping tasks.

I am unclear about whether J and K are truly languages of the future. What I am clear about, however, is that they are both enormously useful and effective languages – well worth the amount of energy that I had to invest to learn how to use them. I am equally clear that APL is a language headed for oblivion. I cannot speak to the issue of whether it is worth you investing the energy to give yourself an alternative or not. I can say, however, that it certainly was worth my effort for me.

And one of the lessons that I have learned from years of experience in this area is that it's very hard to make a profit by second guessing the smart guys. In this case, at least, I decided to avoid that problem and have, so far, been quite happy that I did.

Paritons and Cognitons

Towards a New Theory of Information

The late Gérard A. Langlet, translated & edited by Sylvia Camacho

Abstract

Recent developments in the sciences of Biology, Physics, Chemistry, Computing and Mathematics, together with interdisciplinary studies such as Linguistics, Music and Botany have been particularly impressive. Information handling is what these apparently disconnected disciplines have in common. This paper is the result of analysing ideas and concepts taken from all these viewpoints. The reduction of many algorithms to their most elementary components has shown that it is possible to use them to elucidate more and more natural phenomena. Not unexpectedly, the proposed model for Natural Information Processing is closely related to fractals such as are found in music, clouds, mountains, anatomy, astronomy, turbulence and earthquakes. The mathematical method is to use binary discrete logics to demonstrate the common underlying model. *Parity* is the essential, highly discontinuous, property involved in this theory. This should not surprise specialists in computer science or theoretical physics but may seem less familiar to biologists - although reproduction is indeed a parity and fractal problem. A second important property is *Symmetry/Asymmetry*. No attempt to create the model with continuous functions can be made because parity evolves abruptly, not smoothly, wherever it is found. Much work has still to be done: only observation and thinking can help.

Remarks for readers who know APL

This text has been written to expound a new theory which is, we hope, potentially able to explain many phenomena in a large variety of domains. It has been found and built with the help of APL over several years of patient research. The fundamental reason - or perhaps faith - was that APL is so powerful that it must comprise the foundations of information processing, if not of human thought. This is why the author never stopped revisiting, compressing and simplifying all his algorithms (rejecting parentheses, branches, arrays, trigonometry etc. and even, recently, all arithmetic) until he discovered that the most useful primitive was $\neq$ and the most general and fantastic idiom $\neq \setminus$ from which it is possible to rebuild, with the help of $\wedge$ and $\subset$, anything (see a related paper about the Sierpinski gasket). It is even possible to reconstruct a model of the Universe and to explain its properties ... which, after all, is not surprising if one knows that Nature was operating long before geometry, the theory of numbers, matrix algebra and continuous functions were invented by man.

The reader need not know *APL* at all. What is necessary, for expository purposes, is in the form of *Iverson's notation*. This paper pays a sincere homage to Ken Iverson, whose work indeed helps people to think and not only to program. It is also in homage to Joseph de Kerf whose *APL-CAM* Journal contributes to the renown of *APL* throughout the European Community and even all over the world.

Parity in bit sequences

Consider any sequence of information I in binary form e.g.

01100001011100000 1101100 (1)

The parity of a sequence may be defined as: 0 if the number of 1's is even
1 if the number of 1's is odd.

I contains 1 in 10 different positions, so given the above definition the parity of I is 0. Parity is also the sum of all items of I modulo 2.

Parity is often used when transmitting information on a line to check the quality of transmission, i.e. the integrity of the message. Usually the transmitter inserts an extra bit after each group of bits and the receiver checks that this bit has the same parity as the group; if not this latter is considered 'bad' and the receiver asks for a second transmission. This procedure is a 'checksum modulo 2'. A real checksum, i.e. a counting of all 1 bits, is done by all computers when files are written to tapes and disks and checked when reading from them.

Instead of adding 1 within an arithmetic register, initially set to 0, every time 1 is encountered in the bit sequence and then extracting the parity of the accumulated result, it is more efficient to propagate only the parity of successive bits using the properties of the Exclusive OR (XOR in computer jargon) in Boolean algebra.

Note: We shall use Iverson's mathematical notation in what follows because it has been standardised by ISO [ISO 1989]; all expressions are simple: they can be immediately verified and used 'as is' in interactive mode on many computers, in especial most of the personal ones.

XOR is denoted $\neq$ so that: $0 \neq 0$ is 0 (0 is not different from 0)

$0 \neq 1$ is 1

$1 \neq 0$ is 1

$1 \neq 1$ is 0 (1 is not different from 1).

This operation is commutative. Its contrary (NOT XOR) is denoted $=$ so that $0 = 0$ is 1, $0 = 1$ is 0, etc.

An arithmetic sum is denoted $+/$ so that $+/$ applied to any bit sequence gives the number of 1's within this sequence (upper case SIGMA in mathematics). Of course $+/$ can be applied to any sequence of numbers, integer or not, in order to produce its sum. The result is a scalar when the argument is a one-dimensional array. A

statement such as $+/I$ with I equal to the above sequence (1) would display the number 10 since I contains 10 bits set to 1.

Compare with the FORTRAN90 statement: $S=\text{SUM}(I)$ (with I a numeric vector, i.e. a one-dimensional array), followed by a PRINT or WRITE statement in order to display S (cf [Metcalf 1990] page 182).

The symbol $/$ denotes reduction. So, in $+/I$, vector I is reduced by $+$ and the result is automatically displayed because it is not assigned to any variable name. By contrast $S + +/I$ will assign the result to S without displaying it. So statement S , that is the variable name alone, displays the contents of S . The backslash symbol $\backslash$ denotes *scan*, i.e. a propagation or cumulation.

(There are equivalent statements in programming languages such as *LISP or C++ for parallel computers but not in FORTRAN90).

The result of $\backslash$ is an array of the same shape as the argument: if I is a vector the result is a vector of the same length. The first item of the result is always equal to the first item of the argument and does not depend on the function (e.g. $+$) which is being propagated. The propagation of addition is denoted $+\backslash$ and all intermediate sums are present in the result.

so $+\backslash 1234$ is 13610 (a vector)

while $+/1234$ is 10 (a scalar i.e. the last number of the result of $+\backslash$).

Other functions may be used with $\backslash$:

so $\times\backslash 1234$ is 12624 (i.e. $1!, 2!, 3!, 4!$ in conventional mathematics)

and $\times/1234$ is 24 (i.e. upper case PI in mathematics)

and $\neq\backslash 011000010111000001101100$

gives: 010000011010000001001000

while $\neq/011000010111000001101100$

is 0 (which is the last item of the result of its $\neq\backslash$).

Every item in the result of $\neq\backslash$ applied to any sequence of bits represents the accumulated PARITY from the beginning of the sequence (first item) to the current item. This may be considered as a theorem. The last (rightmost) item, also given by $\neq\backslash$, is the parity of the sequence (corollary).

Since there is no notation in mathematics to represent operations such as $\neq/\neq/=/\backslash$ or a few others we shall define later, we are grateful to Iverson for having invented his own linear notation which considerably simplifies the exposition of concepts and the demonstration of theorems and the development of algorithms for parallel computing. However, his use of indices, exponents and Greek letters will be avoided here.

The bricks of parity logics

In Iverson's notation [Iverson 1962], [ISO 1989], an important notion is 'scalar extension'.

For any scalar dyadic function such as $+$ $-$ $\times$ $\div$ $\neq$ etc. when one of the two arguments is a scalar (or, in many implementations, a one-item array which gets forced to a scalar) the scalar is extended, by intrinsic replication to the dimensions of the other argument. Thus scalar operations are applied itemwise within arrays of any shape and dimension, provided both arguments have the same shape and dimension or provided one argument is scalar or a one item array.

(This was a fantastic simplification and proves nowadays to be so useful for array processors and parallel computers that it has been introduced into many other computer languages).

Thus statements $2\ 2\ 2 + 5\ 6\ 7$ or $2 + 5\ 6\ 7$ or $5\ 6\ 7 + 2$ or $5\ 6\ 7 + 2\ 2\ 2$ are equivalent and produce $7\ 8\ 9$.

Let us come back to the parity domain and take any binary sequence such as:

V

1100110111100001

using only $\neq\ 0\ 1$ as hypothetical bricks, we can generate:

The dummy function whose output equals its input:

$0 \neq V$

1100110111100001

The binary negation NOT:

$1 \neq V$

0011001000011110

Death, flat encephalogram:

$V \neq V$

0000000000000000

Spacefill:

$1 \neq V \neq V$

1111111111111111

(The result of $V \neq V$ is negated)

(Expressions are executed from RIGHT to LEFT)

Let us take W as another sequence of bits with the same length as V :

W

1110010000001100

and when we compare: (The result of $V \neq W$ if negated)

$1 \neq V \neq W$

(Expressions are executed from RIGHT to LEFT)

1101011000010010

with:

$V=W$

1101011000010010 we see it is the same.

From the BRICKS 1 and XOR we have generated NOT and then NXOR. Since 0 is the intrinsic result of 1 XOR 1 it follows that 0 is NO LONGER an ELEMENTARY BRICK 'quod erat demonstrandum'.

Integrals and differentials

$+$ produces the sum of any numerical or logical sequence ('True' and 'False' being respectively designated 1 and 0). Then $+\backslash$ can be used to plot the integral of any function computed at regular intervals provided the number of intervals is big enough for a good approximation in the discrete domain. Similarly, $\neq\backslash$ may be used to plot a *parity integral* on 2 different levels (0 and 1) along the ordinate axis. Then $\neq\backslash\neq\backslash$ i.e. $\neq\backslash$ applied twice is the parity integral of the parity integral. Such a process can be iteratively applied and will lead to interesting results. But before explaining the consequences of these operations in the Boolean domain we note that:

While $+$ and $+\backslash$ may introduce errors in their numerical result because of limited computer precision for rational numbers, $\neq/$ and $\neq\backslash$ will never do so: their result is always exact for any bit sequence.

If $\neq\backslash$ is applied iteratively to any finite sequence of m bits, a PERIODICITY will appear after a finite number c (the cycle) of iterations.

A fundamental question arises: what is the inverse function of $\neq\backslash$? Given the result of $\neq\backslash$ on a sequence is it possible to reproduce the original sequence from it, i.e. to find the 'parity differential'? The answer is yes. The first item of $\neq\backslash$ applied to any sequence I is the first item of the sequence I , so this we already know. Let J be the result of $\neq\backslash I$ and let K be the result of dropping the first item of J and let L be the result of dropping the last item of J . Thus K and L have the same length permitting comparison. Compute $K \neq L$ obtaining M and 'stick' or 'catenate' it to the retained first item of J . This reproduces I .

Expressed in natural language this means that the parity differential I of J is the first item of J followed by the result of the application of the exclusive OR, itemwise, to the first and second items of J , then to the second and third items of J , etc. This is a Boolean finite difference on J .

This procedure is not so simple as its inverse $\neq\backslash$ as it involves cutting the sequence in two locations and sticking the pieces together again after itemwise application of $\neq$; this requires more energy. However, it is noticeable that up to here the only logical operation involved is $\neq$: *cut* and *stick* being structuring operations. Moreover, there is another way of doing it: the *wait-and-see* method.

The Pariton

The phrases *parity integral* and *propagation of the binary difference* are equivalent because the parity integral is the result of the propagation of the binary difference. After a while we shall use only the words *integral* or *integration* and *differential* and omit *parity*.

Let us define the PARITON as the Boolean MATRIX containing as rows, the differing bit sequences of successive parity integrals of an original bit sequence. (The PARITON may be the same as Prof. Stonier's hypothetical 'infor' [Stonier 1990]).

It can also be viewed as a linear cellular automaton (see [Wolfram 1986]). The initial state is a sequence of m bits, m being, to keep things simple, a power p of 2 (i.e. 2 4 8 16 32 etc). The matrix rows are the time-dependent (generations) or successive states of the automaton. If $p=0$ the automaton is static: the pariton has 1 row and 1 column and it contains the initial state. Its parity has the same value and so have all integrals or differentials, 0 or 1. (It behaves like the exponential function whose integral is itself).

For $p=1$, there are 4 different combinations for the initial sequences: 0 0, 0 1, 1 0, 1 1, i.e. the 2-bit representation of the numbers 0, 1, 2, 3. The automaton is static (i.e. unchanging) if the left bit is 0, i.e. for the first two possibilities. This can be generalised to show that if the leftmost half of any sequence is filled with 0 then changes are confined to the right hand half: the automaton is 'degenerate' being equivalent to that for $p-1$ instead of p . It also implies that, if m (the number of bits in the sequence) is not a power of 2 that the automaton if left-filled with 0's will be equivalent to the automaton with p set to the next integer capable of encompassing length m .

The parity integral of 1 0 being 1 1 and conversely, the first dynamic (oscillating) pariton is the square matrix containing these two sequences as rows and so having parity alternating between 0 and 1. An important consequence follows immediately. Such a matrix is the smallest generating pattern of the Sierpinski gasket and thus connects the pariton with *fractal geometry*. See [Langlet 1991].

The parity integral of 1 followed by any number of 0's contains only 1's. This results from the fact that the parity propagated from the left to the right, even to infinity, can only be 1. The parity integral of an all-1 sequence is an alternate sequence 1 0 1 0 ... (grey vector or zebra).

For $p=2$ the pariton will have 4 columns. For the reasons discussed above, only the cases for which there is at least one 1 in the two leftmost positions have to be discussed, i.e. 0 1 0 0, 0 1 0 1, 0 1 1 0, 0 1 1 1, plus the same states as these 4 having

1 instead of 0 in the leftmost position i.e. 1 0 0 0, 1 0 0 1, 1 0 1 0, 1 0 1 1 and 4 having 1 1 in the leftmost position i.e. 1 1 0 0, 1 1 0 1, 1 1 1 0, 1 1 1 1. These 12 states correspond to binary 4,5,6,7,8,9,10,11,12,13,14,15. Choosing any one of these as the initial state and iterating $\neq$ will cause the automaton to fall into one of 3 cycles: 4 7 5 6 4 ..., 9 14 11 13 9 ..., 8 15 10 12 8 ... Thus there will be altogether 3 different possible constructs with periodicity 4 for $p=2$ (or for $2 < m \leq 4$).

[Readers may find it helpful to play using the function below: SMC]

```

r=length cycle value;io;boo;mat
[1]  # show  $\neq$  scan cycle for boolean
[2]  # of given length and value
[3]  io+0  $\diamond$  r+,value
[4]  boo+(length $\rho$ 2) $\neg$ value
[5]   $\rightarrow$ ((2 $\wedge$ boo) $\neq$ value)/err  $\diamond$  mat+(1,length) $\rho$ boo
[6]  l1:boo $\neq$  $\neg$ boo  $\diamond$  mat+mat;boo
[7]  r+r,2 $\wedge$ boo
[8]   $\rightarrow$ ((1 $\wedge$ r)= $\neg$ 1 $\wedge$ r)/end  $\diamond$   $\rightarrow$ l1
[9]  err:'length ',( $\neq$ length),' insufficient for ', $\neq$ value
[10]  $\rightarrow$ 0
[11] end: $\rightarrow$ (32< $\rho$ mat)/out
[12] mat
[13] out:'cycle of ',( $\neq$ 1+ $\rho$ r),' values ', $\neq$ r

```

The periodicity of integrals or, conversely, of parity differentials – that is the length of the cycle (c) will always equal 2 to the power p (integer) for non-degenerate sequences.

An immediate consequence is that the parity differential for any non-degenerate sequence of bits can also be obtained as the (c-1)th parity integral, without cutting and sticking the sequence as shown above (this is the WAIT-AND-SEE method). No algorithm other than the iteration of $\neq$ is necessary. The more difficult way proposed above for parity differential may be ignored unless it is required as a 'shortcut' when periodicity (c) is very large or as a possible repair mechanism when a 'mutation' (unknown error) has disrupted somewhere the perfect regularity of the integrating process.

The cyclic memory

The pariton, like a matrix, is a topological entity. It can take numerous shapes. It is not bound by any Euclidian constraint (not even by the first postulatatum). As soon as periodicity has been quantified we can consider the pariton either as a DISCRETE CYLINDER or as a flat matrix. Every row will lie on a generating line and the successive parities (last items) of every row form the apices of a polygon which, if p is large enough, will look like a circle or section of a cylinder. So let us

examine more attentively the properties of the pariton as a matrix keeping in mind its potential cylindricity.

Considering that successive rows represent different temporal states of the original information in the I sequence and, starting from ANY row randomly, we can retrieve this sequence after performing from 1 to $c-1$ integrations. When c is large enough, information coded in an alphabet of about 32 symbols, e.g. the Roman alphabet and the blank plus some punctuation and separators (.:?-) extracted from an 8-bit ensemble i.e. 256 possible configurations (as is found on almost all computers), the probability that any other row than the one we look for could also contain meaningful information with all successive sequences of 8 bits belonging to 32 among 256 is already quite low (0.2% for 4 characters i.e. 32 bits). It decreases very quickly as c increases. Such a pariton occupies 1024 bits i.e. 128 bytes.

A 128-character sentence (about one and a half lines of text in a book) considered as a body of information, generates a pariton with 1024 rows and 1024 columns occupying 128 kilobytes. This is not much in comparison with the memory available on desk-top computers and negligible by comparison with human brain capacity. We shall see that most practical applications discussed here involve small paritons only.

From the rightmost column R , which contains the parities of all rows and which is, on the cylinder, a circular sequence of bits with no origin and no end and thus atemporal, one can deduce the whole cylinder and thence find the original information. This is done in the following way: we define the minus-one-step circular-shift operation, which is not arithmetic, calling it CSM1. For reasons of symmetry we also define CSP1 i.e. the plus-one-step circular-shift operation.

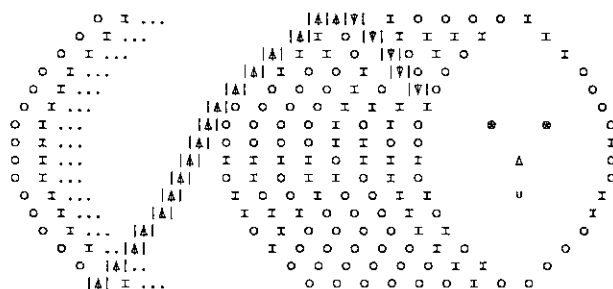
Some implementations of Iverson's notation allow direct definitions such as $CSM1 \leftarrow^{-1} 1 \phi$ and $CSP1 \leftarrow^{+1} 1 \phi$ using ϕ (the circular shift symbol). Circular lists exist in LISP where such definitions could also be established easily. Even in FORTRAN90 a CSHIFT statement has been introduced and runs beautifully on the Connection Machine.

$R \times CSM1$ produces Q , i.e. the last but one column of the pariton matrix (this is a parallel differential operation as shown above). The same application to Q produces P i.e. the column to the left of Q , etc. for $c-1$ iterations.

If the original sequence was degenerate the number of iterations becomes smaller. Just stop when all bits in the same column are 0 or 1 and fill the left columns with 0's in order to get a square matrix. The application of CSP1 instead of CSM1 would yield a completely different pariton: the direction of the shift, i.e. of the spatial orientation (spin) is of primary importance. On the other hand, when the

pariton is filled storing the first integral in the last row, the second integral in the last-but-one row, etc. until the c th integral, i.e. the original sequence in the highest row, then this pariton is the image of the other in a horizontal mirror. So CSP1 is the correct appication to retrieve the mirrored pariton from circular memory, the spin of it being also reversed.

The pariton itself is highly asymmetric, unlike a magic square or an optical hologram. We have not yet explored the properties of the diagonals, which on a cylinder correspond to helices.



Is the worms head a cogniton?

This procedure shows that the circle on the right contains all the information: it resembles the toruses of the ferrite memory of early computers. We will call it a COGNITON. Conversely the left column only conveys the information that the first bit of the original sequence is either 0 or 1. In between the pariton contains only fuzzy circles, i.e. with the correct intermediate coding for the sequence from the first bit to the bit number matching the column position in the matrix.

The period of the left half of the pariton is half that of the whole (c) pariton and the period of the left fourth of the pariton is one fourth of c etc. up to the first column in which all bits are the same and which has, of course, a period of 1. Even if all the columns except the last but one of a pariton are destroyed it is always possible to start from this remnant. Just one bit (the last one on the right) may be wrong as one chooses arbitrarily between 0 and 1. The probability of reading correctly the whole message when the last two columns are scratched or corrupted is still high. One may get a mistake, a misspelling of the last character. Anyway everything that has been said about the pariton has symmetric properties. If one has taken the precaution to code an anti-pariton i.e. starting from left to right, there will be significant redundancy.

32-bit Pariton Matrix coding the 4-character word 'STOP'

(In most computers in which the 128 ASCII characters are the FIRST half of the 256 ones in an 8-bit set.)

Parity Integral No.

0 1 1 0 0 0 1 1 0 0 1 1 0 0 1 1 1 1 0 0 0 1 0 1 0 0 1 1 0 0 0 0 0	1
0 1 0 0 0 0 1 1 1 0 1 1 1 0 1 0 1 1 1 1 0 0 1 1 1 0 1 1 1 1 1 1	2
0 1 1 1 1 1 0 1 0 0 1 0 1 1 0 0 1 0 1 0 0 0 1 0 1 1 0 1 0 1 0 1	
0 1 0 1 0 1 1 0 0 0 1 1 0 1 1 1 0 0 1 1 1 1 0 0 1 0 0 1 1 0 0 1	4
0 1 1 0 0 1 0 0 0 0 1 0 0 1 0 1 1 1 0 1 0 1 1 1 0 0 0 1 0 0 0 1	
0 1 0 0 0 1 1 1 1 1 0 0 0 1 1 0 1 0 0 1 1 0 1 0 0 0 0 1 1 1 1 0	
0 1 1 1 1 0 1 0 1 0 0 0 0 1 0 0 1 1 1 0 1 1 0 0 0 0 0 1 0 1 0 0	
0 1 0 1 0 0 1 1 0 0 0 0 0 1 1 1 0 1 0 0 1 0 0 0 0 0 0 1 1 0 0 0	8
0 1 1 0 0 0 1 0 0 0 0 0 0 1 0 1 1 0 0 0 1 1 1 1 1 1 1 0 1 1 1 1	
0 1 0 0 0 0 1 1 1 1 1 1 1 0 0 1 0 0 0 0 1 0 1 0 1 0 1 0 1 0 1 0	
0 1 1 1 1 1 0 1 0 1 0 1 0 0 0 1 1 1 1 1 0 0 1 1 0 0 1 0 0 1 1 0	
0 1 0 1 0 1 1 0 0 1 1 0 0 0 0 1 0 1 0 1 1 1 0 1 1 1 0 0 0 1 0 0	
0 1 1 0 0 1 0 0 0 1 0 0 0 0 0 1 1 0 0 1 0 1 1 0 1 0 0 0 0 1 1 1	
0 1 0 0 0 1 1 1 1 0 0 0 0 0 0 1 0 0 0 1 1 0 1 1 0 0 0 0 0 1 0 1	
0 1 1 1 1 0 1 0 1 1 1 1 1 1 1 1 0 0 0 0 1 0 0 1 0 0 0 0 0 1 1 0	
0 1 0 1 0 0 1 1 0 1 0 1 0 1 0 1 0 0 0 0 0 1 1 1 0 0 0 0 0 0 1 0	16
0 1 1 0 0 0 1 0 0 1 1 0 0 1 1 1 1 1 1 0 1 0 0 0 0 0 0 0 0 1 1 1	
0 1 0 0 0 0 1 1 1 0 1 1 1 0 1 0 1 0 1 1 0 0 0 0 0 0 0 0 0 1 0 1	
0 1 1 1 1 1 0 1 0 0 1 0 1 1 0 0 1 1 0 0 1 1 1 1 1 1 1 1 1 0 0 1	
0 1 0 1 0 1 1 0 0 0 1 1 0 1 1 1 0 1 1 0 1 0 1 0 1 0 1 0 1 1 1 0	
0 1 1 0 0 1 0 0 0 1 0 0 1 0 0 1 0 1 1 0 1 1 0 0 1 1 0 0 1 1 0 0	
0 1 0 0 0 1 1 1 1 1 0 0 0 1 1 0 1 1 0 1 1 1 0 1 1 1 0 1 1 0 0 0	
0 1 1 1 1 0 1 0 1 0 0 0 0 1 1 0 1 1 1 1 0 1 1 0 1 1 1 1 0 1 0	24
0 1 0 0 0 1 1 1 1 1 0 0 0 1 1 0 1 1 1 0 1 1 0 1 1 1 1 0 1 0 1 0	
0 1 1 1 1 1 0 1 0 1 0 1 0 0 0 1 1 0 0 0 1 1 1 0 0 1 1 1 0 1 1	
0 1 0 1 0 1 1 0 0 1 1 0 0 0 0 1 1 0 0 0 0 1 1 1 0 0 1 1 0 1 1 1	
0 1 1 0 1 0 1 0 0 1 1 0 0 0 0 0 1 0 0 0 0 1 1 1 0 1 0 0 1 0 1	
0 1 1 1 1 0 1 0 1 0 1 0 1 0 0 0 1 1 0 0 0 1 1 1 0 0 1 1 1 0 1 1	
0 1 0 1 0 1 1 0 0 1 1 0 0 0 0 0 1 0 0 0 0 1 0 1 1 1 0 1 0 0 1 0	
0 1 1 0 0 1 0 0 0 1 0 0 0 0 0 0 1 1 1 1 1 0 0 1 0 1 1 0 0 0 1 0	
0 1 0 0 0 1 1 1 1 1 0 0 0 0 1 1 0 1 1 0 1 1 1 0 1 1 1 0 1 0 0 0	
0 1 1 1 1 0 1 0 1 0 1 0 0 0 1 0 0 1 0 0 1 0 1 1 0 0 1 0 0 0 0	32
0 1 0 1 0 0 1 1 0 0 0 0 1 1 1 0 0 0 1 1 1 0 1 0 1 0 1 0 0 0 0	

[-----S-----] [-----T-----] [-----O-----] [-----P-----]

COGNITON
circular memory

Note that in rows 8,16 and 24, 'S' is reproduced, ('ST' in row 16).

The fractal character appears in many places (0 triangles). Note that the 32 bit Sierpinski gasket would be 1 followed by 31 zeros.

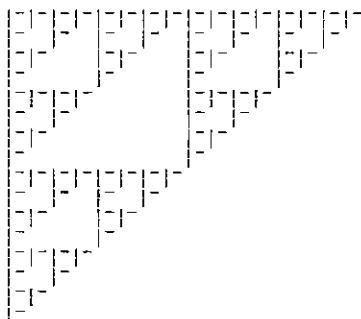
HELIX TRANSFORM of 'STOP' (diagonal from left to right, bottom to top):

01000110000001110001101100011110

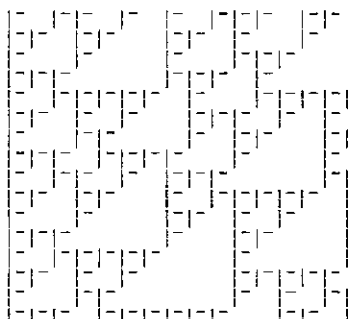
COGNITON for word 'STOP' (ATEMPORAL: with no origin and no end):

01111000110011001110000100110000

The FIRST BIT is the LAST BIT of the first integral i.e. the PARITY of the bit conversion of 'STOP' (matches row 32 in which 14 bits are 1).



Sierpinski Gasket



Matrix of 4-character word 'STOP'

The helical properties of the pariton

In any pariton any item of the main diagonal also results from the combination by $\neq$ of the item on its left and on the item just above. Such a construct can be deduced from rule 90 of the theory of automata (see [Wolfram 1986]) by an affine transform (cf. the theoretical discussion on the Sierpinski gasket in [Langlet 1991]). This rule results from the properties of the Pascal triangle about the parity of combinations. Wolfram's rule is used e.g. in Barbé's fractal model [Barbé 1988] as well as in a more recent proposal in the field of artificial intelligence with two orthogonal inputs for a neurone in a neural network simulation [Dubois 1990].

But the most interesting diagonal is not this one; it is the other one which runs from the left bottom to the right top (and would be the main diagonal in a pariton filled towards instead of downwards by successive integrals).

In any pariton, if any sequence I produces a diagonal D , a sequence equal to D produces a diagonal which contains I .

So D will be named $-I$ from now on (no confusion can arise from the minus sign because the pariton involves no arithmetic).

It means that $-I$ is the helix inverse or helix transform of D .

($-I$ could be written I^{-1} by analogy with matrix algebra).

As a corollary, any row of the matrix has its corresponding helix inverse on the cylinder.

And the other diagonal D' (the main diagonal in our setting) also has some properties. If I is the information the diagonal for D' , taken as the information, is not I . It will be necessary to iterate this regressive application many times (a power of 2) in order to obtain I again.

So the properties of paritons permit a full decoding from any helical path providing no bits have changed in the selected direction.

There is a strong anisotropy between the two possible helix orientations: one of them provides

from a pariton of any size an immediate transform capability but the other only has a very-long-term transform capability.

Moreover, any labyrinthine walk from any row on the left to any row on the right, provided the full path is known (e.g. 3 steps up, 2 steps right, 1 step down etc. just like walking in a ruined city – briefly any 'taxi run', after the theory of automata) should allow the pariton to be reconstructed with full integrity.

>> The pariton is a kind of:

easy to learn (mastery of only one Boolean application is required).

secure (it has – or rather it is – an automatic parity check)

asymmetric (the consequence of monodirectional propagation).

discrete (by definition).

hologram-like (at any scale without loss of precision).

MEMORY

```

r=length matrix value;Dio;boo;vec
[1] a show %\ scan cycle for boolean
[2] a of given length and value
[3] Dio=0 & r+(0,length)P0
[4] boo=(lengthP2)Tvalue
[5] ->((2+boo)%value)/err & vec+value
[6] ll:boo+%boo & r+r;boo
[7] vec+vec,2lboo
[8] ->((1+vec)=-1+vec)/0 & -ll
[9] err:'length ',(%length),' insufficient for ',%value

r=helix nums;Dio;c;left:right
[1] a create nums half of helix transform table
[2] Dio=0 & c+1
[3] r= 3 0 % 32 1 Pnums
[4] left=@(6P2)Tnums
[5] right= 32 6 P0
[6] ll:-(32=c+c+1)+end
[7] a boolean of diagonal into c row of right hand column
[8] right[c;]+Phi(0 0)Phi(6 6)+6 matrix nums[c]
[9] ->ll
[10] end:r+' ',r,' ',%left
[11] r=r, 3 0 % 32 1 P2+@right
[12] r=r,' ',%right

```

[The above functions may be used to create the table below – SMC]

Helix Tranform for all 6-bit configurations

N	produces	N'	N	produces	N'
0	0 0 0 0 0 0 0	0 0 0 0 0 0 0	32	1 0 0 0 0 0 0	51 1 1 0 0 0 1 1
1	0 0 0 0 0 0 1	1 0 0 0 0 0 0 1	33	1 0 0 0 0 0 1	50 1 1 0 0 0 1 0
2	0 0 0 0 0 1 0	3 0 0 0 0 0 1 1	34	1 0 0 0 0 1 0	48 1 1 0 0 0 0 0
3	0 0 0 0 0 1 1	2 0 0 0 0 0 1 0	35	1 0 0 0 0 1 1	49 1 1 0 0 0 0 1
4	0 0 0 0 1 0 0	5 0 0 0 0 1 0 1	36	1 0 0 0 1 0 0	54 1 1 0 1 1 1 0
5	0 0 0 0 1 0 1	4 0 0 0 0 1 0 0	37	1 0 0 0 1 0 1	55 1 1 0 1 1 1 1
6	0 0 0 0 1 1 0	6 0 0 0 0 1 1 0	38	1 0 0 0 1 1 0	53 1 1 0 1 0 1 1
7	0 0 0 0 1 1 1	7 0 0 0 0 1 1 1	39	1 0 0 0 1 1 1	52 1 1 0 1 0 0 0
8	0 0 0 1 0 0 0	15 0 0 0 1 1 1 1	40	1 0 1 0 0 0 0	60 1 1 1 1 1 0 0
9	0 0 0 1 0 0 1	14 0 0 0 1 1 1 0	41	1 0 1 0 0 0 1	61 1 1 1 1 1 0 1
10	0 0 0 1 0 1 0	12 0 0 0 1 1 0 0	42	1 0 1 0 1 0 0	63 1 1 1 1 1 1 1

11	0	0	1	0	1	1	13	0	0	1	1	0	1	43	1	0	1	0	1	1	62	1	1	1	1	0
12	0	0	1	1	0	0	10	0	0	1	0	1	0	44	1	0	1	1	0	0	57	1	1	1	0	0
13	0	0	1	1	0	1	11	0	0	1	0	1	1	45	1	0	1	1	0	1	56	1	1	1	0	0
14	0	0	1	1	1	0	9	0	0	1	0	0	1	46	1	0	1	1	1	0	58	1	1	1	0	1
15	0	0	1	1	1	1	8	0	0	1	0	0	0	47	1	0	1	1	1	1	59	1	1	1	0	1
16	0	1	0	0	0	0	17	0	1	0	0	0	1	48	1	1	0	0	0	0	34	1	0	0	0	1
17	0	1	0	0	0	1	16	0	1	0	0	0	0	49	1	1	0	0	0	1	35	1	0	0	0	1
18	0	1	0	0	1	0	18	0	1	0	0	1	0	50	1	1	0	0	1	0	33	1	0	0	0	1
19	0	1	0	0	1	1	19	0	1	0	0	1	1	51	1	1	0	0	1	1	32	1	0	0	0	0
20	0	1	0	1	0	0	20	0	1	0	1	0	0	52	1	1	0	1	0	0	39	1	0	0	1	1
21	0	1	0	1	0	1	21	0	1	0	1	0	1	53	1	1	0	1	0	1	38	1	0	0	1	1
22	0	1	0	1	1	0	23	0	1	0	1	1	1	54	1	1	0	1	1	0	36	1	0	0	1	0
23	0	1	0	1	1	1	22	0	1	0	1	1	0	55	1	1	0	1	1	1	37	1	0	0	1	0
24	0	1	1	0	0	0	30	0	1	1	1	1	0	56	1	1	1	0	0	0	45	1	0	1	1	0
25	0	1	1	0	0	1	31	0	1	1	1	1	1	57	1	1	1	0	0	1	44	1	0	1	1	0
26	0	1	1	0	1	0	29	0	1	1	1	0	1	58	1	1	1	0	1	0	46	1	0	1	1	0
27	0	1	1	0	1	1	28	0	1	1	1	0	0	59	1	1	1	0	1	1	47	1	0	1	1	1
28	0	1	1	1	0	0	27	0	1	1	0	1	1	60	1	1	1	1	0	0	40	1	0	1	0	0
29	0	1	1	1	0	1	26	0	1	1	0	1	0	61	1	1	1	1	0	1	41	1	0	1	0	1
30	0	1	1	1	1	0	24	0	1	1	0	0	0	62	1	1	1	1	1	0	43	1	0	1	0	1
31	0	1	1	1	1	1	25	0	1	1	0	0	1	63	1	1	1	1	1	1	42	1	0	1	0	1

Up to 6 bits, the Helix inverse (or transform) of any sequence is also a generating line (row) on the same cylinder.

This table also gives helix inverses for 2 and 4-sequences if one ignores the zeros on the left.

2 PRODUCES 3 and 3 PRODUCES 2 (with the two rightmost bits); This couple is the Sierpinski gasket generator (cf. [Langlet 1991]).

The complement of the corresponding Matrix $\begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$ is $\begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$

When using conventional matrix algebra, e.g. to perturbate 'invariant subspaces' with a supposed continuous variation of eigenvalues, if the leftmost zero of the second row becomes EPSILON then difficulties appear: see [Stewart & Sun 1990] for a full mathematical exposition of the subject within the matrix perturbation theory. This cannot happen with the discrete pariton theory.

Pariton Dynamics

If one breaks the corners off the following discrete triangle what remains?

```

  o
 o o
 o  o
o o o o

```

The answer is:

```

  o o
 o  o
 o o

```

Knowing that the upper pattern generates the Sierpinski triangle by autosimilarity, the lower pattern – a hexagon which looks like the structure of the graphite crystal – will create, if we add intermediate points, a pattern of scaling dodecagons. Our brain which tries to find contours everywhere, creating the illusion of continuity, will 'see' circles. This construct will give a picture very close to the famous problem invented by Apollonios of Perga (circa 200 BC) which although a priori continuous is historically the first approach to fractals in plane geometry, long before Cantor, von Koch, Julia, Sierpinski, Mandelbrot and Menger. Let us add that the paradox of Achilles and the turtle can also be considered as a fractal. Solitaire and many other games are fractal ancestors of the cellular automata proposed by Conway and Wolfram.

A cylindrical pariton, seen from the distance along its axis looks like a circle, though it is always a 'power-of-2-gon' without sides. If we consider it as a rotating drum we can build a set of gear wheels. Then the pariton is able to deliver its information (by electric or magnetic influence if we associate with the bits some charges or spins) to many other cylinders: this is exactly what a laser printer does (unfortunately producing only one copy at a time). Pixels or characters are also processed this way by rotary printing presses when publishing books and newspapers.

Several networks can be imagined which will tile the plane with circles and transmit information to other circles at any scale without spending much energy: see the recent study in [Herrmann 1990] about tectonics and turbulence, naturally strongly connected to fractal geometry. Also consider Kolmogoroff's theory in thermodynamics. Refer also to [Bergé 1988 & later] about Bénard cells in convection.

There is a controversy in crystallography: many crystal structures which are conventionally described by Wyckoff positions in Bravais lattices as in the International Tables (Hahn 1983) may be better understood as superimposed layers which, up to a certain extent, look like the Herrmann model [Lima de Faria 1969]. See the examples of spinel and hodgekinsonite in [Langlet 1975]. Many crystals are somehow Apollonian models in 3-D. What is the model of the molecule of methane, the precursor of organisms (after Stanley Miller's experiment)?

To build such constructs (with inner as well as outer tangency) in 2D, 3D and in any hyperspace of dimension N with combinations of $N+1$ hyperspheres you can try the fast general method given in [Langlet 1979]. This was found using the power of Iverson notation.

Is external motion necessary to transmit information?

If the mechanism of propagation of the binary difference is permanent along the generating lines of the drum, the answer is NO. Consider the following schematic construct:

Take 3 fibres, neural columns, pipes or any other objects of more or less the same shape. Suppose that one helix of the first fibre contains a sequence of information D ; then put the second fibre adjacent to the first one by the generating line which contains $\neg D$, i.e. the inverse of the information on the first fibre. The neighbouring generating line will contain the information NOT $\neg D$, i.e. the Boolean negation of the inverse of D . Automatically one helix of the second fibre will contain $\neg \neg D$ (i.e. D except for the first bit of the parity which remains reversed; but anyway this content is not for the moment used). If a third fibre is put side by side along the second, the same effect of an inverse operation will occur automatically: the parity of the first bit also takes its original value. As a consequence one among the helical paths of the third fibre should contain D .

In fact the choice of the adjacent generating line has little importance: if the contacting generating line is randomly chosen on the first fibre it will contain one of the integrals of the inverse of D . So if the contacting line is also randomly chosen between the second and third fibre, it will also contain one of the integrals of the inverse of D . And one of the helices of the first fibre has to contain D . One can then add more fibres and clone information D at will, even at various scales, using the above described spur-gear mechanism. The analogy with the negative copy in photography or with matrices in the recording industry is complete although here a single bit makes the difference between negative and positive 'prints'.

For any sequence of bits the length of which is a power of 2, if D is the helix inverse of 1, the helix inverse of NOT D only differs from 1 by the parity of the first bit.

The best lightweight model to illustrate this is a pack of cigarettes with 20 pieces (25 in Belgium) on which it is possible to draw helices – it is easier on the cork of the filter, which moreover has approximately the proportions of the regular pariton i.e. a length equal to circumference; one can unfold it, draw a diagonal and stick it again around the cotton wool.

Several mountings are feasible. Let us replace the second fibre by a double-size fibre (a cigar – cigars are made rolling tobacco leaves diagonally so that helices are visible: no more drawing is necessary!). This corresponds to a doubling of the number of rows in the pariton matrix; one now sticks the last row before the first row after continuing integration for $2c$ cycles instead of c . The cigar may then be viewed as a *double helix*.

In this case two opposite generating lines of the second fibre always contain the same sequence and if the third fibre is in the same plane as the other two, thus adjacent on the other side, then the original information D is present on the helix of the third fibre which starts in the same plane; however the third fibre is rotated by half a turn (180 degrees) with respect to the first one.

A giant second fibre would allow us to reproduce, on its periphery, a large quantity of clones of the first fibre's information or to catenate several sequences coming from several fibres, using symmetry properties again to ensure synchronism.

The cost of a pariton

The mass m of information within a sequence l is the number of its bits.

Each iteration (integration) costs m times the elementary quantum of cost necessary to apply $\neq$ i.e. XOR to two bits, store the result at the appropriate address and progress by one step along the row (elementary scalar cycle cost).

But the cost of keeping the whole pariton in memory will also be proportional to its size (it is not necessary to keep anything else), which is at the minimum if no 0-filling on the left is done, c times m . Then we must integrate c times. The cost will be the product of these factors.

However, at a given scale, when coding between m and c bits i.e. objects which have approximately the same length (m greater than $c+2$ and not greater than c), or if c is fixed so as to code also smaller sequences in larger paritons, all having the same number of rows, we can take for the elementary unit cost the cost of one complete iteration in one row i.e. the cost of $\neq$ as a whole (elementary vector cycle cost).

Then we shall have to spend mc^2 to create a pariton of mass m .

Pariton and Order

Maxwell would have been fully satisfied to observe that the pariton does his hypothetical demon's work. A rapid glance at any pariton shows that the organisation of information is 'nil' on the left side and maximum on the right side (the cyclic memory).

The binary integral mechanism is a *self-associative auto-organiser*.

The pariton model may indeed be applied 'as is' to neural networks (see [Dubois 1990]), to image processing [Langlet 1991] without any computing, once the

propagation of the difference is established within any system as the main, thus *vital*, mechanism. Is the pariton a prototype for the computers of the next generation?

What is Maxwell demon's task?

To segregate molecules or extract consistent information from noise or to separate isotopes involves the same concept: sorting, which indeed corresponds to the most important algorithm that millions of computers perform every day especially in financial applications or in decision making (strategy). But sorting is also the key to many problems frequently covered by graph theory, the end of which is optimisation (see e.g. [Langlet 1989, 1990] about the Travelling Salesman Problem, correlated also with circuit design, vision, routing, cutting and storing) and by conventional expert systems. Maxwell's demon is the best theoretical expert ever invented by the human brain although not yet fully realised by human technology. The pariton is a cheap model for it.

A huge side effect of the Maxwell demon outcome was the *entropic paradox*. It implies that the pariton is highly related to thermodynamics and energy conversion. There is no reason why what is true for information should be different for molecules or other entities. The discussion is open and pending.

Pariton, Theoretical Chemistry and Morphogenesis

We shall not develop this point in detail here.. However it is worth noting that the pariton model fits the premises, the remarks or the conclusions expressed by many authors in these fields. In particular parity propagation tries to equalise permanently any 'donor-acceptor' couple and the model respects the Löwdin theorem, see [Reed 1988 p903 eq 7b].

We hope it may help specialists to simplify the increasing complexity of computer simulations which, even when super computers are used, are still not able to explain with sufficient accuracy the properties and the interactions of the large molecules which are involved at all steps in biology and biochemistry. And perhaps they will then take into account the fact that temporal cycles are important. For cells see e.g. [He 1991].

Symmetry and asymmetry are closely related to isotropy and anisotropy in crystals and molecules. All organised structures which are observed in nature respect the principle of the highest possible symmetry which is compatible with the most balanced distribution of parities; electric charge is a parity problem, so is relative size. Combining these last two parities optimally leads to many crystal structures. But at a more macroscopic level than the atomic construction set itself, 'parity equilibrium' also leads to helical structures, which themselves have a tendency to form secondary, then tertiary etc...larger helical structures or quasi-

planar networks (see the 4-fold symmetry of duck skin, the plywood-like assemblage of fibrillae) or even both at the same time (chicken embryo cornea, coleopter endocuticle, coelacanth scale, after [Bouligand 1980] gen. ref.).

Specific References [applying to pages 399 to 416 of the original edition]

Let us extract a quotation from [Reed, 1988], ref. 135

<< One of the principal objects of theoretical research in any department of knowledge is to find the point of view from which the subject appears in its greatest simplicity.>>

Buckingham, A.D.; Fowler, P. & Hudson, J. *Theoretical Studies of van der Waals Molecules and Intermolecular Forces*, Chemical Reviews, 88.6 p971 (supermolecule, intermolecular exchange, Pauli principle) (1988)

Grant, J.A.; Williams, R.L. & Sheraga, H.A. *Ab Initio Self-Consistent Field and Potential-Dependent Partial Equalization of Orbital Electronegativity Calculations . . .*, Biopolymers, 30 p935 (formulae 12 & 13) (1990)

He, Q.; Skog, S. & Tribukait, B. *Cell cycle related studies on thymidine kinase and its isoenzymes in Erlich ascites tumours* Cell Prolif. 24.4 fig. 1 (1991)

Hobza, P. & Zahradnik, R. *Intermolecular Interactions between Medium-Sized Systems . . . Successes and Failures* Chem. Rev. 88 (V. Prospects) p894 (1988)

Price, S.L. *The limitations of isotropic site-site potentials to describe a N2-N2 intermolecular potential surface*. Molecular Physics, 58.3 p654 (1986)

Reed, A.E.; Curtiss, L.A. & Weinhold, F. *Intermolecular Interactions from a Natural Bond Orbital, Donor-Acceptor Viewpoint* Chem. Rev. 88 p923 (1988)

Wheatley, R.J. & Price, S.L. *An overlap model for estimating anisotropy of repulsion* Molecular Physics 69.3 p507 (1990)

References [Pages 428, 429 of original edition: SMC]

Avnir, D. (ed.) *The Fractal Approach to Heterogeneous Chemistry*, J. Wiley & Sons Ltd. ISBN 0 471 91723 0 (1989) see Kopelman

Barbé, A. *Invariant Properties in the Binary Difference Field under a certain Conservation Law* Fractal Aspects of Materials, Disordered Systems, Materials Research Society extended abstracts, p171 (1988)

Bergé, P., Pomeau, Y. & Vidal, Ch. *L'Ordre dans le Chaos*, ISBN-2-7056-5980-3, 2nd ed., Hermann, Paris, a: p25 [van der Pol], b: p51, c: p90, d: p225 (1988) [in French]

Bouligand, Y. *La Morphogénèse, de la Biologie aux Mathématiques*, Maloine, Paris, ISBN 2-224-00654-3, Contribution by Favard, P. & Bouligand, Y. Fig 2, p106, Contr. by Bouligand, Y. p121 (1980) [in French]

Caverie, P. *Contribution à l'Étude des Interactions Moléculaires*, Thèse de Doctorat d'État ès-Sciences Physiques, Université Paris VI, CNRS Nr 8214 pII&III (March 1973) [French]

Dubois, D. *Self-organisation of fractal objects in XOR rule-based multilayer Networks*, Neuro-Nimes '90 p555 (1990)

Dumontier, M. & Langlet, G.A. *Taxinomie Linguistique*, Colloque de Taxinomie UITF, UNESCO, Paris (Nov 1990) [French]

Eigen, M. & Schuster, P. *The hypercycle, a Principle of Natural Self-Organisation*, Springer-Verlag, Berlin Heidelberg New York, ISBN 3-540-09293-5 [German] & 0-387-09293-5

[English] (1979)

Hahn,T.(ed), Arnold,H.; Berthaut,E.; Billiet,Y.; Buerger,M.; Burzlaff,H.; Donnay,J.D.H.; Fischer,W.; Fokkema,D.S.; Hahn,T.; Klapper,H.; Koch,E.; Langlet,G.A.; Vos,A.; de Wolff,P.M.; Wondratschek,H.; Zimmermann,H.; *International Tables for Crystallography, Vol. A, Space-group Symmetry*, D. Reidel Publishing Company, Dordrecht:Holland/Boston:USA/Lancaster:GB, ISBN 90-277-2280-3 (1983, rev. 1987) Herrmann,H.J.; Mantica,G. & Bessis,D. *Space-Filling Bearings*, Phys. Rev. Letters 65.26 (1990)

ISO 8485-E/F, *APL Standard*, International Standards Organisation, Geneva, Switzerland (1989) [E:English F:French]

Kopelman, R. *Diffusion-controlled Reaction Kinetics* p296 in [Avnir 1989]

Langlet,G.A. *Extension of the FIGATOM Program to the automatic Plotting of Layers in Close-Packed Structures*, J. Appl. Cryst. 8, p515 (1975)

Langlet,G.A. *New Fast Direct Solution to the Problem of the Sphere Tangent to Four Spheres*, Acta Cryst. A35 p836-7 (1979) see also: *APL et les Empilements de Sphères*, APL-CAM Journal, Belgium, 12.3 p582-590 (1990) [French]

Langlet,G.A. *The Travelling Salesman Problem*, APL Quote-Quad, ACM Press, 20.4 p228 (1990)

Langlet,G.A. *Théorie des Images Fractales*, in *La Technologie de l'Image*, Academia, Louvain-la-Neuve, Belgium. ISBN 2-87209-125-4 (revised edition) (1991) p48-62 [French]

Langlet,G.A. *Variations sur Serpinski, Pascal et Fibonacci*, APL-CAM Journal, BACUS, Belgium, 13.2 (Apr 1991) [French]

Langlet,G.A. *The Dual Structure of Ordered Trees*, APL91, Stanford University, California, USA, APL Quote-Quad, ACM Press, 21.4 (July 1991)

Locquin,M.(ed) *Aux Origines de la Vie*, Fayard/fondation Diderot, Paris, Contribution by Lépinard,D p159 (1987) [French]

Locquin,M. & Langlet,G.A. *Factotum Mycoloc, progiciel interdisciplinaire*, UITF, Paris (1989)

Lima de Faria,J. & Figueiredo,M.O. Z. Kristallogr. 130, p54-67 (1969) see also

Langlet,G.A.; Figueiredo,M.O. & Lima de Faria,J. *The Void Program*, J. Appl.Cryst. 10, p21-23 (1977)

Mandelbrot,B.B. *Les Objets Fractals*, Flammarion, Paris, 3rd ed. ISBN 2-08-211188-1 a:p50 b:p149 (1989)

Metcalfe,M. & Reid,J., *Fortran90 Explained*, Oxford Science Publications, ISBN 0-19-853824-3 p182 (1990)

Peitgen,H.O. & Saupe,D. (ed) *The Science of Fractal Images*, Springer-Verlag, New York, ISBN 0-387-96608-0, Appendix C, p282 (1988)

Pullman,A. & Pullman,B. *Quantum Biochemistry*, Interscience Pub. J. Wiley & Sons, New York (1963) see also *Les Théories Électroniques de la Chimie Organique*, Masson, Paris, p129 fig. 4 (1952) [French] (The 6-cogniton model for benzene solves the difficulties of Rumer's Theorem)

Stonier,T. *Information and the Internal Structure of the Universe*, Springer-Verlag, New York - Berlin, ISBN 3-540-19599-8 [German]: 0-387-19599-8 [English] (1990)

Wolfram,S. *Theory and Applications of Cellular Automata*, Advanced Series on Complex Systems, vol 1, World Scientific, ISBN 9971-50-123-6 pp 11,52,131,503 (1986)

Additional References (most are not quoted in the text)**(Complexity comparison, Related observations)**

Chaitin, G.J. *A Computer Gallery of Mathematical Physics*, IBM Research Report, Yorktown Heights, NY, USA p46 & 52 (March 23rd 1985)

Creutz, M. *Quarks, Gluons and Lattices*, Cambridge University Press, Cambridge (1983)

D'Arcy W. Thompson, *On Growth and Form*, Cambridge University Press, Cambridge, 2nd ed. pp 171, 325, 379, 395, 741 (1952)

Davies, P. *The Forces of Nature*, Cambridge University Press (1988); *Les Forces de la Nature* A. Colin, Paris, ISBN 2-200-24016-3 (1989)

Feynman, R. *The Character of Physical Law*, MIT Press, Cambridge, Mass. USA (1967)

Gaveau, B.; Jacobson, T.; Kac, M. & Schulman, L.S. *Relativistic extension between Quantum mechanics and Brownian motion*, Phys. Rev. Lett., 53 p419 (1984)

Iverson, K.E. *A Programming Language*, J. Wiley & Sons, New York, ISBN 0-471-43014-5 (1962)

Lévy-Leblond, J.M. & Balibar, F. *Quantique*, CNRS-InterEditions, Paris, ISBN 2-7296-0046-9 & 2-222-03345-4 (1984) [French]

Moriyasu, K., *An Elementary Primer for Gauge Theory*, World Scientific, Singapore (1983)

Resnikoff, H.L. *The Illusion of Reality*, Springer-Verlag, New York, ISBN 0-387-96398-7 (1989)

Schrödinger, E. *What is Life?*, Cambridge University Press (1944)

Stewart, G.W. & Ji-guang Sun, *Matrix Perturbation Theory*, Academic Press, Boston, USA, ISBN 0-12-670260-6 p230 (1990)

Watson, J.D.; Hopkins, N.H.; Roberts, J.W.; Steitz, J.A. Weiner, A.M., *Molecular Biology of the Gene*, 4th ed., The Benjamin/Cummings Publishing Co. Menlo Park, Cal. USA (1987) or *Biologie Moléculaire du Gène*, InterEditions, Paris, ISBN 2-7296-0235-6 (1989) [French]

Last-minute Ref:

Bak, P. & Chen, K. *Les Systèmes critiques auto-organisés*, Pour la Science, M2687, 161, p52-60 (March 1991); see also *Nature*, 342, 6251, 780 (1989) and *Phys. Lett.* 147, 5-6, 297 (1990)

The late Gérard A. Langlet

Commissariat à l'Energie Atomique, Direction des Sciences de la Matière,
Département de Recherches sur l'État Condensé, les Atomes et les Molécules,
Service de Chimie Moléculaire,

Laboratoire de Recherche en Intelligence Naturelle,

Centre d'Études de Saclay, F-91191-Gif sur Yvette, France

Boolean Hexagonal Automata

by Angela Coxe and Cliff Reiter

Cellular automata are local rules that are used to evolve discrete arrangements of cells. Perhaps the most famous cellular automaton is Conway's Game of Life [2]. It is based upon the iteration of simple rules on a rectangular lattice of Boolean valued cells. Incredibly rich structures have been discovered and there is a growing literature of amazing facts about that automaton [4].

One-dimensional cellular automata are also known to have rich structure, even when considering Boolean automata on size three neighbourhoods [7]. Wolfram's book, *A New Kind of Science* [7], suggests that simple automata are the principle behind many forces, whether they be artistic, physical, or social, in the world. Even without a grand view of automata, they are clearly related to important techniques, such as local filters for image processing.

One intriguing automaton that Wolfram describes is defined on a hexagonal lattice; its behavior has some features suggestive of a snowflake. A cell is either frozen (1) or not (0); it stays the same at the next generation unless it is currently unfrozen and exactly one of its neighbours is frozen, in which case it freezes. In this note we will investigate cellular automata on hexagonal lattices and their implementation in J. First we consider Wolfram's snowflake automaton.

Wolfram's Snowflake Automaton

A hexagonal lattice may be viewed as a collection of points arranged like the circles in Figure 1. A hexagonal array of cells may be implemented via a rectangular array where alternate rows are imagined to be offset by half a position. In a rectangular arrangement of cells, the nearest neighbour neighbourhoods consist of 3 by 3 blocks of cells. In Figure 1, the cell labelled 4 could be thought of as the centre of a 3 by 3 block of cells, which are labelled 0-9. However, in the hexagonal view, the cell labelled 4 has six neighbours, 0 1 5 7 6 3, listed clockwise and shown in bold. We consider this hexagon to be a left configuration in the 3 by 3 neighbourhood. On alternate rows, the hexagonal neighbourhoods will alternate between left and right configurations. The following 12 utilities implement the hexagonal automaton and allow us to view it. We explain how the utilities work below.

```
pad=: ([:-$@]++:@[) {. ($@)+[) {. ]
rep=: [ # # "1
```

```

hx_sh=(e.&0 1@(4:|i.@#)|."0 1])@:(2&rep)
lnbr=(4 0 1 5 7 6 3)&{@,
rnbr=(4 1 2 5 8 7 3)&{@,
cen={.
nnbr=:+/@:.
lhxauto=(1: = nnbr)`1: @.cen
perext2= ({:,|,|.)"1@:({::,|,|.
perext2e= ({:,|,|.)"1@:(|,2&|.
hhxa= 1 : '(2 1',:3 3)&(lhxauto@:u. ;._3)'
hxauto= [: ,/ lnbr hhxa@:perext2 ,:"1 rnbr hhxa @:perext2e

```

First, pad allows us to create initial configurations. Thus, the following gives a simple configuration with a single initial frozen speck.

```

2 pad 3=i.2 2
0 0 0 0 0 0
0 0 0 0 0 0
0 0 0 0 0 0
0 0 0 1 0 0
0 0 0 0 0 0
0 0 0 0 0 0

```

If we iterate the automaton twice, we see the following. However, the symmetry is hard to see.

```

hxauto^:(2) 2 pad 3=i.2 2
0 0 0 0 0 0
0 0 1 0 1 0
0 0 0 1 1 0
0 1 1 1 1 1
0 0 0 1 1 0
0 0 1 0 1 0

```

By doubling the number of pixels (imagine 2 by 2 blocks to be a cell) and rotating alternate pairs of rows by one, the hexagonal symmetry becomes reasonably apparent. However, it is still easier to see the symmetry in the figures.

```

hx_sh hxauto^:(2) 2 pad 3=i.2 2
0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 1 1 0 0 1 1 0 0
0 0 0 0 1 1 0 0 1 1 0 0
0 0 0 0 0 1 1 1 1 0 0 0
0 0 0 0 0 1 1 1 1 0 0 0
0 0 1 1 1 1 1 1 1 1 1 1
0 0 1 1 1 1 1 1 1 1 1 1
0 0 0 0 0 1 1 1 1 0 0 0
0 0 0 0 0 1 1 1 1 0 0 0
0 0 0 0 1 1 0 0 1 1 0 0
0 0 0 0 1 1 0 0 1 1 0 0

```

The centre and left neighbourhood is produced from a 3 by 3 cell via `lnbr`; the right-hand version is produced by `rnbr`. We consider the resulting list of 7 numbers to be the hexagonal neighbourhood. The centre and number of neighbours (with value 1) of a hexagonal neighbourhood are given by `cen` and `nnbr`. The heart of the automaton is simple to describe. Recall that frozen corresponds to 1. Thus, the value of a cell only changes value when it is currently 0 but has exactly one neighbour with value 1. On a hexagonal neighbourhood, `lhxauto=(1: = nnbr)`1:@.cen` gives 1 if the centre is 1; otherwise the result is the result of the test of whether there is exactly one neighbour, which completely describes the desired automaton. Now these need to be put together on all neighbourhoods (3 by 3 blocks of cells in the rectangular view). The key idea is all 3 by 3 neighbourhoods are selected, and the local automaton is applied. The adverb `hhxa` (half hexagonal automaton) accomplishes that on either the left or right neighbourhoods; the adverb argument is accordingly either `lnbr` or `rnbr`. Notice that `cut_3` with a left argument of `(1 2, :3 3)` is perfect for getting the 3 by 3 tessellations offset by 1 along one axis and 2 along the other. The final automaton, `hxauto`, applies the periodic extension appropriate to each axis, applies the local automaton and then the results are intertwined.

Figures 2 and 3 show iterations 107 and 127 on an initial configuration with a single initial point. While there is some structured growth at iteration 107, the growth fills in on iteration 127. This is true near other powers of two, hence the structures we see are typical for this automaton.

Make a Movie

We can create an animation of the evolution of this automaton using the `image3` addon [6]. The following function may be used to create 128 frames of the evolution; the subsequent lines illustrate how that may be done and then be assembled into a movie. The result may be viewed at [3].

```
load 'addons\image3\prevare.ijs'

mk_hxauto_fseq=:4 : 0
wb=.255,:0 0 0 NB. palette
b=.x. pad 0=:i.2 2
for_k. i. x. do.
  (wb {~ hx_sh b) write_image y.,(nfmt k),'.png'
  b=. hxauto b
end.
x.
)
```

```

128 mk_hxauto_fseq 'temp\s_f_auto'
fns=: 'sf_auto*.png' files_in 'temp\'
12 fseq_to_png_mov fns; 'temp\s_f_auto.mov'

```

The movie shows that while the automata exhibits beautiful growth; it also confirms our remark that only limited types of behaviour appear.

Snowflakes

Bentley and Humphreys [1] took thousands of photographs of snowflakes and ice crystals. Figure 4 shows four examples which give a glimpse of the diversity of snowflakes. See [5] for a wonderful discussion of snowflake growth. We see that real snowflakes have a far more diverse structure than seen in Wolfram's automaton. In the next section we briefly investigate other automata on a hexagonal lattice that maintain the symmetry of a hexagon.

Other Hexagonal Automata

The symmetries of the hexagon (which is the dihedral group denoted D_6) are the six-fold rotations along with reflections through opposite vertices. Our convention is to use a list of the centre and then 6 neighbours to represent a hexagonal neighbourhood. Thus, the rotations of the hexagon correspond to rotation of the last 6 elements and the mirror symmetry corresponds to reversal of those elements. Thus, we can apply all the symmetries to a list of length six via `d6`, which is defined below.

```

d6=: [: , / ((i.6) " _ |. "0 _ ]) "1 @: (, : |.)
d6 'abcef'
abcef
bcefa
cefab
efabc
fabce
abcef
fecba
ecbaf
cbafe
bafec
afecb
fecba

```

Then all the Boolean values possible on the 6 neighbours may be classified into 13 classes by using "key" where the first element from the ordered application of `d6` marks the class.

With further experimentation, we see that nontrivial Boolean automata of this type seem to have a hexagonal frame after 127 iterations. This is not surprising given the fact that in order to be physical, we should probably require that 0 neighbourhoods remain 0. Moreover, the initial configuration involves only one other configuration that could allow growth, namely, a 0 cell with exactly one neighbour. If growth is to occur, that configuration must become 1. That is exactly the Wolfram rule that allows change. Hence, that rule tends to create an envelope of the behaviours that occur.

The automata we have studied do not exhibit the same behaviours seen in ordinary snowflakes. However, these automata have very rich behaviours. We have also seen that they are readily implemented in J and easily presented with an animation.

References

- [1] W. A. Bentley and W. J. Humphreys, *Snow Crystals*, Dover Publications, New York, 1962.
- [2] E. Berlekamp, J. Conway, and R. Guy, *Winning Ways For Your Mathematical Plays*, Academic Press, New York, 1982.
- [3] A. Coxe and C. Reiter, Auxiliary Materials for Boolean Hexagonal Automata, http://www.lafayette.edu/~reiterc/mvp/hx_auto/index.html
- [4] A. Hensel, Conway's Game of Life, <http://hensel.lifepatterns.net/>
- [5] K. Libbrecht, Snow Crystals, <http://www.its.caltech.edu/~atomic/snowcrystals/physics/physics.htm>
- [6] C. Reiter and Z. Reiter, Image3 Addon, www.jsoftware.com, to appear.
- [7] S. Wolfram, *A New Kind of Science*, Wolfram Media, Champaign, 2002.

Angela M. Coxe
Lafayette College Box 8916
Easton, PA 18042 USA
coxea@lafayette.edu

Cliff Reiter
Department of Mathematics
Lafayette College
Easton, PA 18042 USA
reiterc@lafayette.edu

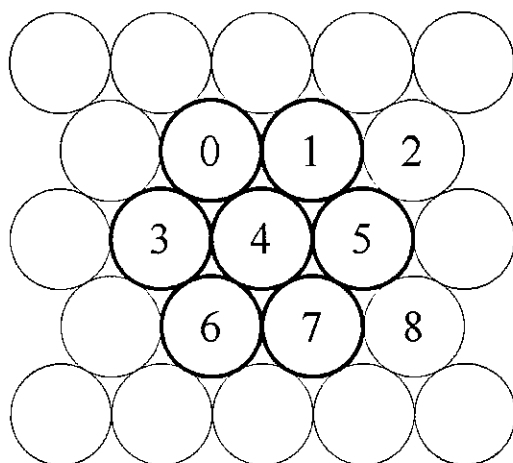


Figure 1. A hexagonal arrangement of cells with one rectangular neighbourhood, with cells numbered 0-8, and the corresponding hexagonal neighbourhood, with cells in bold.

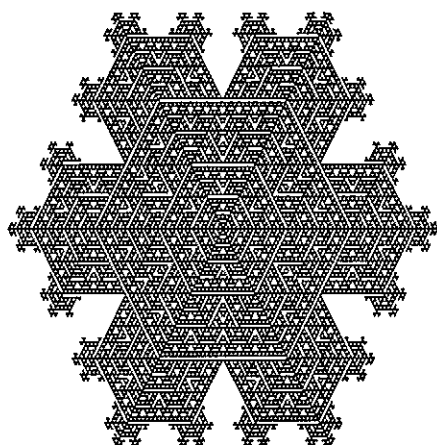


Figure 2. Iteration 107 of Wolfram's snowflake automaton.

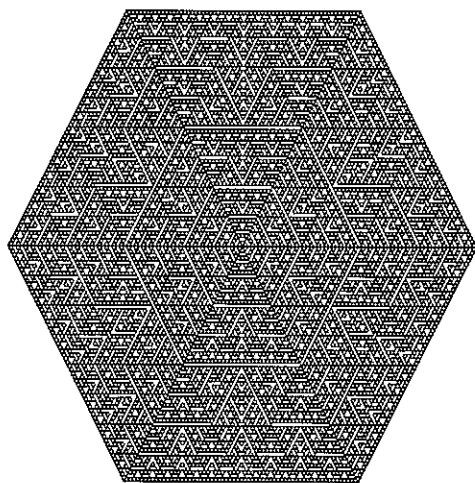


Figure 3. Iteration 127 of Wolfram's snowflake automaton.

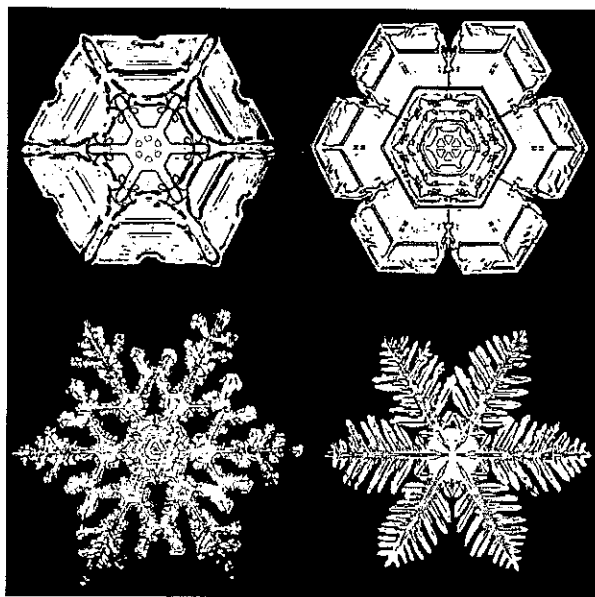


Figure 4. Four of Bentley and Humphreys' snowflakes.

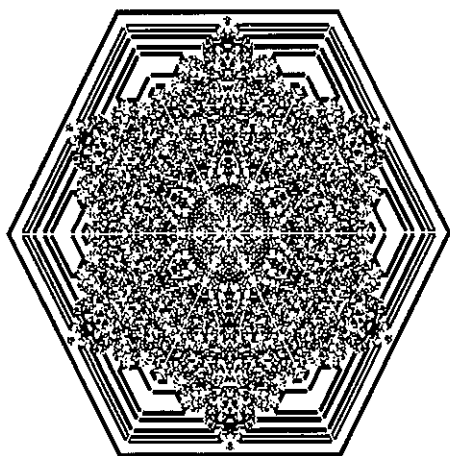


Figure 5. Automaton with hexagonal symmetry (rule 25629998).

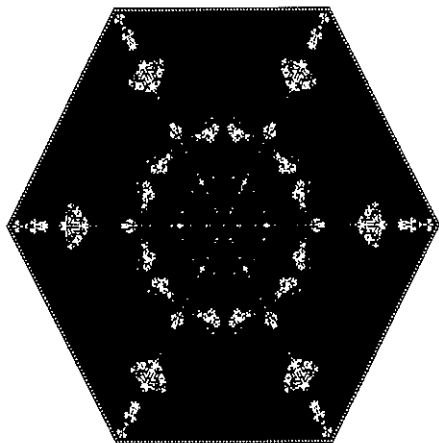


Figure 6. Automaton with hexagonal symmetry (rule 27541687).

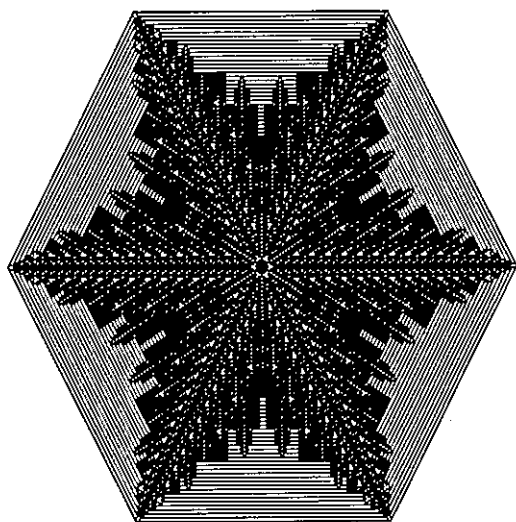


Figure 7. Automaton with hexagonal symmetry (rule 32824527).

Provisional Unicode Reference

proposed by Adrian Smith

Background

Before you can type APL code into Notepad, you really need a decent (probably fixed pitch) font which includes the APL symbols. Accordingly, I have remapped (and augmented) my APLSans font and created something called APL385 Unicode. This is supposed to have everything in every $\square$ AV of every APL ever seen on the planet, including NARS and anything else anyone might ever write about in Vector. It will soon be available on the Vector website and will be freely re-distributable.

For reference, here is everything that the font currently has, everything that is in the Unicode APL section that it does **not** have (you will see why) with appropriate comments where there is scope for disagreement.

If I missed anything, let me know and I may add it in.

Old Font	Unicode Hex	New Font	Description and Notes on Usage
!	0021	!	Factorial
*	002A	*	Power (All APLs)
+	002B	+	Plus
-	002D	-	Minus
/	002F	/	Compress
<	003C	<	Less than
=	003D	=	Equal
>	003E	>	More than
?	003F	?	Random
\	005C	\	Expand
^	005E	^	Logical AND
	007C		Remainder
~	007E	~	Without (APL2 and latest Dyalog)
..	00A8	..	Each
-	00AF	-	Negative
x	00D7	x	Multiply
÷	00F7	÷	Divide

The "Arrows" subset

←	2190	←	Assign
↑	2191	↑	Take
→	2192	→	Goto
↓	2193	↓	Drop

The "Mathematical Operators" subset

Δ	2206	Δ	Delta
∇	2207	∇	Del (Dfns = self)
∈	220A	∈	Membership
∘	2218	∘	Jot (outer product/compose)
∨	2228	∨	Logical OR
∩	2229	∩	Intersect (please implement!!)
∪	222A	∪	Nub/Union (ditto)
⋅	2235	⋅	Paw = Dieresis-dot (APL2)
~	223C	~	Tilde (alternative to ASCII)
≠	2260	≠	Not equal to
≡	2261	≡	Match
≢	2262	≢	Notmatch (Dyalog only)
≤	2264	≤	Lessthan or Equal
≥	2265	≥	Greater or Equal
⊂	2282	⊂	Enclose
⊃	2283	⊃	Disclose/Pick
⊖	2296	⊖	Rotate on first axis
⌊	22A2	⌊	Right tack (unused except Sharp)
⌋	22A3	⌋	Left tack (unused except Sharp)
⌈	22A4	⌈	Encode
⌋	22A5	⌋	Decode
◊	22C4	◊	Diamond (APL2 and latest Dyalog)

The "Miscellaneous Technical" subset

⌈	2308	⌈	Max
⌋	230A	⌋	Min
⌊	2336	⌊	I-Beam (APLSV, now unused)
⌋	2337	⌋	Squad (APL2, APL+Win, Sharp)
⌋	2339	⌋	Domino
⌋	233B	⌋	Quad-Jot (APL2 - never used)
⌋	233D	⌋	Reverse

/	233F	/	Slashbar
\	2340	\	Slopebar
□\	2342	□\	Sandwich (APL2, APL*PLUS/PC 17)
□←	2347	□←	Quad-Left (APL.68000)
□→	2348	□→	Quad-Right (APL.68000)
⌘	2349	⌘	Transpose
⌞	234B	⌞	GradeUp
⌟	234E	⌟	Execute
□↑	2350	□↑	Quad-Up (APL.68000)
▽	2352	▽	GradeDown
⌘	2355	⌘	Format
□↓	2357	□↓	Quad-Down (APL.68000)
△	2359	△	DeltaUnderbar
⌘	235D	⌘	Lamp
□	235E	□	QuoteQuad
⊙	235F	⊙	Log
⌘	2361	⌘	Snout (PLUS/PC 170 unused)
▽	2362	▽	Frog (PLUS/PC 171, NARS Dual)
*	2363	*	Sourpuss (NARS Power)
∘	2364	∘	Hoot = Rank (Sharp APL only)
∘	2365	∘	Holler (APL*PLUS/PC 228 unused)
⌘	2368	⌘	Frown = Commute (NARS, Dyalog)
⌘	236A	⌘	CommaBar (catenate first)
⌘	236B	⌘	DelTilde (Lock function)
⊖	236C	⊖	Zilde (constant (ι0))
⌘	2371	⌘	APL Nor (watch for 22BD also)
⌘	2372	⌘	APL Nand (watch for 22BC also)
ι	2373	ι	Iota (NOT the Greek char)
ρ	2374	ρ	Rho (ditto)
ω	2375	ω	Omega (Dfns and NARS DDef)
⌘	2377	⌘	Find
ι_	2378	ι_	Iota-underbar (APL2 only)
α	237A	α	Alpha (Dfns and NARS DDef)
□	2395	□	Quad (in recent Unicode spec)

The "Geometric Shapes" subset

□	25AF	□	Quad (in Arial Unicode MS)
◇	25CA	◇	Diamond (Dyalog 10 - now moved)
○	25CB	○	Circle (Pi times ...)

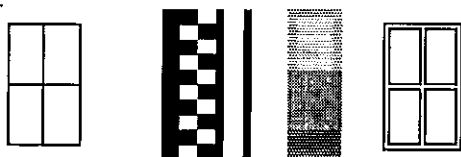
Oddballs in the "Miscellaneous Technical" section

None of these are in "APL385 Unicode" as I have never seen them used.

⌘=	2338	Quad-equal
⌘◊	233A	Quad-diamond
⌘○	233C	Quad-circle
○○	233E	Circle-jot
⌘/	2341	Quad-slash
⌘<	2343	Quad-less than
⌘>	2344	Quad-greater than
+	2345	Left vane
+	2346	Right vane
⌘_	234A	Decode-underbar
⌘√	234C	Quad-OR
⌘Δ	234D	Quad-Delta
↑_	234F	Upwards vane
τ_	2351	Encode-overbar
⌘^	2353	Quad-AND
⌘∇	2354	Quad-Del
↓_	2356	Downwards vane
'_	2358	Quote-underbar
◊_	235A	Diamond-underbar
○_	235B	Jot-underbar
○_	235C	Circle-underbar
⌘:	2360	Quad-colon
u	2366	DownShoe-style
c	2367	LeftShoe-style
>¨	2369	GreaterThan-dieresis
~	236D	Stile-Tilde
;-	236E	Semicolon-underbar
⌘≠	236F	Quad-notEqual
⌘?	2370	Quad-Question
α_	2376	Alpha-underbar
ω_	2379	Omega-underbar

Box-drawing characters (shown at 16/16)

Only the single/double boxes and blocks are included.



These must be set with no leading to join up vertically. The shaded blocks are best avoided these days, but APL2 has them.

Other Odds and Ends

These are in the APL2 product, so just for completeness, here they are.

Del	007F	⌠	House (for APL2 use only)
Peseta	20A7	℞	Peseta (also in APL2)

Cyrillic (basic alphabet only)

А Б В Г Д Е Ж З И Й К Л М Н О П Р С Т У Ф Х Ц Ч Ш Щ Ъ Ы Ь Э Ю Я

а б в г д е ж з и й к л м н о п р с т у ф х ц ч ш щ ъ ы ь э ю я

Comments welcome on the quality of the glyphs. All I have to go on is a splendid Чайковский record that Sasha gave me in Toronto.

Greek (in the style of Frutiger Linotype)

Α Β Γ Δ Ε Ζ Η Θ Ι Κ Λ Μ Ν Ξ Ο Π Ρ Σ Τ Υ Φ Χ Ψ Ω

α β γ δ ε ζ η θ ι κ λ μ ν ξ ο π ρ σ τ υ φ χ ψ ω

I believe that with the released version of PocketAPL, Dyadic and APL2 are now fully in-line with the mappings. Also IBM will include a 'clip as Unicode' option in the next update, which should allow reliable copy-paste between two of the major vendors. Maybe APL2000 and MicroAPL would like to follow this lead!

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

Contents

Technical Correspondence		128
Rain-graphs to PowerPoint-land via OLE	Jouko Kangasniemi	130
At Play with J: Beware Scholes!	Gene McDonnell	137

TECHNICAL CORRESPONDENCE

From: John R Clark

2nd December 2002

Been playing around with Dynamic Functions in Dyalog APL and the Nim arithmetic functions listed in VECTOR 19:1 pages 108 – 118. After reading the article by Eugene McDonald and the quoted letter of Professor Richard Guy, I realized the the methods for handling cases where 8 is a multipican is far far too complicated. The nice thing about APL is you can make mathematics very simple, provided of course you think the APL way. For Nim you start by representing the values in terms of powers of 2.

For addition you actually represent the values in base 2 and then do a 2 base of the $\neq$ of the values. Tis trivial. For multiplication you distribute the values of one with the other then multiply the pairs taking into account the exceptions from regular multiplication for 0, 1, 8, and the squares of 2 and 4. The functions for building these tables are:

```
VCODE8←{
    A 8 factors into 4 and 2 just regroup
    A (8 8) (8 4) or (8 2) are only possible
[1]      =/ω:MULT"DIST(MULT 4 4),(MULT 2 2)  A (8 8) case
[2]
[3]      4∈ω:MULT"DIST(MULT 4 4),2          A (8 4) case
[4]      2∈ω:MULT"DIST(MULT 2 2),4          A (8 2) case
[5]      }

```

▽

```
VDIST←{
[1]      a b←ω          A Split multiplicans
[2]      a←((2 2 2 2τa)/8 4 2 1),(a=0)/0  A Convert a
[3]      b←((2 2 2 2τb)/8 4 2 1),(b=0)/0  A Convert b
[4]      ,a◦.,b          A distribute pairs
[5]      }

```

▽

```
VMULT←{
[1]      0∈ω:0          A zero multipican case
[2]      1∈ω:×/ω        A identity case
[3]      8∈ω:CODE8 ω    A handle 8 as multipican
[4]      =/ω:1.5×1>ω    A square of 2 or 4 case
[5]      ×/ω            A regular multiply case
[6]      }

```

▽

```

      NAD+{      R Nim Addition
[1]      R 0,± handles single values and nested values
[2]      2±÷/(2 2 2 2 2 2)τ0,±ω
[3]      }
      ▽

```

NMX+{NAD MULT"DIST μ}

Addition table:

NAD "(0,15)±.,0,15

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	0	3	2	5	4	7	6	9	8	11	10	13	12	15	14
2	3	0	1	6	7	4	5	10	11	8	9	14	15	12	13
3	2	1	0	7	6	5	4	11	10	9	8	15	14	13	12
4	5	6	7	0	1	2	3	12	13	14	15	8	9	10	11
5	4	7	6	1	0	3	2	13	12	15	14	9	8	11	10
6	7	4	5	2	3	0	1	14	15	12	13	10	11	8	9
7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8
8	9	10	11	12	13	14	15	0	1	2	3	4	5	6	7
9	8	11	10	13	12	15	14	1	0	3	2	5	4	7	6
10	11	8	9	14	15	12	13	2	3	0	1	6	7	4	5
11	10	9	8	15	14	13	12	3	2	1	0	7	6	5	4
12	13	14	15	8	9	10	11	4	5	6	7	0	1	2	3
13	12	15	14	9	8	11	10	5	4	7	6	1	0	3	2
14	15	12	13	10	11	8	9	6	7	4	5	2	3	0	1
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

Multiplication table:

NMX"(0,15)±.,0,15

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	2	3	1	8	10	11	9	12	14	15	13	4	6	7	5
0	3	1	2	12	15	13	14	4	7	5	6	8	11	9	10
0	4	8	12	6	2	14	10	11	15	3	7	13	9	5	1
0	5	10	15	2	7	8	13	3	6	9	12	1	4	11	14
0	6	11	13	14	8	5	3	7	1	12	10	9	15	2	4
0	7	9	14	10	13	3	4	15	8	6	1	5	2	12	11
0	8	12	4	11	3	7	15	13	5	1	9	6	14	10	2
0	9	14	7	15	6	1	8	5	12	11	2	10	3	4	13
0	10	15	5	3	9	12	6	1	11	14	4	2	8	13	7
0	11	13	6	7	12	10	1	9	2	4	15	14	5	3	8
0	12	4	8	13	1	9	5	6	10	2	14	11	7	15	3
0	13	6	11	9	4	15	2	14	3	8	5	7	10	1	12
0	14	7	9	5	11	2	12	10	4	13	3	15	1	8	6
0	15	5	10	1	14	4	11	2	13	7	8	3	12	6	9

Multiplication is interesting, not only is it communicative, associative, but +/ and +/ yields the same set of values. The 1 15 has all the values as each row except the first.

Rain-graphs to PowerPoint-land via OLE

by Jouko Kangasniemi (jouko.kangasniemi@tt.fi)

This is a brief story of APL-PowerPoint co-operation. PowerPoint has nearly a monopoly in the presentation application-market. One has to use it more or less by force. The same phenomena can be recognised in the word processor market and in the spreadsheet market.

Presentations with business graphics are often shown by PowerPoint. If you already have excellent tools like APL & RainPro and you mainly deal with PDFs, the graph side of your life is already covered quite well. PP is not interesting to someone who can do Rain. But still you occasionally have to deliver stat information in PP format.

So what should I do? Should all the beautiful linecharts with nice thick lines to be done again. PP graphics is actually the same MsGraph as in Excel and you can not do anything special with it. It is even a bit trickier to use than Excel. At least for me. Is there a way to get the beautiful Rain-graphs directly to Mickeysoft-world with a minimal amount of tears? With no need to invent the same things again and being forced to maintain two versions of the same thing?

If you can't invent, copy! So why don't I make a metafile of the image and copy it to clipboard and then paste it to PowerPoint? Not a bad idea. One can move one chart manually, but if the copy-paste is frequent, some sort of automation is in demand. OLE-automation is the answer here. The technique is very similar to Excel's and Word's OLE-interfaces, but the terms are naturally a bit different. So this rehearsal is about how to make PP dance, or at least walk slowly, through OLE from Dyalog APL.

Beginning is easy in this case. Let's make a chart. Load Rain and write a bit of code. ("Otsikko" is head in Finnish for those who wonder...)

```
ch.Set('Head' 'Otsikko')
ch.Bar 10
PG←ch.Close
```

Variable PG can be copied to clipboard with the function Clip that comes with Rain.

```
#.PostScrp.Clip PG
```

So we are halfway home, or what? We have done the graph and it's sitting waiting at the clipboard. Unfortunately there is still lot to do and we must dive into the inner parts of PowerPoint.

First we have to start the damn thing. Write

```
p←'PP'□WC'Oleclient' 'Powerpoint.Application'
```

and it has to be made visible

```
p □WS'visible' 1
```

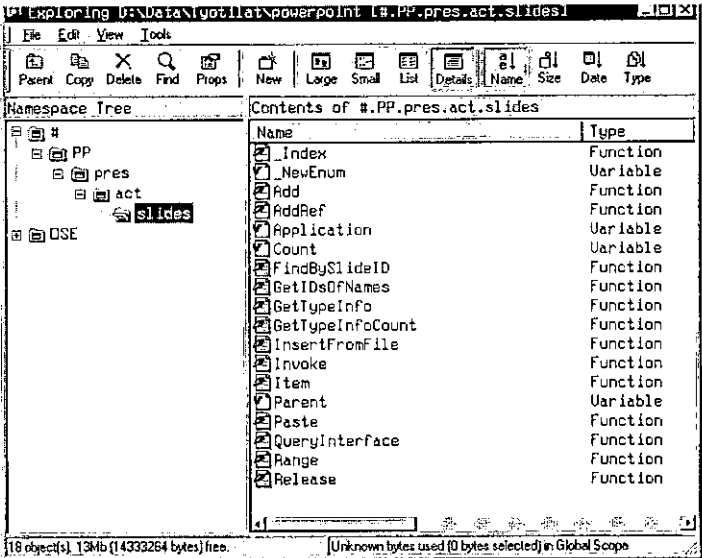
As a technical detail I should mention that I use Dyalog APL version 8.2. That is why I still use objects the old way. Version 9 introduced the dot-syntax, but somehow this traditional way is still more familiar.

Adventures in the OLE-land are often based on wild guess, trial and error and lots of lost time, at least in my case. But of course you get aid by asking properties and methods of the functions

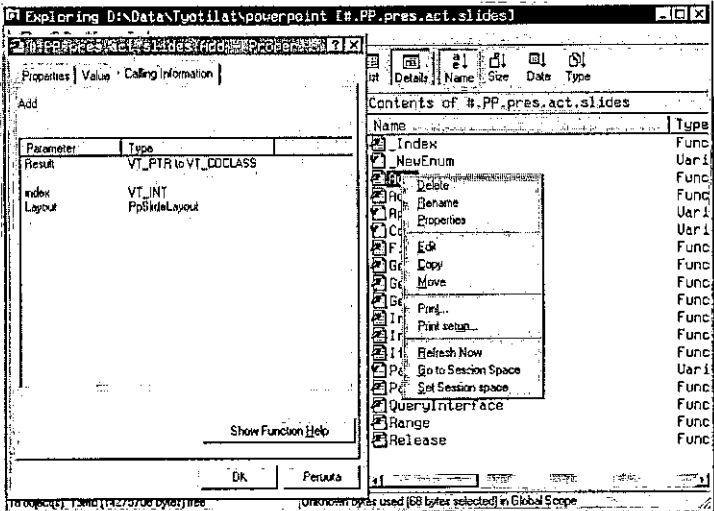
```
'PP' □WG'PropList'
Type Active Hwnd Visible WindowState Height Width Top Left VBE A
ddIns Creator ActivePrinter OperatingSystem Version Build FileF
ind FileSearch Assistant Caption Name Path CommandBars SlideSh
owWindows ActivePresentation ActiveWindow Dialogs Windows Presen
tations ClassName Event Data Handle ClassID KeepOnClose Method
List VerblList Typelist HelpFile LastError Locale AutoBrowse In
stanceMode ChildList EventList PropList
```

From this piece of text we can guess that the word "Presentations" might be useful. A good way to speed up the beating of the OLE-objects is also to search for OLE tips from the internet's dark Visual Basic section and to try them the APL-way. It saves time.

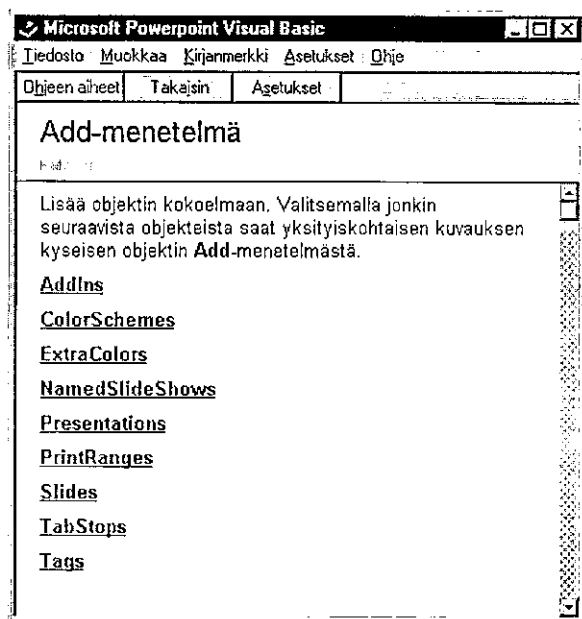
The wonders of objects can also be investigated with the excellent Dyalog resource tool, that gives us the objects and methods of those OLE plums. For example, from inside the "slides" object we find the following:



If you press the right button of the mouse you get a menu. "Properties" gives you useful information related to objects and methods and how to use them. Especially the "Calling Information"-tab is often rewarding.



And we are lucky, Show Function Help-button gives us a good Microsoft help topic. Usually the help does not help at all, only makes one angry, but after a while something useful is bound to be found (Again some Finnish below).



Back to the subject. Now that we have the program running, we need a presentation. It can be done with the presentations-object. Let's say

```
pres+'PP.pres' WC PP.Presentations
```

and then the "Add" method

```
3 NQ pres'Add', or PP.pres.Add 1
```

which gives us an empty presentation with no slides at all. We must get those next. The journey that continues towards a slide has to be done with active presentation. Let's define

```
act+(pres, 'act') WC PP.ActivePresentation
```

A "Slides"-object can be created as a child of the presentation:

```
slid+(act, 'slides') WC (act, 'Slides')
```

So here we are, ready to add the slide with the "Add"-command

```
3 □NQ slid'add' 1 'pplayoutBlank'
```

"One" is the position of the new slide and pplayoutBlank is the type of the new slide. Just an empty slide. There are around dozen different slide types. A single presentation can contain several slides. The number of these can be queried:

```
(pres, '.act.slides')□WC'Count'
1
```

If we want the new slide to be the last one, we can call it Count+1 of course.

We must paste the metafile from the clipboard through the current active window. Let's ask who is active at the moment

```
actv←(pres, '.actv')□WC PP.ActiveWindow
```

View-object is the thing that is next in demand in order to do get the chart from the clipboard

```
view←(pres, '.actv.vi')□WC(↑actv, '.View')
```

and finally we say "Paste" and bang we have a PP chart

```
3 □NQ view'Paste'
```

Place and size of the image can be altered with OLE and animations etc. can be added to make the owner of the presentation happy and to make the audience suffer.

An image can also be added from a file. There we need object "Shapes", and it has a method called "AddPicture" that does the trick. The clipboard way is faster and easier.

The show can be started automatically. To do that, we need an object called "Slideshowsettings".

```
sls←'PP.pres.act.sls'□WC PP.pres.act.SlideShowSettings
```

And the show can go on...

```
3 □NQ sls'Run'
```

It is best to save the presentation with SaveAs-method. With it you do not have to care if there already is a file with the same name.

```
3 □NQ act 'SaveAs' 'd:\data\esitykset\koe'
```

The presentation is closed with the Close-method.

```
3 [NQ act 'Close'
```

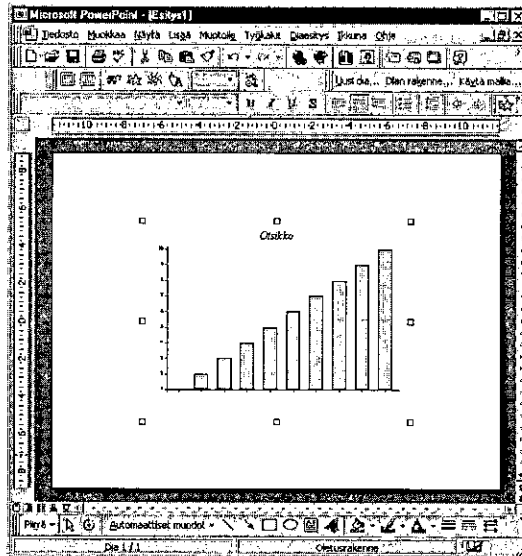
and it can be reopened

```
3 [NQ pres 'Open' 'd:\data\esitykset\koe.ppt'
```

To make PowerPoint go away the Quit command is not enough. All the presentations must be closed, then Quit and then we must delete the 'PP'-namespace.

```
3 [NQ p 'Quit' ♦ [ex p
```

As a conclusion it can be mentioned that this simple operation took a considerable amount of time doing it the first time. It pays to save frequently because Dyalog tends to crash occasionally with OLE trial-and-error development. All the useful objects and methods that are found must be documented right away. And I should have studied the PP object model at the beginning, not after several hours of trial and error. On the other hand now it is a bit clearer to me. There are lots of possibilities in what can be done with PP and OLE. If someone has already gone further please let me know. And here is proof that it works:



What About Dyalog 9?

by the Editor (stf@apl.it)

I admit it: after all the time since Dyalog 9 was released, I fail to see what is so exciting about using all those quotes when dealing with OLE Automation. The developers at Dyadic have done an excellent job to simplify the interaction between APL and an application like Microsoft PowerPoint. Therefore here's my quick rephrasing of Jouko's snippets, in modern syntax.

```
ch.Set 'Head' 'Testa' A "Testa" is of course Italian for "Head"
ch.Bar t10
PG←ch.Close
#.PostScrp.Clip PG      A Nothing new this far

'PP' ←wc 'OleClient' 'Powerpoint.Application'

PP.Visible←1                A Let there be light!
pres←PP.Presentations.Add 1  A a new Presentation
pres.Slides.Add 1 PP.ppLayoutBlank A a new Slide
PP.ActiveWindow.ViewType←PP.ppViewNormal A to prepare for Paste
PP.ActiveWindow.View.Paste  A Paste the Rain-generated picture
pres.SlideShowSettings.Run  A Run the slideshow
pres.SaveAs 'd:\data\esitykset\koe' A Save the new presentation
pres.Close                  A Close it

)erase pres

PP.Presentations.Open'd:\data\esitykset\koe' A Reload it
PP.Presentations.(Item 1).Close             A Close it again

PP.Quit                A That's all folks!

)erase PP
```

In other words, thank you John!

At Play With J: Beware Scholes!

by Eugene McDonnell

Beware Scholes!

This article is about a J version of the Black-Scholes formulas, the brainchild of Myron Scholes and the late Fischer Black. The document:

<http://bradley.bradley.edu/~arr/bsm/pg04.html>

gives a lot of information on the formula and its creators (which won the surviving creator the Nobel Prize in economics in 1997), and if you want to find out more about Black and Scholes or the theory behind their formula, I recommend it.

A *call* is an option to *buy* a stipulated amount of stock at a specified time and price, and a *put* is an option to *sell* ditto. A person might acquire a call option who expects the price of the asset to rise. The Black-Scholes formulas enable the seller of the option to determine quite accurately what price to charge for such options.

Here are the formulas in conventional mathematical notation:

$$C = SN(d_1) - Xe^{(-rT)}N(d_2)$$

$$P = Xe^{(-rT)}N(-d_2) - SN(-d_1)$$

$$d_1 = \frac{\ln(S/K) + (r + v^2/2)T}{v\sqrt{T}}$$

$$d_2 = d_1 - (v\sqrt{T})$$

C = Theoretical Call Premium

S = Current Stock Price

P = Theoretical Put Premium

X = Option Strike Price

r = Risk-Free Interest Rate

v = volatility, or Standard Deviation of Asset Price

T = Time in years until strike date

N = Cumulative Standard Normal Distribution

ln = Natural Logarithm

Many different programming languages have been used to write programs for these formulas. The document:

<http://home.online.no/~espehaug/SayBlackScholes.html>

contains a couple of dozen of these programs, written in these languages:

C#	O'Caml
C++	Pascal
Fortran	Perl
Haskell	PHP
HP48	Python
Icon	Real Basic
IDL	Rebol
JAVA	Scheme
JavaScript	S-Plus
K	Squeak
Maple	Transact SQL
Mathematica	VBA
Matlab	

The programs are in one of two forms, both adhering closely to the original mathematical formulas shown above. Some have separate programs for calls and puts; some exploit the family resemblance of calls and puts and so write just one general program that requires an additional parameter to indicate whether a solution for a call or a put is desired. Here is a typical general program, this one written in C++:

```
Double BlackScholes(char CallPutFlag, double S, X, T, r, v)
{
double d1, d2

d1=(log(S/X)+(r+v*v/2)*T)/(v*sqrt(T));
d2=d1-v*sqrt(T);

if(CallPutFlag) == 'c'
return S * CND(d1)-X * exp(-r*T)*CND(d2)
elseif(CallPutFlag == 'p')
return X * exp(-r * T) * CND(-d2) - S * CND(-d1);
}
```

This program includes as its first argument the letter 'c' for a call option, and 'p' for a put option, and then discriminates between the two by an if/elseif control structure. Otherwise, it follows the Black-Scholes formulas closely. The entry includes a long separate program for the required cumulative normal distribution function, as do many of the other entries.

At present the document has no contribution written in J (or APL, for that matter). The rest of this paper describes the evolution of J programs for the Black-Scholes formulas. Five different people made transformations of the formulas that ended in a J version radically different from all the others.

My attention was first called to this subject by a message from Hu Zhe to the J Forum that uses separate functions for call and put.

```
load 'c:\j406\system\packages\stats\statdist.ijs'
cnd =: 3 : 'normalprob 0, 1, __, y.'

d1 =: 3 : 0
'S X T r v' =. y.
((^S%X)+(r+-: *:v)*T)%(v*%:T)
)

d2 =: 3 : 0
'S X T r v' =. y.
((^S%X)+(r--: *:v)*T)%(v*%:T)
)

BlackScholesCall =: 3 : 0
'S X T r v' =. y.
(S*cnd d1 y.) - (X*(^r*T)*cnd d2 y.)
)

BlackScholesPut =: 3 : 0
'S X T r v' =. y.
(X*(^r*T)*cnd -d2 y.) - (S*cnd -d1 y.)
)
```

These are reasonably concise and straightforward. They show what was to be expected: that J, as well as any other programming language, can translate the mathematical notation directly into computer programs. Notice that he loads a J library function for the cumulative normal distribution.

Shortly after this appeared, Oleg Kobchenko sent the following version, a single function for both calls and puts, that incorporates d1 and d2:

```
BlackScholes=: 4 : 0
'S X T r v' =. y.
d1=. ((ln S%X)+(r+-: *:v)*T)%(v * sqrt T)
d2=. d1 - v * sqrt T
(S, X * exp-r*T) (-/ . * cnd)&(-^:x.) (d1, d2)
)
```

The lines forming d1 and d2 are like those in the C++ program. The last line is an instance of array thinking. They exploit the similarity of the call and put functions. The put option definition can be rewritten.

Here are the call and put options, with put in its new form.

```
c =: (s*cnd( d1))-((x*exp-r*t)*cnd( d2))
p =: -(s*cnd(-d1))-((x*exp-r*t)*cnd(-d2))
```

This shows that p differs from c solely in the use of negation of d1 and d2, and in negating the overall result. Kobchenko exploits this by rearranging things so that a left argument of 0 or 1 discriminates call and put, respectively,

More abstractly, the last line of his function can be written as:

```
a ((b c)&d) e
(d a)(b c)(d e)
(d a) b (c(d e))
(d      a      ) b      (c (d      e      ))
(-^:x.)(S,X*^-r*t)(-/ . *) (cnd(-^:x.)(d1,d2))
```

This shows the conditional negation of the left and right hand sides, the application of cnd to the right hand side, and the difference of the product, so that for a call we would have:

```
c =: ( s,x*exp-r*t) -/ . * cnd( d1,d2)
```

and for a put we would have:

```
c =: (-s,x*exp-r*t) -/ . * cnd(-d1,d2)
```

At the same time that Kobchenko was working on his array approach, I had been working on the other main part of the program, the formation of d1 and d2. I wrote down the definition of d2:

```
d2 =: d1      v*%:t
```

Then I replaced d1 by its definition, and with a bit of algebra arrived at:

```
d2 =: ((^:s%x)+(r--:::v)*t)%(v * %:t)
```

and if you compare this with the definition for d1, you will find that the only difference is that (r+-:::v) is changed to (r--:::v). This being the case, it was simple to replace the two lines defining d1 and d2 by a single line that forms a two-item list d that uses the fork (+,-):

```
d =: ((^:s%x)+(r(+,-)--:::v)*t)%v*%:t
```

This permitted the definition of BlackScholes to become:

```
BlackScholes =: dyad define
's x t r v' =. y.
d =: ((^ . s % x) + (r (+, -) -: *: v) * t) % v %: t
(s, x ** ^ - r * t) (- / . * cnd) & (- ^: x.) d
)
```

The only thing about this that I found not to my liking was the need to specify a left argument to indicate call or put. Happily for me, just about this time Arthur Whitney posted a message to the K forum that showed that *v* can be used to discriminate the two cases, by using it positively for call, and negatively for put. Thus it became possible to do without the left argument, and write:

```
BS =: monad define
'S X T r v' =. y.
d =. ((^ . S % X) + T * r (+, -) -: *: v) % v %: T
- / (S, X ** ^ - r * T) * cnd d
)
```

Notice that I have separated the parts of $(- / . *)$, giving, I believe, a program easier to explain and understand.

Here are examples of call and put. The result for put is negative, and this differs from the usual put result, which is positive. The negative result can be useful to distinguish a call result from a put result. If a positive put result is necessary, a magnitude sign (|) can be placed in front of the last line of BS.

```
yc =: 60 65 0.25 0.08 0.3
BS yc
2.13337
yp =: 60 65 0.25 0.08 _0.3
BS yp
_5.84628
```

We haven't ended quite yet. Perhaps you remember the article by Ewart Shaw in *Vector* 18.4, in which he defined the error function erf using J's hypergeometric conjunction:

```
erf =: (* & (%: 4 p_1) % ^ @: *) * [: 1 H. 1.5 *: NB. A&S 7.1.21 (right)
```

and then defined the cumulative distribution function of the normal distribution by:

```
cnd =: [: -: 1: + [: erf % & (%: 2) NB. A&S 26.2.29 (solved for P)
```

All of the functions written in other languages must do something special to define `cnd`, either using a library function, or writing the definition using approximation A&S 26.2.16.

I'm going to contribute `BS`, `erf`, and `cnd` to the Black-Scholes web site, but in the following training-wheels versions so that the innocent reader may come close to understanding them without having to learn any J.

```
BS =: monad define
'S X T r v' =. y.
d=.((ln S dv X) + T * r (+,-) hlf sqr v) dv (v * sqrt T)
diff (S , X * exp - r * T) * cnd d
)

erf =: monad define          NB. A&S 7.1.21 (rightmost)
((2 * y.) dv (sqrt pi)) * (exp - y. ^ 2) * (1 H. 1.5) y. ^2
)

cnd =: monad define          NB. A&S 26.2.29 (solved for P)
(1 + erf y. * sqrt 0.5) dv 2
)
```

Where:

```
diff =: -/
dv   =: %
exp  =: ^
hlf  =: -:
ln   =: ^.
pi   =: 1p1
sqr  =: *:
sqrt =: %:
```

Index to Advertisers

Dyadic Systems Ltd	6
MicroAPL Ltd	2
Renaissance Data Systems	62
Vector Back Numbers	28

All queries regarding advertising in VECTOR should be made to Gill Smith, at 01439-788385, Email: apl385@compuserve.com.

Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue n+1 and discuss themes for issues n+2 onwards. Please send the text of submitted articles (hardcopy with diskette as appropriate) to the Vector Working Group via:

Vector Administration, c/o Gill Smith
Brook House
Gilling East
YORK, YO62 4JJ
Tel: +44 (0) 1439-788385
Email: apl385@compuserve.com

Authors wishing to use Word for Windows should contact Vector Production for a copy of the APL2741 TrueType font, and a suitable Winword template. These may also be downloaded from the Vector web site at www.vector.org.uk

Camera-ready artwork (e.g. advertisements) and diskettes of 'standard' material (e.g. sustaining members' news) should be sent to Vector Production, Brook House, Gilling East, YORK YO62 4JJ. Please also copy us with all electronically submitted material so that we have early warning of possible problems.

Subscribing to Vector

Your Vector subscription includes membership of the British APL Association, which is open to anyone interested in APL or related languages. The membership year runs from 1st May to 30th April. The British APL Association is a special interest group of the British Computer Society, Reg. Charity No. 292,786

Name: _____

Address: _____

Postcode / Country: _____

Telephone Number: _____

Email Address: _____

- | | | |
|---|------|--------------------------|
| UK private membership | £20 | ☐ |
| Overseas private membership | £22 | ☐ |
| Airmail supplement (not needed for Europe) | £4 | ☐ |
| UK Corporate membership | £100 | ☐ |
| Corporate membership overseas | £135 | ☐ |
| Sustaining membership | £500 | ☐ |
| Non-voting UK member (student/OAP/unemployed) | £10 | ☐ |

PAYMENT – in Sterling or by Visa/Mastercard/JCB only

Payment should be enclosed with membership applications in the form of a UK Sterling cheque to "The British APL Association", or you may quote your Mastercard, Visa or JCB number.

I authorise you to debit my Visa/Mastercard/JCB account

Number: Expiry date: |

for the membership category indicated above,

- ☐ annually, at the prevailing rate, until further notice
☐ one year's subscription only

Signature: _____

Data Protection Act:
The information supplied may be stored on computer and processed in accordance with the registration of the British Computer Society.

Send the completed form to:

BAA, c/o Rowena Small, 12 Cambridge Road, Waterbeach, CAMBRIDGE CB5 9NJ, UK

Fax: +44 (0) 1653 697719

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

2002/2003 Committee

Chairman	Adrian Smith 01439-788385 apl385@compuserve.com	Brook House Gilling East YORK YO62 4JJ
Secretary	Anthony Camacho 0117-973-0036 acam@blueyonder.co.uk	11 Auburn Road Redland BRISTOL, BS6 6LS
Treasurer	Nicholas Small 01223-570850 treas.apl@bcs.org.uk	12 Cambridge Road, Waterbeach, Cambridge CB5 9NJ
Journal Editor	Stefano Lanzavecchia stf@apl.it	c/o APL Italiana Corso Vercelli 54 20145 - Milano Italy
Projects and Publicity	Dr Alan Mayer 01792-205678x4274 a.d.mayer@swansea.ac.uk	European Business Management School, Swansea University, Singleton Park, SWANSEA SA2 8PP
Webmaster	Ray Cannon 01252-874697 ray_cannon@compuserve.com	21 Woodbridge Road, Blackwater, Camberley, Surrey GU17 0BS
Activities	Jon Sandles 01904-612882 jon_sandles@csi.com	138 Burton Stone Lane, YORK YO30 6DF
School Project	Stephen Taylor sjt@lambenttechnology.com	
Administration	Rowena Small 01223-570850 treas.apl@bcs.org.uk	12 Cambridge Road, Waterbeach, Cambridge CB5 9NJ

Journal Working Group

Editor:	Stefano Lanzavecchia	see above
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Advertising:	Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Support Team:	Jonathan Barman (01488-648575), Anthony & Sylvia Camacho (0117-973-0036), Ray Cannon (01252-874697), Stephen Taylor (077-1340-0852) Bob Hoekstra (01483-771028), Jon Sandles (01904-612882), Marc Griffiths	

Typeset by APL-385 with MS Word for Windows

Printed in England by Short-Run Press Ltd, Exeter

VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for "A Programming Language" – an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on most mainframes, workstations and personal computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

Causeway Graphical Systems Ltd
The Maltings, Castlegate,
MALTON, North Yorks YO17 7DP
Tel: 01653-696760
Fax: 01653-697719
Email: sales@causeway.co.uk
Web: www.causeway.co.uk

Dyadic Systems Ltd
Riverside View, Basing Road,
Old Basing, BASINGSTOKE,
Hants, RG24 0AL
Tel: 01256-811125
Fax: 01256-811130
Email: sales@dyadic.com
Web: www.dyadic.com

Insight Systems ApS
Nordre Strandvej 119G
DK-3150 Hellebæk
Denmark
Tel: +45 70 26 13 26
Fax: +45 70 26 13 25
Email: info@insight.dk
Web: www.insight.dk

MicroAPL Ltd
The Roller Mill, Mill Lane
Uckfield
E Sussex TN22 5AA
Tel: 01825-768050 Fax: 01825-749472
Email: MicroAPL@microapl.demon.co.uk
Web: www.microapl.co.uk/apl

Compass Ltd
Compass House
60 Priestley Road
GUILDFORD, Surrey GU2 5YU
Tel: 01483-514500

APL2000 Inc
One Research Court
Suite 140
Rockville MD 20850
USA
Tel: +1 (301) 208 7150
Email: sales@apl2000.com
Web: www.apl2000.com

HMW Computing
Hamilton House,
1 Temple Avenue,
LONDON EC4Y 0HA
Tel: 0870-1010-469
Email: HMW@4xtra.com

Soliton Associates Ltd
Groot Blankenberg 53
1082 AC Amsterdam
Netherlands
Tel: +31 20 646 4475
Fax: +31 20 644 1206
Email: sales@soliton.com